

Hierarchical Sequential Event Prediction and Translation from Aviation Accident

Report Data

by

Xinyu Zhao

A Dissertation Presented in Partial Fulfillment
of the Requirements for the Degree
Doctor of Philosophy

Approved June 2022 by the
Graduate Supervisory Committee:

Hao Yan, Chair
Yongming Liu
Feng Ju
Ashif Iquebal

ARIZONA STATE UNIVERSITY

August 2022

ABSTRACT

Sequential event prediction or sequential pattern mining is a well-studied topic in the literature. There are a lot of real-world scenarios where the data is released sequentially. People believe that there exist repetitive patterns of event sequences so that the future events can be predicted. For example, many companies build their recommender system to predict the next possible product for the users according to their purchase history. The healthcare system discovers the relationships among patients' sequential symptoms to mitigate the adverse effect of a treatment (drugs or surgery). Modern engineering systems like aviation/distributed computing/energy systems diagnosed failure event logs and took prompt actions to avoid disaster when a similar failure pattern occurs. In this dissertation, I specifically focus on building a scalable algorithm for event prediction and extraction in the aviation domain. Understanding the accident event is always the major concern of the safety issue in the aviation system. A flight accident is often caused by a sequence of failure events. Accurate modeling of the failure event sequence and how it leads to the final accident is important for aviation safety. This work aims to study the relationship of the failure event sequence and evaluate the risk of the final accident according to these failure events. There are three major challenges I am trying to deal with. (1) Modeling Sequential Events with Hierarchical Structure: I aim to improve the prediction accuracy by taking advantage of the multi-level or hierarchical representation of these rare events. Specifically, I proposed to build a sequential Encoder-Decoder framework with a hierarchical embedding representation of the events. (2) Lack of high-quality and consistent event log data: In order to acquire more accurate event data from aviation accident reports, I convert the problem into a multi-label classification. An attention-based Bidirectional Encoder Representations from Transformers model is developed to achieve good performance and interpretability. (3) Ontology-

based event extraction: In order to extract detailed events, I proposed to solve the problem as a hierarchical classification task. I improve the model performance by incorporating event ontology. By solving these three challenges, I provide a framework to extract events from narrative reports and estimate the risk level of aviation accidents through event sequence modeling.

ACKNOWLEDGMENTS

This dissertation would not have been possible without the contributions of many people. I would like to express my sincere appreciation towards my advisor Dr. Hao Yan, for his patient guidance and support. I am grateful that I learned so much from him and had the opportunity to work with him on those interesting research projects. I also want to express my deepest gratitude to all the committee members: Dr. Yongming Liu, Dr. Ashif Iquebal and Dr. Feng Ju, for their valuable feedback on my dissertation.

This research was supported by funds from NASA University Leadership Initiative program. I sincerely appreciate Dr. Anupa Bajwa and Dr. Yongming Liu for organizing this program which provided me opportunities to solve important problems in the aviation domain.

I would also like to thank all my great colleagues for their support and encouragement, including Yutian Pang, Yunyin Kang, Dorukhan Sergin, Zhiyang Zheng, Jiajing Huang, Menghan Liu, Jiayu Huang, Jiuyun Hu, Yanzhe Xu, Feifan Wang, Xiushuang Li, Xufeng Yao, Fei Gao, Zuwei Guo, Boyu Lin, Fengchen Wang. Thank you for making this journey so memorable.

Lastly, I owe the greatest debt of gratitude to my family. I want to thank my parents, Yansheng Wang and Xilong Zhao, for their love and support. They are my best audience and my best friends. I also want to thank my fiancée, Chan Wang, for encouraging me to achieve my goals.

TABLE OF CONTENTS

	Page
LIST OF TABLES	vii
LIST OF FIGURES	viii
CHAPTER	
1 INTRODUCTION	1
1.1 Background	1
1.2 Challenges and Expected Original Contribution	3
1.3 Dissertation Organization	6
2 AVIATION FAILURE EVENT PREDICTION AND RISK QUANTIFI- CATION WITH HIERARCHICAL EMBEDDING	7
2.1 Introduction	7
2.2 Literature Review	10
2.2.1 Event Embedding and Prediction Methods	11
2.2.2 Data-driven Aviation Accident Analysis and Risk Analysis ..	14
2.3 NTSB Accident Report and Event Taxonomy	15
2.4 Proposed Tree-based Embedding Method for Event Prediction Meth- ods and Risk Quantification	18
2.4.1 Taxonomy for Aviation Events and Risk Quantification Prob- lem Formulation	18
2.4.2 Proposed Event Embedding Learning and Prediction with Hierarchical Encoder Decoder	19
2.4.3 Online Risk Updating	23
2.4.4 Implementation Details	26
2.5 Demonstrations and Results	27
2.5.1 Data Description	27

CHAPTER	Page
2.5.2	Encoder Embedding Coefficient Visualization 31
2.5.3	Decoder Embedding Coefficient Visualization 32
2.5.4	Event Prediction Accuracy 33
2.5.5	Risk Quantification 37
2.6	Conclusion 39
3	MULTI-LABEL TEXT CLASSIFICATION FOR AVIATION ACCIDENT REPORT 41
3.1	Introduction 41
3.2	Literature Review 46
3.2.1	Information Extraction for Narrative Reports 46
3.2.2	Multi-label Text Classification 49
3.3	Methodology 50
3.3.1	Review of Sequence Generation Model 51
3.3.2	Review of BERT Based Model 53
3.3.3	Proposed BERT Model for Aviation Event Extraction 55
3.4	Experiment setup 57
3.4.1	Data Description 58
3.4.2	Data Preprocessing 58
3.4.3	Implementation Details 60
3.4.4	Evaluation Metrics 60
3.4.5	Performance Evaluation 61
3.4.6	Sensitivity Analysis on the BERT Model 63
3.4.7	BERT Model Interpretation and Attention Visualization ... 67
3.5	Conclusion 69

CHAPTER	Page
4 HIERARCHICAL CLASSIFICATION FOR AVIATION ACCIDENT REPORTS	70
4.1 Introduction	70
4.2 Literature Review	76
4.2.1 Information Retrieval for Narrative Reports	77
4.2.2 Hierarchical Classification	78
4.3 Methodology	80
4.3.1 Review of Related Methodology	80
4.3.2 Proposed Model for Hierarchical Multi-label Classification ..	84
4.4 Experiment Setup	93
4.4.1 Data Description	93
4.4.2 Implementation Details	94
4.4.3 Evaluation Metrics	95
4.4.4 Models for Comparison	96
4.4.5 Performance Evaluation	97
4.4.6 Rare Category Identification	99
4.5 Conclusion	102
5 CONCLUSION.....	104
REFERENCES	106
APPENDIX	
A NTSB DATA.....	114

LIST OF TABLES

Table		Page
1.1	An Example of the Event Log from NTSB	2
2.1	Prediction Strategy	23
2.2	Example of Coded Failure Event Sequences	28
2.3	Prediction Accuracy	29
2.4	Risk Estimation Accuracy	38
3.1	Performance Comparison of Different Methods	62
3.2	Explore Influence of Attention on BERT Classifier	65
3.3	Exploration of Label Embedding	67
4.1	Global Comparison for Different Models with Micro-precision, Micro-recall and Micro-f1	98
4.2	Macro F1 for Different Models under Different Sample Size	99
4.3	Macro Precision for Different Models under Different Sample Size	100
4.4	Macro Recall for Different Models under Different Sample Size	100
4.5	Category-wise F1 Score Comparison under Few-shot Learning Setting ..	101
A.1	Codes for Occurrence Defined by NTSB	115

LIST OF FIGURES

Figure	Page
2.1 Hierarchical Structure of the Sequential Events	17
2.2 Model Architecture	21
2.3 Estimated Hierarchical Tree Structure	27
2.4 Bar Plot for Flight Phases	29
2.5 Transition Diagram in the Phase-level	30
2.6 Transition Diagram in the Occurrence-level	30
2.7 Encoder Embedding for Phase and Occurrence	32
2.8 Decoder Phase Embedding	34
2.9 Decoder Occurrence Embedding	35
2.10 Occurrence Prediction Accuracy under Different Phases	35
2.11 Left: Risk Quantification Case Study with X-axis as the Observed Events at Current Time, and Y-axis as the Risk Score. Right: Relia- bility Diagram for Risk Quantification	39
3.1 Aviation Accident Report to Event Sequence: Report 20001208X07734 from NTSB	43
3.2 Encoder Decoder with Attention Architecture	51
3.3 Self Attention Architecture	54
3.4 Different Architectures for Multi-label Classification with BERT	57
3.5 Bar Plot for Flight Phases	59
3.6 Precision, Recall, and F1 by Failure Events with BR-SVM.	62
3.7 Precision, Recall, F1 by Failure Events with Seq2seq-LSTM	63
3.8 Precision, Recall, F1 by Failure Events with BERT+Attention	63
3.9 Precision, Recall, F1 by Failure Events for BERT without Attention. ..	65

Figure	Page
3.10 Embedding Visualization with TSNE.....	66
3.11 Attention Visualization for Airframe Malfunction	68
4.1 Label Statistics for 100 Most Frequent Fine-level Labels in NTSB	73
4.2 An Example of Label Taxonomy Defined by NTSB.....	73
4.3 Aviation Accident Report to Event Labels	74
4.4 Overview of the Two Stage Hierarchical Attention.	86
4.5 Coarse Level Model Architecture.....	87
4.6 Connection Matrix Between Occurrence Labels and Subject Labels	94
4.7 An Example of Label Taxonomy Defined by NTSB.....	95
4.8 Model Comparison under Different Sample Size	99

Chapter 1

INTRODUCTION

1.1 Background

Sequential event prediction or sequential pattern mining is a well-studied topic in the literature. There are a lot of real-world scenarios where the data is released sequentially. People believe that there exist repetitive patterns of event sequences so that we can predict future events. For example, many companies build their own recommender system to predict the next possible product for the users according to their purchase history. The healthcare system discovers the relationships among patients' sequential symptoms to mitigate the adverse effect of a treatment (drugs or surgery). Modern engineering systems like aviation/distributed computing/energy systems diagnosed failure event logs and took prompt actions to avoid disaster when a similar failure pattern occurs.

Most of the applications mentioned above can only make predictions with simple representation. When it comes to the rare event sequence with a more complex representation, it may lose the prediction power. However, these complex hierarchical representations are very common in the following applications. 1) In the recommender system, we always see that the online store is organized as *category* $\rightarrow$ *subcategory*. Such a hierarchical structure is frequently applied in the recommendation algorithms. It has been shown that the recommendation performance would be improved through incorporating the hierarchical structure since the similarity in the hierarchical layer means similar properties in items and similar preferences in users Wang *et al.* (2015). 2) In the healthcare system, accurate prediction of patients' symptoms makes person-

alized healthcare possible, and it relies on the large volume of medical record. However, most patients only have a limited number of records McCormick *et al.* (2011), and some symptoms can be rare to find. To address this, the patients and symptoms can be grouped together for better prediction for patients with rare symptoms. 3) In the engineering systems, many maintenance or event logs exist to describe the repetitive failure event sequence to take prompt actions Sahoo *et al.* (2003). However, failure events tend to be unique and rare, which makes it hard to predict. Similarly, to tackle the sparsity issue of the failure events, a proper hierarchical structure can be built to describe the failure event so that the failure event can be modeled at different granular levels.

<i>Phase</i>	<i>Occurrence</i>	<i>Subject</i>
Cruise	Rotor failure/malfunction	Rotor drive system, tail rotor drive shaft
Cruise	Rotor failure/malfunction	Lubricant, grease
Descent - emergency	Forced landing	Autorotation
Emergency landing	Hard landing	Terrain condition

Table 1.1: An Example of the Event Log from NTSB

In this dissertation, we specifically focus on building a scalable algorithm for hierarchical event embedding and prediction in the aviation domain. Understanding the accident event is always the major concern of the safety issue in the aviation system. Typically, a final flight accident event is often caused by a sequence of

failure events. Accurate modeling of the failure event sequence and how it leads to the final accident is important for aviation safety. This work aims to study the relationship of the failure event sequence and evaluate the risk of the final accident according to these failure events. The aviation event log data is collected by the National Transportation Safety Board (NTSB) NTS (2008). From Table. 1.1, we can see the hierarchical structure of the accident events. "*Cruise*" denote the phase of flight. "*Rotor failure/malfunction*" further describes what happened during the cruise phase. "*Rotor drive system, tail rotor drive shaft*" introduces more details about the problem with the rotor. The table also describes a failure event sequence during an accident, which starts with "*Rotor failure/malfunction*" and ends with "*Hard landing*". Overall, the challenges of the accident event data will be discussed in the next chapter.

1.2 Challenges and Expected Original Contribution

Challenge I (Modeling Sequential Events with Hierarchical Structure)

Sequential event prediction is a well-studied area and has been widely used in proactive management, recommender systems, and healthcare. One major assumption of the existing sequential event prediction methods is that similar event sequence patterns in the historical record will repeat itself, enabling us to predict future events. However, in reality, the assumption becomes less convincing when we are trying to predict rare or unique sequences. Furthermore, the representation of the event may be complex with hierarchical structures. For example, the high-level representation is an abstract of multiple mid-level events and they may include multiple rare low-level events. Finally, a more complex temporal correlation may also exist between the sequential events.

Expected original contribution to address challenge I

We aim to solve this issue by taking advantage of the multi-level or hierarchical representation of these rare events. To take advantage of the multi-level event representation, we proposed to build a sequential Encoder-Decoder framework to predict the event sequences by taking advantage of the event hierarchical structure. More specifically, in the encoding layer, we built a hierarchical embedding representation for the events. In the decoding layer, we first predict the high-level events and the low-level events are generated according to a hierarchical graphical structure. We propose to link the encoding decoding layers with the temporal models for future event prediction. We further talked about applying the proposed model into the failure event prediction according to the aviation accident reports and have shown improved accuracy as well as great model interpretability.

Challenge II (Lack of high-quality and consistent event log data) The 2nd challenge lies in the automatic extraction of the sequential events in the NTSB aviation accident database. The NTSB aviation accident database recorded information about civil aviation accidents since 1982. The data are organized in a relational database where several tables jointly describe a specific accident. Finally, we end up with 61671 accidents in 62570 aircraft. Each accident is represented with a sequence of failure events which is very useful information for risk analysis. The current aviation industrial relies on experts to manually identify the event from the aviation narrative reports. Such labeling process is subjective and inconsistent. One issue within the NTSB recording criticized by researchers is that most of the narrative reports are short, and likely causes were not be fully identified. Another issue is that different groups of people label the NTSB report before 2008 and after 2008, introducing inconsistency for the labeling process. Thus, the NTSB dataset exists the inconsistency within the definition of the event among the dataset, which prevents researchers from making full use of the dataset.

Expected original contribution to address challenge II We convert the problem into a multi-label classification problem which enables us to extract the consistent events automatically. Furthermore, the developed algorithms are able to identify the important sentences through the attention mechanism. This work provides a comprehensive comparison from traditional information extraction methods like Binary Relevance Support Vector Machine to some novel deep learning algorithms like Seq2seq Learning and BERT. Through extensive analysis of BERT based framework, we discuss the benefits and challenges of adapting the pre-trained BERT model into the aviation domain.

Challenge III (Ontology based event extraction) In order to learn an efficient information extraction framework, we convert the problem into a supervised learning task. According to Table. 1.1, we can see the taxonomy defined for aviation failure events which includes *Phase*, *Occurrence* and *Subject*. In the previous work, we have built a multi-label classifier on the occurrence level due to its proper label size. However, in other cases, we need a more detailed description of the accident where we need to refer to the *Subject* level. Thus, it comes to us if we can also build the accurate classifier on *Subject* level. It is very challenging to directly apply our previous method to the Subject level since the label size increased a lot and the sample size for each *Subject* label is very limited. There are 58 *Occurrence* labels but 1432 *Subject* labels. Only 46 *Subject* labels have the sample size greater than 1000, and 241 labels have the sample size greater than 100.

Expected original contribution to address challenge III

Our ongoing work tries to frame the problem as a few-shot learning problem. To be more specific, we are trying to combine the label embedding technique and the attribute based learning framework to improve our multi-label classifier with a limited sample size.

1.3 Dissertation Organization

The dissertation is organized as follows. Chapter 2 discusses the event sequence modeling with a hierarchical structure based on the NTSB database. To achieve this, efficient event embedding learning and risk uncertainty quantification will also be discussed based on the developed hierarchical event embedding models. Chapter 3 discusses the extraction of the failure events from narrative reports with multi-label classification. To achieve this, the state-of-the-art Bidirectional Encoder Representations from Transformers (BERT) model will be combined with the event embedding techniques for more efficient learning. Finally, Chapter 4 combines the hierarchical event structure, discussed in Chapter 2 and the event extraction framework discussed in Chapter 3 for better rare event extraction and translation. We formulate it as a few-shot learning problem and extract the rare events more efficiently through label embedding and attribute-based learning.

Chapter 2

AVIATION FAILURE EVENT PREDICTION AND RISK QUANTIFICATION WITH HIERARCHICAL EMBEDDING

2.1 Introduction

The development of the next generation national air transportation system (NextGen) (FAA, 2013) is essential to improve the safety and efficiency of the air traffic operation. In complex aviation systems, a small error might initiate a sequence of undesirable events, which will lead to an accident. Thus, it is of critical importance to understand past events and how it contributes to future accident. One crucial task of NextGen is understanding the root cause of an accident event and predicting it before it happens. Such prediction can be used in the decision-support tools (DSTs) (Schuster and Ochieng, 2014) to inform the pilot about the risk of the accident and possible future failure events for better risk mitigation.

In the literature, people have developed many accident models to study the root cause of the accidents. The accident models typically break an accident into separate components, such as the failure events. More specifically, a typical accident is often the result of a sequence of failure events with complex temporal or casual relationships along the sequence.

Modern aviation systems tend to show a complex correlation between humans, institutions, mechanical and electrical components. Thus, it requires the accident model to consider the complex interactions of all failure events. The study of correlation can convey a more meaningful understanding of the accidents but requires a more complex framework that is hard to be generalized.

The availability of the past aviation accidents databases, NTSB (national transportation safety board) has collected more than eighty thousand detailed accident records beginning from 1982 (NTS, 2008). Moreover, these accident reports have been carefully labeled by the aviation engineers to break down the entire accident report into a sequence of many failure events. Overall, it will take tremendous effort to conduct a comprehensive analysis of all the accident report data considering the complex interactions of all accident report data, given the large number of unique accidents.

In this paper, we will focus on building an event prediction model from the event logs from historical aviation accidents and relate these events to the final aviation risk. In other words, the goal of the model is two-fold: 1) understand how the history of past events can be used to predict future events. 2) quantify the uncertainty of the event sequence and link the risk estimation with partial event observations to a specific time.

The problem of event prediction and risk quantification based on the aviation risk event data is very challenging due to the following four reasons: 1) *Large number of rare accidents*: there is often a large number of rare accident events in the dataset. Many accident events only happen a few time and representing these events in one-hot encoder results in a high-dimensional space. 2) *complex temporal relationships* between the sequence of events: For example, the instrument meteorological condition (i.e., refers to the bad weather condition) first coupled with the mechanical failure could lead to higher risk compared to mechanical failure only. However, the traditional risk model made a very simple relationship, which is not suitable to model the complex temporal dependency. 3) *hierarchical taxonomy structure of the events*: The general event prediction usually didn't consider incorporating the hierarchical structure. in a specific field. For example, according to the taxonomy defined in aviation accident

recordings, we can define an event according to a hierarchical structure under different granular levels. For example, an accident event in the aviation accident report can be defined into three levels: phase code, occurrence code, and subject code. For example, the phase code describes the current phase that the airplane is in, such as take-off, cruise, descent, landing, etc. The occurrence code describes major types of failure that may happen to an aircraft, including the '*lose of control*', '*air-frame malfunction*', '*in-flight collision*', etc. Under each occurrence code, such as '*in-flight collision*', there may be the subject codes describing the collision, including collision with water, terrain, or other objects. 4) *Risk Uncertainty Quantification*: Quantifying the risk of aircraft is challenging given only partially observed event sequence up to a certain time. Ideally, the model should give larger at the beginning of the event sequence and the uncertainty would decrease with more observed events. This specific hierarchical structure actually lets us make response to an event more efficient. Moreover, in our experiment, we show that the prediction accuracy can benefit a lot through incorporating the taxonomy information.

The availability of different massive aviation accident events has enabled the possibility of changing the conventional failure event prediction model into deep learning-based methods. Deep representation learning, draws significant interest and has achieved great success in natural language processing (Manning and Schutze, 1999; Collobert *et al.*, 2011), representation learning (Hong *et al.*, 2015), and generative learning (Goodfellow *et al.*, 2014). Compared to the traditional ML approaches, the Deep Representation Learning model has the following advantages to learn the event embedding: 1) Deep representation learning models has the flexibility to model the complex temporal information in the failure events; 2) Deep representation learning model has enough flexibility to take advantage of the large aviation report dataset.

In this paper, we propose to extend the traditional DL methods for the aviation

failure event prediction and risk quantification by the following three modules: 1) we propose a hierarchical encoder-decoder method and event embedding method to represent discrete events with the hierarchical tree structure with embedding vectors. The embedding technique is widely used in Natural Language Processing (Manning and Schutze, 1999). It enables us to reduce the high dimension issue with a large corpus of failure events. Since the event vectors are learned from the corpus, the vector is able to convey the temporal dependency. 2) We apply Long Short Term Memory (LSTM) to capture the complex temporal relationships along with the event sequences with the hierarchical Encoder-Decoder framework to consider the hierarchical decoder considering the taxonomy structure. 3) We propose a sampling-based risk uncertainty quantification method based on the learned embedding of the events, which quantifies the uncertainty of the risk events with only partially observed data.

The rest of the paper is organized as follows. Section 4.2 reviews the studies performed in the general field of event prediction and aviation accident analysis. Section 2.3 introduces the overall problem definition. Section 2.4 introduces the proposed method for sequential event prediction and online risk updating. Section 2.5 gives the result of applying the proposed methods into the NTSB accident report data. Finally, Section 2.6 concludes the paper and discusses the future work.

2.2 Literature Review

In this section, we will review related works in the literature. In particular, we will focus on the following two major research areas: 1) machine learning methods for event embedding learning and sequence prediction methods; 2) aviation accident event modeling and risk analysis.

2.2.1 Event Embedding and Prediction Methods

A lot of research has been devoted to event sequence prediction in the field of machine learning and data analysis. More specifically, we will review the following three areas: 1) Event sequence prediction; 2) Hierarchical event prediction; 3) Event embedding learning.

Event Sequence Prediction

The association-rule-based method has been proposed to predict the next event over time (Rudin *et al.*, 2011) and provide theoretical properties. Similarly, the network-based method (O'Madadhain *et al.*, 2005) is used to represent the events pairs as edges and predict the events based on temporal link prediction algorithm and node ranking. The recurrent neural network is used to combine the event order and the duration of the event to predict the next event data based on the data-driven Aviation Accident (Li *et al.*, 2017). However, all aforementioned methods do not consider the complex hierarchical structures of the events, which is common in many engineered systems.

Hierarchical event prediction

Modifications have been made to the traditional methods to consider the hierarchical structures of the events. Traditional association rule method is to find the most confident rule for $a \rightarrow b$ where $p(b|a) = \frac{\#(a \cup b)}{\#a}$. a is the observed sequences, and b is the next possible item. However, it is quite common that we may have very low support, which means the number of a is very small. In order to exclude those rules which have low support but high confidence, people usually define the minimum support to avoid those rules. But, under certain situations, those low support rules

might be critical. For example, in the healthcare system, a lot of patients' symptoms are pretty rare but are closely related to other symptoms. Similar to the failure event modeling, the casual chain of system failure is unique. To tackle data sparsity, people come up with several heuristic ways to deal with this issue, such as adjusting the confidence score by putting a penalty term K as $p(b|a) = \frac{\#(a \cup b)}{\#a + K}$ to improve the low support observations. Han and Fu further bring in the definition of multiple-level association rules and mention that the minimum support should be specified separately at different levels (Han and Fu, 1999). Kim et al. proposed to add a relation matrix among different levels of rules and showed that the algorithm performs better than single-level methods in a sparse environment (Kim and Kim, 2003). The above works show that the hierarchical structure of the model overcomes the data sparsity issue and can be leveraged for real-world applications. It can also reduce the computational complexity since we can remove the less frequent rules earlier at a higher level. However, there are two limitations that lie in association-rule-based methods. First, the confidence score is designed for simpler relationships. It can be misleading and tend to predict frequent events without considering the influence of previous events. Second, when the dataset is large, these algorithms are typically very computationally expensive.

Recently, several deep-learning-based frameworks have been developed to learn the hierarchical embedding for the events and then predict the future event based on the embedding representation. Hu et al. proposed a context-aware hierarchical Long Short Term Memory (LSTM) for the event prediction. The model encodes different levels of events using a two-level LSTM layer (Hu *et al.*, 2017). Different levels of the products have been incorporated into the personalized product search (Ai *et al.*, 2017). Overall, deep-learning-based frameworks have been shown to be powerful in capturing complex sequential relationships. In this work, we further show

that we can improve the performance of the aforementioned frameworks by adding the hierarchical tree structure on the decoding layer.

Event Embedding Representation

The idea of embedding representation comes from natural language processing by representing discrete words with continuous vectors according to their surrounding context (Rumelhart *et al.*, 1986). For example, word2vec techniques utilize the large corpus of word sequences and produce a vector space, typically with several hundred dimensions, to represent millions of words. Therefore, the embedding representation is capable of dealing with the dimensionality issue and improving the generalization behavior of the model (Bengio *et al.*, 2003). Following this idea, the embedding vectors for more complex semantic structures such as sentence and structured events have been studied (Hill *et al.*, 2016; Granroth-Wilding and Clark, 2016). The aforementioned embedding techniques are for general domain text. Here, we like to review several methods specifically designed for event embedding. A feature event embedding algorithm (Lee *et al.*, 2020) is proposed for statistical script learning. Furthermore, the idea of event embedding is proposed for the stock prediction (Ding *et al.*, 2016), crime rate data (Zhu and Xie, 2019), and social event (Du *et al.*, 2016). However, these embedding algorithms typically do not consider the hierarchical structure of the events. Our previous workshop paper on utilizing the hierarchical structure has shown effectiveness for aviation event prediction (Zhao *et al.*, 2021). However, there is no clear indication on how such event embedding can be used to predict the risk and its uncertainty. This paper proposes a unified event embedding, event prediction, and risk quantification methodology and also discuss how such prediction uncertainty can be further calibrated.

2.2.2 Data-driven Aviation Accident Analysis and Risk Analysis

In this subsection, we will briefly review the data-driven aviation accident analysis methods. More specifically, we will focus on the aviation accident analysis and aviation risk analysis methods.

Aviation Accident Analysis

With the emergence of event data collected from aviation accident reports, we are able to build a more meaningful analysis to help improve aviation safety. The aviation reports from aviation incident database such as ASRS (Aviation Safety Reporting System), NTSB typically contains raw narrative data and some summarized information such as weather conditions, aircraft type and failure events. The failure events are the key events that influence the result of an incident and are coded according to different taxonomy in the incident reporting system. Many works have been done for analyzing the event data from accident reports. Given the defined events, it has been identified the high-risk failure events sequence for helicopter accidents through a frequentist approach considering the level of injury and damage of each accident (Rao and Marais, 2015). Instead of using a simple sequential model, other works applied more complex techniques such as Neural Networks (Desell *et al.*, 2014) and Bayesian Network (Zhang and Mahadevan, 2020) to analyze the aviation accident. Another work similar to ours considered representing each accident using continuous vectors. It applied Long Short-term Memory to build a prediction model to decide the severity of the accident, given the embedding of the accident (Srinivasan *et al.*, 2019). However, most of these works does not consider the hierarchical structure of the events, therefore cannot be used for

Aviation Risk analysis

The modern socio-technical system is built on precise cooperation among multiple components. But the unexpected accident still occurs. Every small failure can lead to a disaster. The research on studying the accident helps us better understand the cause of an accident so that we can avoid similar things happening in the future. Due to the complexity of large systems, the accident is generally considered as a combination of various smaller failures (Perrow, 1984; Reason, 1997). After identifying the important failure components, an accident model is needed, which is used to describe the complex casual relationships. Traditional causation models such as Failure Modes and Effects Analysis and Fault Tree Analysis assumed the failure events have a linear relationship following the temporal order (Gilchrist, 1993; Ericson and Ll, 1999). However, since the sequential model lacks the ability to capture complex correlation, some modern accident models are developed based on system theory (Leveson, 2004), Bayesian Networks (Hänninen, 2014), Neural Networks (Rezaie Moghaddam *et al.*, 2011).

2.3 NTSB Accident Report and Event Taxonomy

We will use the definition from the NTSB coding system in this article (NTS, 2008). The aviation accident is represented with a sequence of failure events. Each failure event comprises three different level of representations.

- **Phase codes:** All occurrences include information about the phase of flight in which the occurrence took place. Phase of flight refers to the point in the aircraft operation profile in which the event occurred.
- **Occurrence codes:** Multiple occurrences can be used to describe a mishap event. Each occurrence is coded separately, and a corresponding phase of operation is entered with each occurrence.

- **Subject codes:** Sequence of events subject codes are used to identify the individuals, equipment, processes, or phenomena that contributed to the mishap event. Subject codes are used in conjunction with the modifier codes, person codes, and the cause/factor designator to provide the details of the mishap event and the probable cause. Each subject code references a specific event occurrence, and may include multiple codes. When more than one code is used per occurrence, codes are assigned a sequential *seq_event_no* (in *seq_events* table) to distinguish between multiple codes.

The highest level is the phase of the flight. There are 51 identified phases and the five phases with most accidents are '*Landing – roll*', '*Landing – flare/touchdown*', '*Descent – uncontrolled*', '*Descent – emergency*' and '*Takeoff – initial climb*'. Under the phase level, there are 55 occurrences further defined to describe the specific events. The five occurrences with most accidents are '*In flight collision with terrain*', '*Forced landing*', '*Loss of control – in flight*', '*In flight collision with object*' and '*Loss of control – on ground/water*'. Subjects are used to provide more details about the occurrences like the identified individuals, equipment, processes or phenomena. There are 1598 defined subjects and the subjects with most accidents are '*Terrain condition*', '*Weather condition*', '*Object*', '*Directional control*' and '*Fluid, fuel*'.

Fig. 2.1 shows an illustration of how different levels related to each other via an accident that happened on April 2, 1997. Through the detailed report from the NTSB, we are able to realize what happened during the accident. At 19:56, the pilot reported to Los Angeles Air Route Traffic Control Center (ARTCC) controller that he lost his alternator, and he was **switching to his standby generator**. At 19:58, the pilot called requesting a lower altitude and stated that he was **in the clouds without any lighting**. At 20:02, he added that his gauges were broken.

From 20:02 to 20:09, the controller is trying to assist the pilot to find the nearest airport, and the pilot **reported various problems with his flight instruments, including gauges, compass, altimeter, and vacuum pump**. At 20:10, the pilot replied to the controller that his bank indicator, DG, and HSI were providing conflict information. At 20:14, the pilot said that he was going to **lose control of the flight** because of the conflict information from two gauges as well as being unable to stay on a compass heading. The last recorded transmission from the pilot is around 20:15:37 and the report shows that the airplane was **collided with snow-covered mountainous terrain** in a wilderness area on the north rim of the Grand Canyon in Arizona at 20:16. The airplane was destroyed. The pilot and the passengers received fatal injuries. We will use this example for the online risk quantification in Section 2.5.

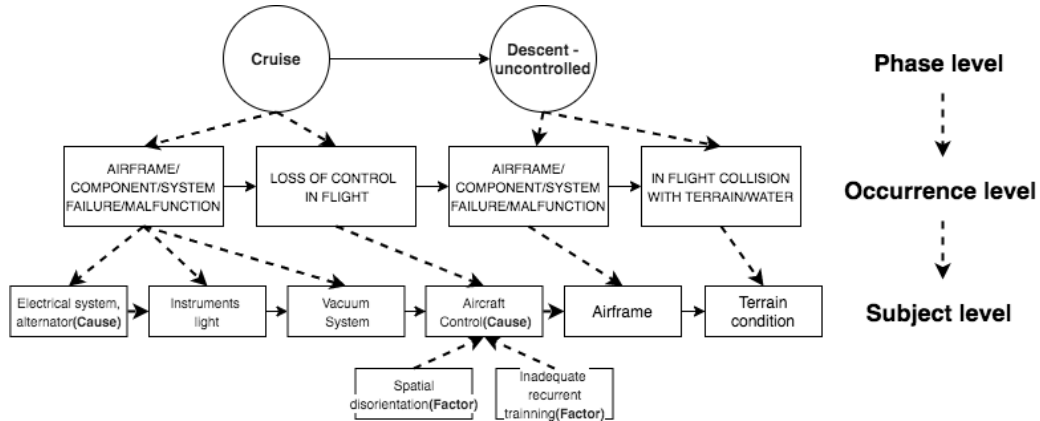


Figure 2.1: Hierarchical Structure of the Sequential Events

Here, we can see that the events are represented in different levels with embedding vectors and there are three major consideration in this problem:

- The event embedding representation of each failure event should consider the temporal relationship according to the accident recordings.

- The event embedding should consider the hierarchical relationship of the event taxonomy information.
- The learned event embedding will be used for future tasks, such as the risk prediction.

2.4 Proposed Tree-based Embedding Method for Event Prediction Methods and Risk Quantification

In this section, we will develop the methodology for modeling the sequential aviation events by a tree-based embedding method. We will start with the problem formulation and taxonomy for the aviation events in Section 2.4.1. Then, we will discuss the proposed tree-based hierarchical event embedding and sequential event prediction in Section 2.4.2. Finally, we propose an efficient online risk updating and uncertainty quantification in Section 2.4.3.

2.4.1 Taxonomy for Aviation Events and Risk Quantification Problem Formulation

We first would like to define our problem mathematically. Given the hierarchical structure of the event representation, we use a tuple to represent each event as a tuple $e^t = (p^t, o^t, s^t)$, Phase, Occurrence, Subject from corresponding corpus P, O, S . Some important phases of the aircraft include take-off, cruise, descent, landing, etc. More specifically, we denote the Given each phase, there may be different types of event occurrences, including the loss-of-control, collision, etc. Finally, the subject-level information is a more detailed description of the occurrence, such as the collision types. However, given that p^t, o^t, s^t are the one-hot encoding representation of the events. The proposed algorithm can be also generalized to event with more layers of hierarchical representation, such as $(e_1^t, e_2^t, \dots, e_h^t)$, where the e_1^t is the top-level event

category, e_i^t is the i^{th} -level event category. The goal is to predict the next event e^{t+1} given the entire event history $\{e^t\}_{t=1,\dots,T}$.

The first goal of the proposed framework is to predict the next event e^{t+1} given the entire history of event $\{e'_t; t' = 1, \dots, t\}$ considering the event hierarchical representation $e^t = (e_1^t, e_2^t, \dots, e_h^t)$.

The second goal is to predict and quantify the risk r_t based on the partially-observed event sequences $\{e^1, \dots, e^t\}$. In this paper, we propose a sampling-based risk estimation and uncertainty quantification method that takes learned event embedding representation to predict the risk score and its uncertainty. In other words, the goal is to estimate conditional distribution $P(r_t|\{e^1, \dots, e^t\})$.

2.4.2 Proposed Event Embedding Learning and Prediction with Hierarchical Encoder Decoder

In this section, we discuss our method for event embedding and prediction. Moreover, we aim to learn the embedding representation of the failure event sequence consisting these three levels of hierarchy. Here the proposed method would consist three components: 1) **the hierarchical embedding network** $P(z_t|e_t; \theta_e)$ with parameter θ_e , which focuses on compressing the high-dimensional event representation e_t into a lower-dimensional feature vector z_t . 2) **the dynamic transition model** $h_{t+1} = f(h_t, z_t; \theta_{tr})$ with parameter θ_{tr} , which will be used to model the dynamic transition of such latent lower-dimensional feature vector over time. 3) **the hierarchical tree-based decoder** $P(e_{t+1}|h_{t+1}; \theta_d)$ with parameter θ_d , which use the tree structure of the events hierarchy to predict the future event e_{t+1} given the feature vector h_{t+1} .

Fig. 2.2 shows the overall architecture that combines the hierarchical embedding network, sequential transition model, and hierarchical tree-based decoder and how

they can work together to predict the future event e^{t+1} from the event history $\{e^{t'}\}_{t' < t}$.

Finally, these three components will be trained in an end-to-end manner by maximizing the overall likelihood function $P(\{e^t\}_{t=1}^T; \theta)$, where $\theta = \{\theta_e, \theta_{tr}, \theta_d\}$ are all the model parameters. By decoupling the joint likelihood, we can have that

$$P(\{e^t\}_{t=1}^T; \theta) = P(e_1; \theta) P(e_2|e_1; \theta) \cdots P(e_T|\{e^t\}_{t=1}^{T-1}; \theta) = \prod_t P(e_t|\{e^{t'}\}_{t'=1}^t; \theta)$$

Assuming that h_t represented the information collected through sequence $\{e^k\}_{k=1}^t$ so that it can be used to represent the decoder only based on the encoder

$$P(e_t|\{e^{t'}\}_{t'=1}^t; \theta) = P(e_{t+1}|h_{t+1}; \theta_d)$$

Hierarchical Embedding Network

For event at time t , we first project (p^t, o^t, s^t) to their corresponding embedding layer with parameters $U \in \mathbb{R}^{n_c \times |P|}$, $V \in \mathbb{R}^{n_c \times |O|}$, $W \in \mathbb{R}^{n_c \times |S|}$ as $c_p^t = \sigma(U p^t)$, $c_o^t = \sigma(V o^t)$, $c_s^t = \sigma(W s^t)$, where $|P|$ is the number of phases, $|O|$ is the total number of occurrences, $|S|$ is the total number of subject code, and n_c is the size of the en. Furthermore, these three embedding are concatenated into $c_t = [c_p^t \ c_o^t \ c_s^t]$. We further add a linear layer to consider the complex correlation between and within different levels. An activation layer is added at the end to incorporate the non-linearity with $z_t = \sigma(H c_t)$. Here, we can use the ReLU layer for the activation function $\sigma(\cdot)$.

Sequential Model

To model the transition of event history, we will use consider the following two different sequential model, Markovian model and LSTM model.

- Markovian Model: Here, we assume that the future event e_{t+1} only depends on the encoded feature z_t , where $P(e_{t+1}|z_t; \theta_{tr})$. Moreover, we will model the events such as the phase, occurrence, and subject given z_t separately.

- Long Short Term Memory: LSTM is a kind of Recurrent Neural Network which is also able to capture the temporal dependency through a recurrent architecture. Comparing to the standard RNN, LSTM is able to deal with long term dependency. In general, LSTM deals with the long term issue by adding a cell state and the gating technique. The cell state keeps all the important information from past events and the gate decides which information is important.

$$c_{t+1}, h_{t+1} = f_{lstm}(c_t, z_t; \theta_{tr}),$$

where f_{lstm} is the standard LSTM model. c_t is the combination of the cell state and memory state in the LSTM model. h_t is the output of the LSTM model at time t and will be used in the decoder.

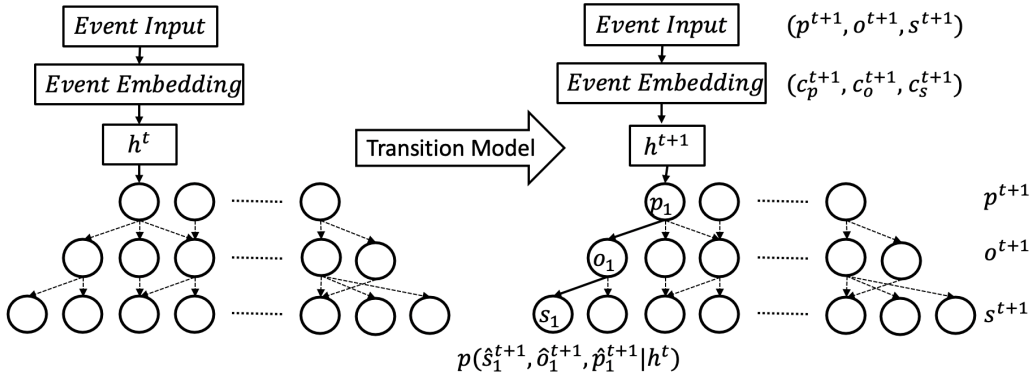


Figure 2.2: Model Architecture

Decoding Network

After getting the hidden state from previous events, we need to devise a decoding function to predict the failure event in next stage. Notice that our failure event is composed of phase, occurrence and subject according to the taxonomy. Thus, we

propose a hierarchical tree-based decoding function

$$P(p_i^{t+1}, o_j^{t+1}, s_k^{t+1} | h_t; \theta_d) = P(p_i^{t+1} | h_t; \theta_d) P(o_j^{t+1} | p_i^{t+1}, h_t; \theta_d) P(s_k^{t+1} | o_j^{t+1}, h_t; \theta_d). \quad (2.1)$$

Each term can be calculated through the Softmax function using hidden state and decoder event embedding. Here, $\theta_d = \{U', \{V'_i\}_{i=1, \dots, |P|}, \{W'_j\}_{j=1, \dots, |O|}\}$ is the decoder parameter (i.e., also known as the decoder embedding parameter), which includes the decoder parameters for phase $U' \in \mathbb{R}^{n' \times n_p}$, decoder parameters for occurrence under phase i as $\{V'_i\}_{i=1, \dots, |P|} \in \mathbb{R}^{n' \times n_{po,i}}$, and decoder parameters for subject under occurrence j as $\{W'_j\}_{j=1, \dots, |O|} \in \mathbb{R}^{n' \times n_{os,j}}$. Here we use n' to represent the dimension of the decoder vector and $n_p, n_{po,i}, n_{os,j}$ to denote the number of phases, the number of occurrences under phase i , and the number of subject codes under subject j , respectively. p_i^t, o_j^t and s_k^t denote the phase, occurrence, and subject of flight at time t is the i -th, j -th, and k -th item in phase corpus, respectively. Since we need to compute the conditional probability across different levels, we consider to build different output vectors for same occurrence under different phases of flight. The intuition behind it is that under different phases, the same occurrence will have different impact on the aircraft and lead to different event sequences. Thus, we need to build a "conditional embedding vector" Here we use $u'_i, v'_{i,j}, w'_{j,k}$ to represent the i -th column in the decoder parameters of U' , j -th column in V'_i and , k -th column in W_j .

$$\begin{aligned} P(p_i^{t+1} | h_t) &= \frac{\exp u'_i{}^T h_t}{\sum_i \exp u'_i{}^T h_t}, \\ P(o_j^{t+1} | p_i^{t+1}, h_t) &= \frac{\exp v'_{i,j}{}^T h_t}{\sum_j \exp v'_{i,j}{}^T h_t}, \\ P(s_k^{t+1} | o_j^{t+1}, h_t) &= \frac{\exp w'_{j,k}{}^T h_t}{\sum_k \exp w'_{j,k}{}^T h_t} \end{aligned}$$

Finally, the decoder parameter θ_d will be learned from the data during the end-to-end training.

Event Prediction

Here, we would like to discuss how the proposed method can be used to predict the event at the next time. Specifically, We would discuss on how to predict the future events if the higher level information is given. For example, if we do not know the phase and occurrence. The algorithm will need calculate the prediction by the summation of all possible phase p_i and occurrence o_j when doing the subject-level prediction as $\sum_{i,j} P(s_k^t|p_i, o_j, h_{t-1})P(o_j|p_i, h_{t-1})P(p_i|h_{t-1})$. However, if the phase-level information is known as p_i^t , we can use this information in the prediction of the subject level so that the summation over the phase is not needed as $\sum_j P(s_k^t|p_i^t, o_j, h_{t-1})P(o_j|p_i^t, h_{t-1})$ so that the prediction can be more accurate. Here, we would like to give the prediction algorithms given different scenarios, depending on whether phase and occurrence information are given.

Table 2.1: Prediction Strategy

	Phase Prediction	Occurrence Prediction	Event Prediction
Unknown Phase	$P(p_i^t h_{t-1})$	$\sum_i P(o_j^t p_i, h_{t-1})P(p_i h_{t-1})$	$\sum_{i,j} P(s_k^t p_i, o_j, h_{t-1})P(o_j p_i, h_{t-1})P(p_i h_{t-1})$
Known Phase	-	$P(o_j^t p_i^t, h_{t-1})$	$\sum_j P(s_k^t p_i^t, o_j, h_{t-1})P(o_j p_i^t, h_{t-1})$
Known Occurrence	-	-	$P(s_k^t p_i^t, o_j^t, h_{t-1})$

2.4.3 Online Risk Updating

In this section, we will discuss how to use the existing event prediction framework for online risk updating. The proposed risk quantification framework include two components. First, an uncertainty calibration method is proposed to calibrate the

uncertainty of the event prediction in Section 2.4.3. Second, we propose a sampling-based method for online risk quantification based on the learned embedding in Section 2.4.3.

Uncertainty Calibration of Event Prediction

Neural Network based methods are often criticized to be overconfident on certain cases which is not suitable for safety-related problems. Since most of accident events don't happen very frequently, NN prediction results prefer those frequent events which will lead to bad results for our downstream task, event risk quantification. Thus, we consider to calibrate the event prediction results using temperature scaling where we can adjust parameter T in the final Softmax layer to get a softer distribution over all events.

$$\begin{aligned} P(p_i^{t+1}|h_t) &= \frac{\exp(u_i'^T h_t/T)}{\sum_i \exp(u_i'^T h_t/T)}, \\ P(o_j^{t+1}|p_i^{t+1}, h_t) &= \frac{\exp(v_{i,j}'^T h_t/T)}{\sum_j \exp(v_{i,j}'^T h_t/T)}, \\ P(s_k^{t+1}|o_j^{t+1}, h_t) &= \frac{\exp(w_{j,k}'^T h_t/T)}{\sum_k \exp(w_{j,k}'^T h_t/T)} \end{aligned}$$

It worth noting that T will not change the largest value of the Softmax layer, but only adjust the prediction dispersion. When $T \rightarrow 0$, the prediction will collapse to the largest value of the Softmax distribution and when $T \rightarrow +\infty$, the distribution will become a uniform distribution. Typically, T is selected by the reliability diagram, which plots of the relative frequency of an event and against the forecasting probability of an event. Overall, if the relative frequency match the probabilistic of the event in a positive 45 degree line, the uncertainty of the probabilistic classifier for event prediction is perfectly calibrated. Here, calibration of the distribution is important because we do not only care about the mode of the distribution, but we aim to

use this perfectly calibrated prediction model to sample the full event sequences to quantify the risk uncertainty due to the event path uncertainty, as shown in Section 2.4.3.

Sampling-based Risk Modeling

To better quantify the uncertainty of the event path leading to the final accident, we propose a sampling-based approach to first sample the full path of the event sequences and then use the sampled path of events to estimate the risk level of the event sequence based on the learned event embedding.

1. **Path Sampling:** At each time step t_n , assume that we have observed event sequences $\{e^t\}_{t=1,\dots,n}$, we will sample possible future event sequences based on the event prediction model to get $\{e^t\}_{t=1,\dots,N}$. To achieve this, we will start with sampling event $e_{t'}$ from the calibrated distribution of $P(e^{t'+1}|e^{t'})$, obtained from the hierarchical event prediction model, proposed in Section 2.4.2, recursively for $t' = n, \dots, N$ until a terminating state is obtained.
2. **Risk Learning from Event Embedding:** For each sampled path, we further project each event into its embedding representation $\{\mathbf{e}^{i,t}\}_{t=1,\dots,N_i}$ according to the proposed tree-based embedding. In this work, we propose to utilize a simple average pooling layer to aggregate all events embedding in each accident for simplicity. After the average pooling layer, we finally obtain a list of accident embedding $\mathbf{e}^i$, which will be served for the training purpose. We then propose to train a supervised learning model to link the accident embedding $\mathbf{e}^i$ and the risk of the event r_i as $r_i = g(\mathbf{e}^i)$.

Here, the risk for the aircraft is quantified by three classes, namely the "Destroyed", "Substantial" and "Minor/None". Here, the supervised learning model $g(\cdot)$

can either be a probabilistic classifier or a deterministic classifier. Here, we have tested the performance of different supervised learning classifiers and find that different supervised learning models lead a similar prediction accuracy. In this paper, we use a random forest model as the final supervised learning model.

2.4.4 Implementation Details

Here, we would like to discuss the implementation details for the proposed algorithm. The algorithm can be derived into the training and testing phase. Furthermore, we use 50045 data for training and 12511 data for testing.

During the training phase, the events $e^t = (p^t, o^t, s^t)$ for $t = 1, \dots, T$ are fully observed. In this case, we can use the likelihood function $P(\{e^t\}_{t=1}^T; \theta) = \prod_t P(e^t|h_t; \theta)$. For each $P(e^t|h_t; \theta) = P(p^t|h_t; \theta)P(o^t|p^t, h_t; \theta)P(s^t|o^t, h_t; \theta)$. Here, the conditional probability $P(o^t|p^t, h_t; \theta)$ and $P(s^t|o^t, h_t; \theta)$ requires the hierarchical tree-structured of the events being estimated from the training data. If any of the training data has transitioned between the two events, the edge of the tree will be connected. Here, we would like to visualize the estimated tree structure of the phase level, occurrence level, and subject level in Fig. 2.3. From Fig. 2.3, we can conclude that the tree pattern of failure event of the aviation system is actually quite complicated. For example, the same occurrences can actually happen in different phases, which result in a complicated network structure.

Here, in the training phase, we apply AdamW as the optimizer and switch to SGD to ensure a stable learning curve. We select the same learning rate as 0.001. The algorithm is implemented based on Pytorch 1.4.0 on a workstation with Intel(R) Core(TM) i7-5930K CPU @3.50 GHz CPU and accelerated with a single NVIDIA GTX 1080 Ti GPU.

During the testing phase, given the future $e^t = (p^t, o^t, s^t)$ is often unknown, we

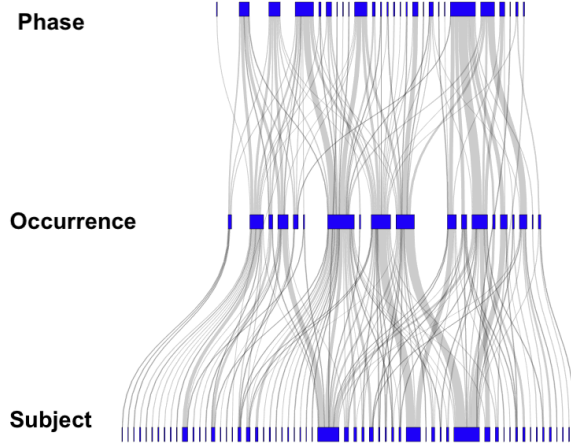


Figure 2.3: Estimated Hierarchical Tree Structure

need to do the summation of all possible o_t to estimate the subject event s_t . One distinct advantage of the proposed hierarchical model is the ability to use high-level information, as shown in Table 2.1. More specifically, if the future information of Phase, Occurrence, the proposed method is able to take advantage of such information to give a better prediction in the subject-level.

2.5 Demonstrations and Results

In this section, we will first introduce the NTSB data in Section 2.5.1. Then, the embedding of the failure events learned from the proposed model will be discussed through low dimension visualization in Section 2.5.2. We showed a comparison of several methods to illustrate the effectiveness of the proposed event prediction model in Section 2.5.3.

2.5.1 Data Description

In this section, we will introduce the dataset and our preliminary analysis. The NTSB aviation accident database recorded information about civil aviation accidents

since 1982. The data are organized in a relational database, where several tables jointly describe a specific event. During the data processing, we find the table of interest and merge those tables based on the primary key '*ev_id*'. Finally, we end up with 61671 accidents, and 62570 aircraft are recorded in those accidents. Each accident is represented with a sequence of failure events, as described in Section 3. Table.2.2 shows two examples of the event sequences. The meaning of each failure event is coded for convenience according to the NTSB coding manual. In our study, we represent the failure events from different levels using continuous vectors. We then compare the predictive performance of discrete representation with several different embedding techniques.

Table 2.2: Example of Coded Failure Event Sequences

ev_id	Failure Event Sequences	
20070403X00368	Phase	540 → 540 → 553 → 553
	Occurrence	240 → 240 → 250 → 220
	Subject	20000 → 24015 → 24566 → 20200
20040427X00519	Phase	572 → 572 → 572
	Occurrence	260 → 260 → 300
	Subject	24539 → 34343 → 24500

Fig. 3.5 shows the distribution of the selected occurrences in selected flight phases. From this figure, we can see that some of the common occurrences can happen in several phases of the flight.

Furthermore, we would like to visualize the transition diagram in the Phase level and occurrence level. For visualization convenience, we filter the event chains according to their frequencies. We only kept transitions that occur more than 100

Table 2.3: Prediction Accuracy

Prediction Condition		One-hot-encoding +Markovian	Tree Embedding + Markovian	Word2Vec + LSTM	Tree Embedding + LSTM
Phase Unkown	Phase	0.6052	0.6092	0.5995	0.6287
	Occurrence	0.5283	0.5308	0.5478	0.5497
	Subject	0.3055	0.3063	0.3089	0.3377
Phase Known	Occurrence	0.7607	0.8252	0.7995	0.8297
	Subject	0.2086	0.3635	0.4196	0.4649

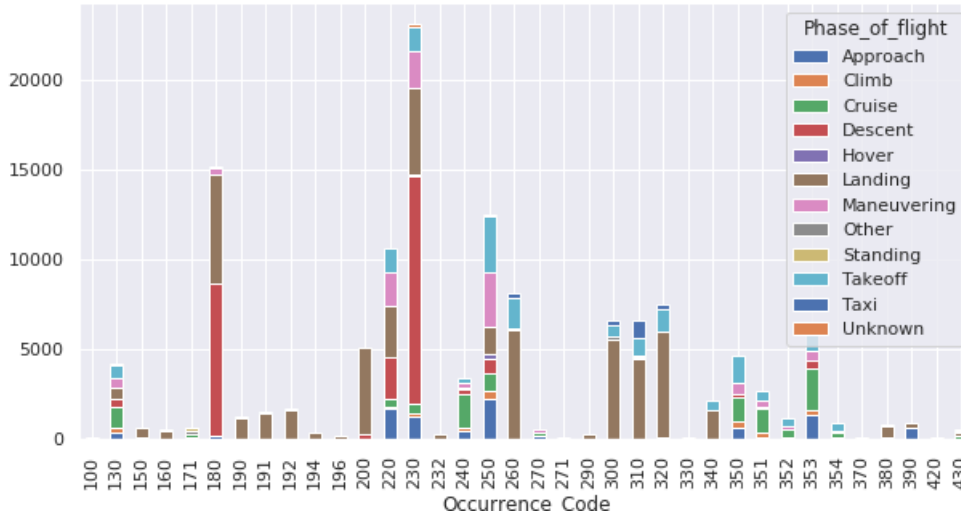


Figure 2.4: Bar Plot for Flight Phases

times and have transition probability over 0.3 for simplicity. Otherwise, the transition diagram would not be feasible for visualization purpose. *In the phase level*, the transition diagram of the most common phases is shown in Fig. 2.5, which shows the phase-level information of the event relationship. *In the occurrence level*, we would like to emphasize that the transition diagram is actually very different according to different phases. For example, in the landing phase, the absorbing state could be *forced landing*, *in-flight collision with terrain/water*, *nose over*, or *on-ground collision with object*. However, in the *take-off* phase, the absorbing state will become *nose*

over only. This shows the importance of considering the higher-level information to predict the transition.

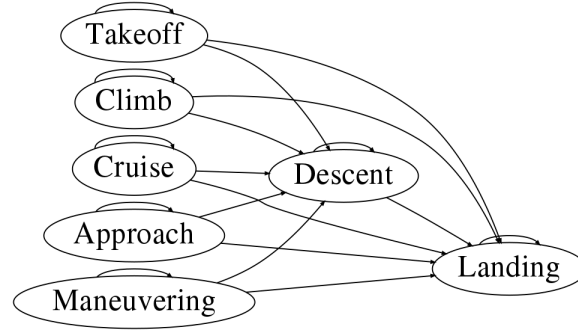
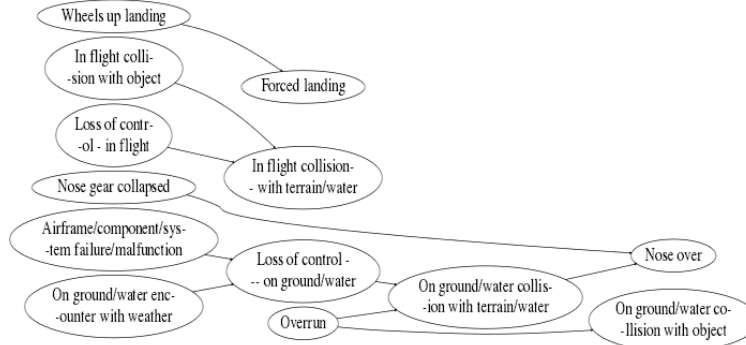
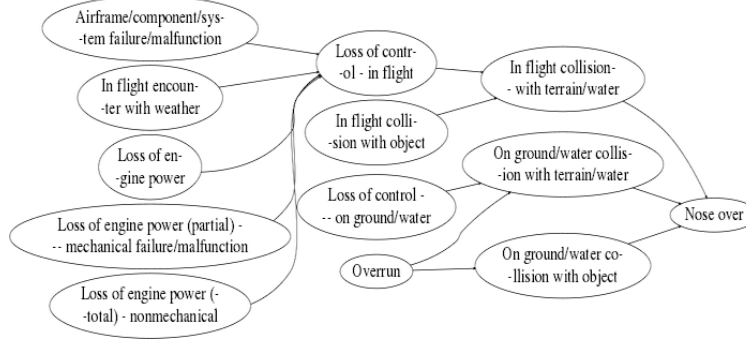


Figure 2.5: Transition Diagram in the Phase-level



(a) **Occurrence-level** Transition Under the *Landing Phase*



(b) **Occurrence-level** Transition Under the *Take-off Phase*

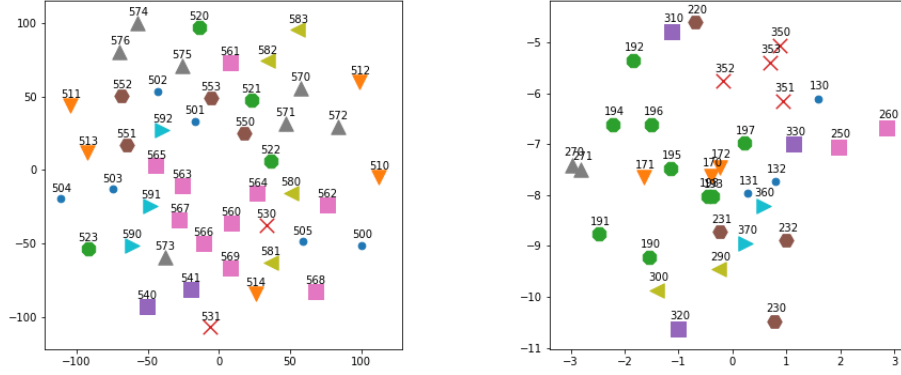
Figure 2.6: Transition Diagram in the Occurrence-level

2.5.2 Encoder Embedding Coefficient Visualization

In this section, we would like to visualize some illustrative results of different the encoder embedding layer coefficients U, V, W for different flight phases, occurrences, and subject codes. Given the embedded dimension is 30, we propose to project them into two dimensions for visualization. We select perplexity as 23 and each plot is trained with 10000 iterations at a learning rate 100. We represent a similar phase or occurrence events with the same node type (i.e., shape and color) in the plot.

Fig. 3.10 (a) shows the embedding **in the phase level**. We can see some clear clusters for phases like 560-568 (in the pink square), which denote *approach* phase. There are also some nodes with similar meanings, such as 570-576 (in the grey triangle), which means *landing* phase, but ends up dividing into two clusters. According to the definition of the codes we know that 574, 575, 576 mean *emergency landing phases* and 570, 571, 572 mean *normal landing phases*. That is the reason that these two parts are separated.

There are also clear clusters **in the occurrence level**, like 350-353 (in red cross mark), which denote *loss of engine power*. 270 and 271 denote *collision among aircrafts*. And 170-172 denote *fire explosion*. Through the embedding, we can also solve the redundancy issue that occurred in the original NTSB coding system. For example, both 220 and 310 denote *collision with objects*. From the embedding visualization, we can see that the embedding representation of 220 and 210 are indeed grouped together. Our results show that we are able to preserve the similarity of the events using the continuous representation. Furthermore, the continuous representation is typically much more efficient due to the reduced dimensionality.



(a) Phase Embedding, some clusters are given here: 570, 571, 572 mean *normal landing phases*. 574, 575, 576 mean *emergency landing phases*. 560-568 mean the *approach phase*.

(b) Occurrence Embedding, some clusters are given here, 350-353 mean the *loss of engine power*. 270-271 mean *collision among aircrafts*. 220 and 310 mean *collision with objects*

Figure 2.7: Encoder Embedding for Phase and Occurrence

2.5.3 Decoder Embedding Coefficient Visualization

In this section, we would like to visualize some illustrative results of different the encoder embedding layer coefficients $U', \{V'\}, \{W'\}$ for different flight phases, occurrences, and subject codes. One distinct difference from the decoder coefficient is designed based on the tree structure, as shown in Fig. 2.3. This implies that there is only one phase embedding matrix denoted as U' , but there can be multiple occurrence embedding matrix $\{V'_i\}_{i=1, \dots, |P|}$. For example, here, we like to show the embedding matrix for the decoder occurrence under the *take-off phase* and the *landing phase* in Fig. 2.9. Similarly, given the embedded dimension is 30, we propose to project them into two dimensions for visualization.

Fig. 2.8 shows the decoding phase embedding. Similar clusters to the encoding embedding can also be found here like 560-568. We can also find some cluster are

not obvious in the encoding embedding like 501-504 as the *standing* phase. 520-523 refer to the *takeoff* phase. In the decoder embedding, we can see that the *takeoff* phase is on the top right of the figure and the *landing* phase is at the bottom left of the figure, which is more meaningful to us since those two phases have very different transition diagrams. we further discover the conditional decoding embedding vector for occurrence. Fig. 2.9a denote the occurrence embedding under *takeoff* phases and Fig. 2.9b shows the embedding under *landing* phases. We can see interesting patterns from the difference between those two visualizations. For example, we can see clear cluster for 190-194 (*gear collapsed*). However, there are no such clusters in the landing phase. The intuition behind this observation is that different kinds of *gear collapsed* will have a similar impact under *takeoff* phase but will have a very different impact on event sequences under *landing* phase. Similarly, 350-353 (*loss of engine power*) will have a more consistent impact on aircraft under the *landing* phase comparing to the *takeoff* phase.

2.5.4 Event Prediction Accuracy

In this section, we build a predictive model to evaluate the performance of different embedding vectors. Here, we compare 4 different methods. In general, we compare three different representations of the failure events, which are one-hot-encoding, word2vec encoding, and the proposed tree-based embedding. We further compare two different sequential models, namely the Markovian model and the LSTM model. Our results show that, through representing discrete events with continuous vector, we can improve the prediction accuracy with either Word2Vec or Tree Embedding. Furthermore, LSTM shows better performance comparing to the Markovian model given Tree embedding of failure events. Finally, the tree embedding shows better performance compared to the general embedding method Word2Vec in our experiments.

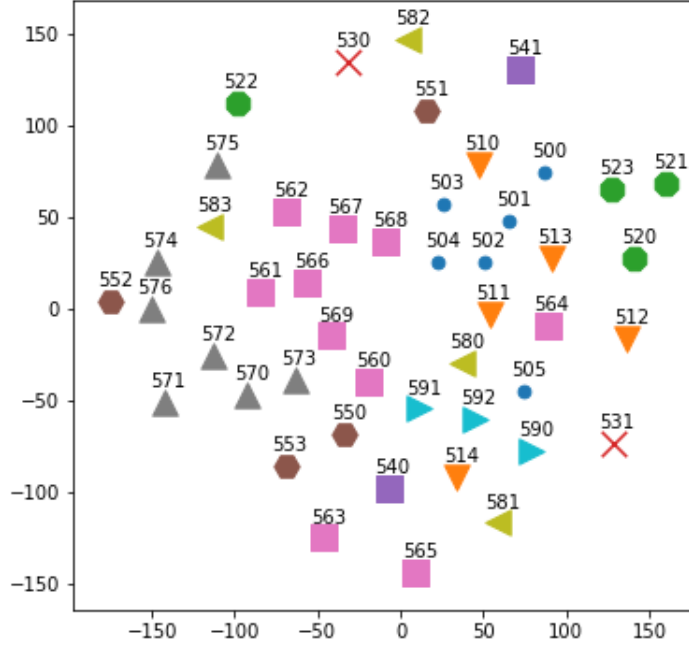


Figure 2.8: Decoder Phase Embedding

In an aviation accident, the phase of aircraft is usually known ahead of time. Here we also present the results given known flight phase. We show that the proposed method is able to achieve 0.8291 accuracy for occurrence level, which is a huge improvement from 0.5497.

Furthermore, we can plot the occurrence prediction accuracy under different phases. Fig. 2.10 shows the comparison between the baseline method and the proposed method under different phases. We can see that the proposed method improves the occurrence prediction accuracy in all phases. In phases like *standing*, *takeoff* and *maneuvering* the proposed method improved a lot from the Markovian methods. As illustrated in the previous section, the limitation of traditional sequential mining models like Markovian methods or the association rule-based method is the inefficiency

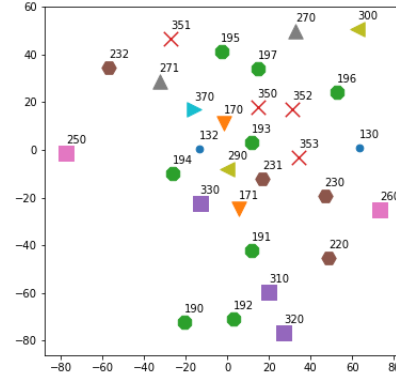
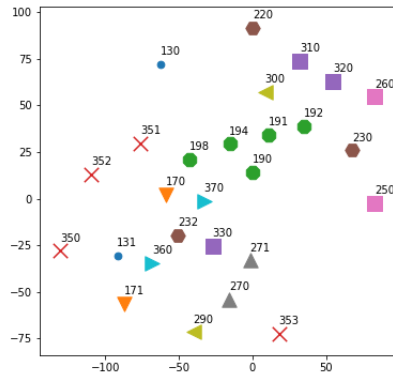


Figure 2.9: Decoder Occurrence Embedding

to model rare events with multi-level hierarchical structures and inability to model the long-term dependency. However, in our problem, the event at the next time step not only depends on the past event sequences but also some higher-level information like flight phases. If we ignore the relationship among multiple levels of information, it is hard to make predictions on future events.

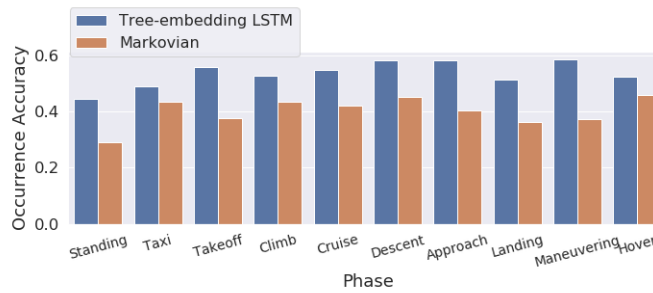


Figure 2.10: Occurrence Prediction Accuracy under Different Phases

Here we further investigate the reason we achieve better results comparing to

other benchmarks without considering the multi-level information.

- The other benchmark methods, such as the Markovian models, cannot deal with the unseen event sequences. The aviation accident is usually unique and rare, so it is important to deal with events that have never occurred before. The proposed model is able to deal with the new events according to their similar sequential pattern calculated according to the event embedding. The embedding representation of events enables us to find the most similar event for a new failure event and then make predictions. For example, occurrence codes 190-194 denote different types of gear collapsed related accidents. However, we can see from Fig.3.5 that 194(*complete gear collapsed*) is rarely happened, thus it is not sufficient to make predictions based on it due to limited of observations. However, from Fig. 2.9a, we can see that under *takeoff* phase, embedding representation for 190-194 are closely positioned. Thus, we can learn from other types of gear collapsed accidents and make similar predictions when it is under *takeoff* phase.
- Under different phases, the transition between occurrences is very different. The other benchmark methods, such as the Markovian method or the association rule-based method, can only follow a simple rule for the transition, which does not consider the high-level information. The proposed model incorporates this multi-level structure according to the hierarchical decoding layer. Through building up the conditional embedding vectors, we are able to capture different transition patterns under different phases. A straightforward way to illustrate this problem can be referred to Fig.2.6b and Fig.2.6a. We can see the most frequent transition patterns are completely different under two phases. More specifically, under different phases, the failure event related to *nose gear col-*

lapsed will lead to *nose over* under *landing* phase. But under *takeoff* phase it also has chance to *nose down*.

- The sequential relationship is complex when we are trying to deal with multi-level event sequences. Traditional methods like association rule or Markovian models will require a lot of time to make predictions based on past event sequences. The deep learning-based model provides an efficient way to propagate the information from the beginning of the event to the end and capture its complex sequential relationship at the same time. One example of the sequential influence from across levels can be seen from Fig. 2.5. When the aircraft is in the *cruise phase*, it tends to stick on this status until a trigger event like gear collapsed or loss of engine power. However, it is hard to incorporate such information into the Markovian model. In the case of the deep learning model, we concatenate multi-level information into a hidden vector. Then, the lower-level information can influence the higher level status. When some abnormal events happened, the aircraft at the *cruise phase* will transit into the *descent* or *landing phase*.

2.5.5 Risk Quantification

In this section, we will show the result of the risk quantification based on the full or partial event sequences and quantify its uncertainty. Table. 2.4 shows the prediction accuracy with the proposed sampling approach under different prediction horizons (e.g., 1-step, 2-step or 3-step ahead) for the proposed tree embedding methods with LSTM-based and Markovian-based prediction models. From the results, we can show that the proposed LSTM-based event prediction approach achieve the best accuracy. Beyond that, we can also maintain pretty good accuracy even for a 3-step ahead

prediction.

Table 2.4: Risk Estimation Accuracy

Phase Status	Prediction Horizon	Tree Embedding+ Markovian	Tree Embedding+ LSTM
Phase unknown	1-step	0.818	0.839
	2-step	0.8	0.803
	3-step	0.778	0.784
Phase Known	1-step	0.818	0.839
	2-step	0.801	0.810
	3-step	0.782	0.798

Finally, we would like to visualize the predicted risk path along with the sequential accident in a real accident event. The accident happened on April 2, 1997. A Cessna T210N, N6172C, collided with snow covered mountainous terrain in a wilderness area on the north rim of the Grand Canyon in Arizona. The airplane was destroyed and the certificated private pilot/owner and his passenger received fatal injuries. This is the same accident introduced in detail in Section 2.3. In Fig. 2.11a, we would like to visualize the predicted risk and its uncertainty (i.e., in the y-axis) against the sequence of the observed events (i.e., in the x-axis). Here, the risk score is normalized by assigning "2" to the "destroyed" and "1" to the "sustainable" damage with "0" to "little or no" damage. It can be seen that the initial uncertainty is pretty large due to the number of possible future event paths being large. In the later stage, the uncertainty is pretty small and the aircraft is almost certain to be destroyed after the aircraft has lost its control. To show that the risk uncertainty is correctly quantified, we also plot the reliability diagram in Fig. 2.11b. In order to calculate the reliability

diagram, we first binned the prediction results according to their confidence. As shown in figure, the rightmost bin represents the samples that has confidence between 0.9 to 1. Then, we evaluate the prediction accuracy by each bin which is shown in the y-axis. The goal of the calibration is to let predictions with large confidence score be more accurate. Overall, if the reliability diagram follows a 45-degree line, it implies that the predicted model with lower confidence indeed gets lower accuracy, which implies the model uncertainty is correctly calibrated. Here, it is clear that the reliability diagram follows the 45-degree line, and the risk uncertainty is correctly calibrated with the proposed path sampling and temperature scaling approach.

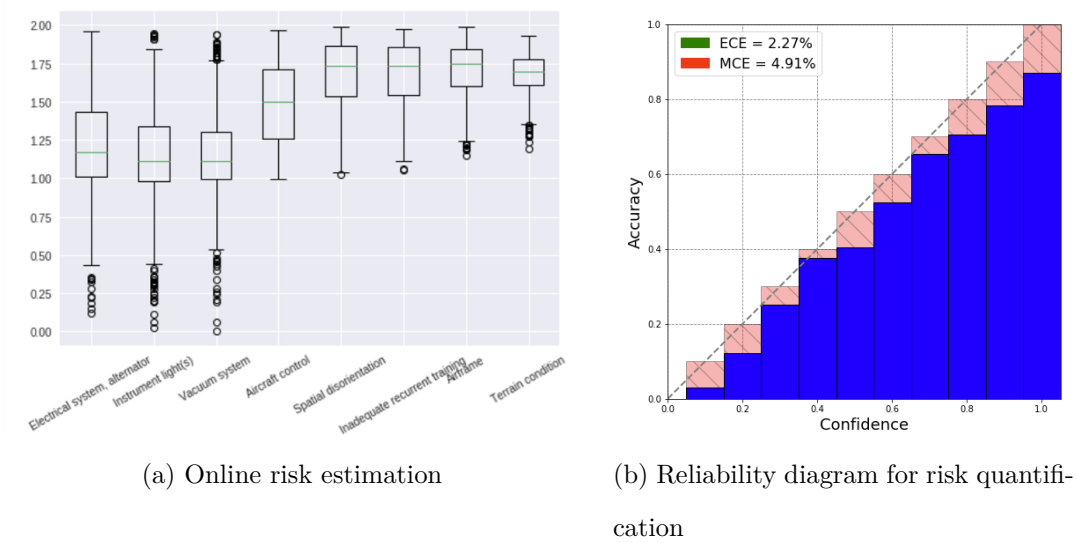


Figure 2.11: Left: Risk Quantification Case Study with X-axis as the Observed Events at Current Time, and Y-axis as the Risk Score. Right: Reliability Diagram for Risk Quantification

2.6 Conclusion

In this work, we talked about how to leverage the collection of event data from aviation accident report to improve aviation safety. More specifically, we proposed a

way to predict the possible event sequences so that people are able to provide effective actions to avoid the accidents. There are three major contributions to this work. First, we proposed a hierarchical encoder-decoder framework with the domain knowledge of the taxonomy of failure event can be used in the embedding model. Second, we combined the proposed hierarchical encoder-decoder framework with the LSTM to capture the complex temporal relationship among failure events. Our results show that the hierarchical encoder-decoder combined with the LSTM has better accuracy comparing to several benchmark methods for event prediction. Furthermore, the embedding representation also provides a more meaningful result.

Chapter 3

MULTI-LABEL TEXT CLASSIFICATION FOR AVIATION ACCIDENT REPORT

3.1 Introduction

In air traffic management (ATM), the analysis of the past accident is of critical importance. For example, many existing efforts focus on developing the next-generation national air transportation system (NextGen). A key component of NextGen is the capacity to discover and analyzes safety risks and supports safety enhancements to mitigate risk. To understand the historical accident, the accident reports provide valuable information. Detailed analysis of the accident report is important to avoid those threats in the future. Traditionally, manual root cause analysis is conducted on these accident reports case Wiegmann and Shappell (2017). Thus, to represent reports in a standard and structured format, different aviation taxonomies are identified as the most critical causes/events/factors and designing information extraction algorithms to extract that information automatically. However, such manual analysis is often very time-consuming and labor-intensive, given that it often requires many efforts from domain experts. Furthermore, It might also lead to labeling inconsistency due to the subjectivity of different people. Those challenges lead to the severe quality issue for the event labels of the accident reports Rao and Marais (2020).

Overall, there are significant challenges in analyzing the accident report. 1) *Unstructured text data*: the narrative reports have very different lengths and it poses significant challenges to traditional supervised learning methods. 2) *Rare events*: Since we are working with reports that record aviation accidents/incidents, every

case is unique and rarely repeated. 3) *Multiple events in each report*: There are multiple events corresponding to each accident report. Furthermore, the number of events is typically unknown. 4) *Irrelevant information*: There are a large number of irrelevant information in the text data, and it is unknown which sentence typically corresponds to which event.

To analyze the aviation accident report data, some previous works have been developed. For example, some researchers from the NASA group Ferryman *et al.* (2006) analyzes aviation reports from Aviation Safety Reporting System (ASRS) Connell (1999) and identifies 14 important human performance factors. The following work by Posse et al. Posse *et al.* (2005) discussed how to design a template-based information extraction framework. Another structured representation of a narrative report is by converting it to a sequence of events. According to the definition from National Transportation Safety Board (NTSB), the event can be defined as "a distinct major cause of relative significance that leads to an accident or incident" NTS (2008). Fig. 4.2 provides an example from NTSB that illustrates the relationship between narrative report and event sequence. The left side is the accident report, while the left side is a two-level event abstraction of the report defined by NTSB. According to the taxonomy defined by NTSB, occurrence code represents significant events, and subject code provides a more detailed description of the occurrence code.

With the enrichment of event taxonomy of aviation accidents, many researchers built accident models based on those critical events. Those event log data helps us identify the riskiest casual path in helicopter accidents Rao and Marais (2018). By collecting enough evidence about the correlation between events and severity of aviation accidents, we can come up with a better prevention strategy whenever an accident happens Zhang and Mahadevan (2021). Existing works also talked about how to build an efficient taxonomy-based information retrieval system for the avia-

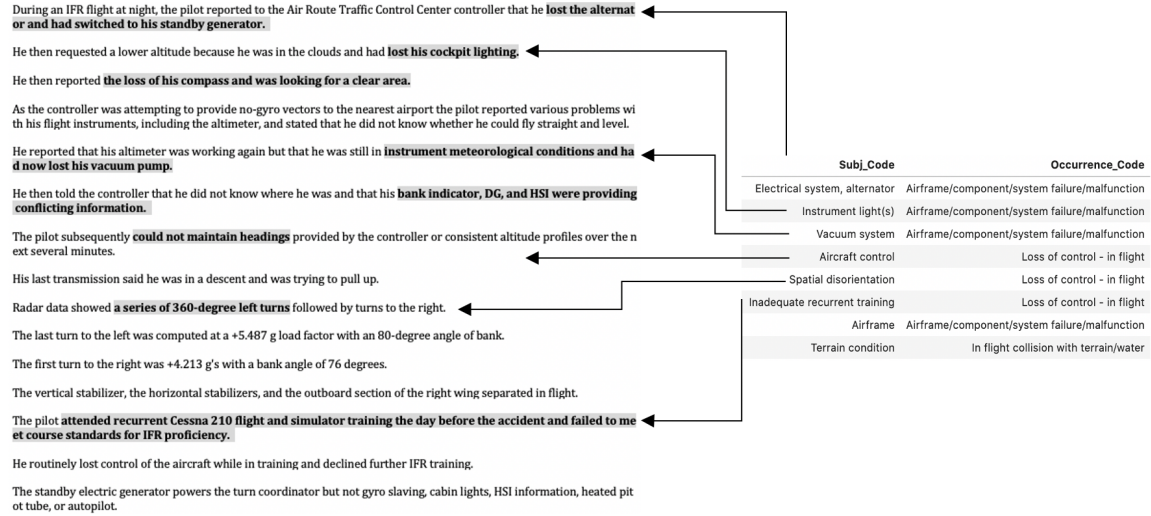


Figure 3.1: Aviation Accident Report to Event Sequence: Report 20001208X07734 from NTSB

tion domain Tanguy *et al.* (2016). Overall, translating the aviation accident report narrative to standard event coding can provide much compact and consistent representation and is often much easier for future analysis, such as causal discovery and risk quantification. Thus, it inspires us to build an efficient, consistent, and fully automatic labeling framework to convert unstructured text data into structured event sequences. Targeting this challenge, we are trying to provide a framework to accurately and consistently convert the unstructured text data into a structured format with Natural Language Programming (NLP).

In order to extract multiple events from the accident reports, we formulate the problem as a multi-label text classification problem, given that each report contains multiple critical events. Binary relevance is one of the most popular multi-label classification methods that usually serve as a baseline. It converts the multi-label classification problem into multiple binary classification problems Zhang *et al.* (2018). However, the naive assumption neglects the correlation between different labels. From

Fig. 4.2, we can see the chronological order among events which shows the casual correlations. Thus, without considering the label correlation, it will influence the performance in our case. With the rapid growth in Neural Networks, we can build more flexible architecture considering label correlation. Sequence-to-sequence architecture achieves enormous success in machine translation due to its flexibility in the input and output format Bahdanau *et al.* (2014). That inspires people to consider the label correlation with this type of framework. Yang et al. applies the sequence generation framework to deal with the multi-label classification problem and proves its effectiveness on correlation representation Yang *et al.* (2018).

Even though deep learning-based methods have shown their power in text classification, it usually requires a large amount of training data. The rare event issues in the dataset mentioned above have become one of the biggest challenges for training a Neural Network. Previous work that extracts events from reports by constructing templates or finding keywords requires much human labor and can only be conducted on a small set of reports Posse *et al.* (2005). Beyond that, different from traditional text classification problems in social media Sriram *et al.* (2010) or newspaper Kumaran and Allan (2004), aviation reports usually contain a lot of terminology and abbreviation. The uniqueness and specific aviation language can lead to the scarcity issue in narrative reports, which increases the challenge of training a Neural Network from scratch.

The recent development of the NLP community shows that we can take much advantage of the pre-trained deep language representation models like GPT Radford *et al.* (2018), ELMo Peters *et al.* (2018), and BERT Devlin *et al.* (2018) which have to achieve the state-of-the-art performance in sentiment classification and question answering. Those pre-trained models provide us meaningful input representations by pre-training on a huge language corpus, which relieves the burden of training a Neu-

ral Network from scratch. Among those pre-trained models, Bidirectional Encoder Representation from Transformers (BERT) outperforms its predecessors and achieves the best performance in many NLP tasks. It has been shown that much less data is required to achieve relatively good results through fine-tuning these pre-trained models Sun *et al.* (2019); Chu *et al.* (2016); Howard and Ruder (2018). With the powerful text representation, many works have developed BERT-based multi-label classification frameworks and have shown its effectiveness Chang *et al.* (2019); Chalkidis *et al.* (2019).

This paper aims to build a narrative to event extraction framework by utilizing the pre-trained BERT model for the aviation domains. Furthermore, attention layers have been incorporated to improve the interpretability and further increase the event extraction accuracy. We also compare the model performance with several benchmark methods in the literature, including the binary relevance, the sequence-to-sequence prediction, and the BERT-based classifier without attention layers on NTSB reports. To evaluate, we compare the model performance according to the F1 measure, precision, and recall rate. We find that BERT-based event extraction methods can outperform the others while maintaining a reasonable model interpretation. As a summary, the significant contributions of this paper are listed as follows:

1. We discussed how to fine-tune the pre-trained BERT model for the aviation domain. Moreover, we also incorporated the attention mechanism into the BERT model and talked about the benefits of visualizing the relationship between reports and event sequences.
2. Since most of the previous works are based on the ASRS dataset, few works discussed the challenge on the other well-organized aviation report corpus - NTSB. Our work provides a solid baseline and an effective classifier for the

NTSB report system.

3. Our method can provide an intermediate attention layer, which illustrates which particular sentences/words each event is extracted from. This can provide an interpretative guideline on why such an event is extracted from the narrative accident report.
4. Our results show that we can achieve 70.8% F1 score, which validates the effectiveness of the proposed methods.

The paper is organized as follows. We first provide an overview of existing works on extracting structured knowledge from the narrative reports in Section III. We also discuss recent progress in multi-label text classification. In Section IV, we will review the state-of-art methods and introduce the proposed framework. In Section V, we present the implementation details and experiment results. We finally conclude this paper and explore the future work in Section VI.

3.2 Literature Review

In this section, we will review some existing methods in the literature for event analysis in the accident report. More specifically, we will focus on reviewing two major subareas: information extraction for narrative reports and multi-label text classification.

3.2.1 *Information Extraction for Narrative Reports*

Efficiently analyzing the narrative reports enables us to identify threats from past incidents and accidents and provide prevention strategies to avoid them. However, due to the large volume of emerging reports, manual analysis of reports becomes increasingly tricky. Ludovic et al. estimate that there are around 600 per month

reports for a large airline company Tanguy *et al.* (2016). We can also find an increasing amount of reports from the ASRS dataset where we have 4600 incident reports in 1981 while 10800 incident reports in 2019 Dong *et al.* (2021). Thus, more researchers have started developing algorithms that can handle narrative reports on a large scale. Traditionally, research works mainly focus on designing templates or finding keywords to extract useful information, which requires a lot of human efforts Posse *et al.* (2005). It is first argued that the keywords and expressions identified by a human cannot be exhaustive, which is limited to a small-scale evaluation Abedin *et al.* (2010). Thus, the authors further proposed an automatic keyword extraction strategy via semi-supervised semantic lexicon learning. They also show that the task can be converted into a text classification problem. Constructing multiple Support Vector Machine(SVM) text classifiers, they formulate the multi-label classification problem as multiple binary classification problems, the schema as Binary Relevance. Later on, more researchers deal with the information extraction problem in the supervised learning format. Different machine learning methods have been proposed such as Bayesian logistic regression Yelundur *et al.* (2016), multi-label Hoeffding trees Shi *et al.* (2017), latent semantics analysis Robinson (2018), etc. Those works have successfully reduced human labor in this challenge, but the algorithms' performance highly relies on the quality of linguistic features. As mentioned in Abedin *et al.* (2010), it is not sufficient to use simple linguistic features like N-gram or other semantic lexicon defined by researchers. Those features serve as a good starting point, but we still need huge effort to discover better features feeding our model. Besides the difficulty in defining features, all methods above treat the multi-label classification problem as an independent binary. As mentioned earlier, the naive assumption neglects the relationship among labels which will hurt the performance. In order to deal with those two challenges, people start looking into deep learning. Deep learning models

have achieved huge success in various tasks since they can learn a powerful input representation. In other words, it benefits from powerful computational resources. It can automatically find good features through training a neural network Goodfellow *et al.* (2016). The success of deep learning motivates many researchers to bring its power into analyzing narrative reports. Zhong *et al.* used word embedding to represent the construction accident reports and further applied a Convolution Neural Network (CNN) to automatically extract linguistic features for the classification task Zhong *et al.* (2020). Another work that focuses on the construction narrative reports proposed a Convolutional bidirectional long short-term memory (C-BiLSTM)-based method. It has shown that the model outperforms classical machine learning models like SVM, naive Bayes, and logistic regression Zhang *et al.* (2020). There are also existing works in the aviation domain. Dong *et al.* (2021) train an attention-based long short-term memory (LSTM) model to identify important factors in aviation incident reports in ASRS. It is worth noting that the proposed methods not only achieve the state-of-art performance but also provide us the explanation ability through attention mechanism Vaswani *et al.* (2017). Zhang *et al.* Zhang *et al.* (2021) also developed an LSTM based framework but focused on evaluating the risk level of the other important aviation report corpus, NTSB. All those works have built a solid foundation for us to explore advanced algorithms further. In this paper, we will focus on the event extraction task in the NTSB report. We are aiming to make a comprehensive comparison between SVM and advanced deep learning-based multi-label classification methods. Unlike previous works, this is the first paper that provides a solid baseline for the NTSB reports. Furthermore, we bring BERT into the aviation domain, which helps us benefit from powerful pre-trained models in the NLP domain.

3.2.2 Multi-label Text Classification

Multi-label classification considers the case where we may have one or multiple labels in a sample. Most previous work considers multi-label classification as multiple independent binary classifications, also called binary relevance approach. Many following researchers focus on exploiting the label dependencies. Pairwise decomposition (PW) is one of those strategies that create pairwise sub-problems or build related label combinations Tsoumakas *et al.* (2010). Another framework called Classifier chains Dembczynski *et al.* (2010) considers the label relationship by incorporating previous predictions into the prediction of subsequent labels. Those types of frameworks are efficient when the label set is small. The computational complexity will increase exponentially as the amount of labels increases. In other words, conducting large-scale multi-label classification requires more flexible architecture and computational power. With the recent progress in deep learning, emerging works have started solving multi-label tasks with innovative Neural Network (NN) structures. It has been shown that simple NN models with rectified linear units, dropout, and AdaGrad can already achieve state-of-the-art performance Nam *et al.* (2014). By incorporating a dynamic pooling operation and a rank loss function that considers a label dependency, a novel CNN-based multi-label classifier is proposed in Liu *et al.* (2017). Another LSTM based framework was proposed in Yang *et al.* (2018), which includes a novel decoder structure that helps capture the label dependency. It is worth noting that the sequence-to-sequence mechanism takes input text as a sequence of tokens and predicts the labels sequentially. Beyond the novel NN architecture, pre-trained models enable researchers to experiment with a language model trained on a massive corpus with unattainable computer resources. BERT is one of the most popular pre-trained language models by Google, which considers the corpus more than 3000 million words

and takes three days on 16 TPU v3 chips to finish the pre-training Devlin *et al.* (2018); You *et al.* (2019). However, different from the expensive pre-training process, it only takes at most 1 hour on a single Cloud TPU or a few hours on GPU for fine-tuning a pre-trained BERT model to achieve state-of-the-art results. We can easily convert raw text into meaningful deep representation for various NLP tasks like question answering, semantic role labeling, and text classification by taking advantage of those powerful language models, learned from the large number of corpus. A recent work by Change et al. shows the major challenge of applying BERT for multi-label classification is to capture the label dependency and scale the model when the size of the label set is huge Chang *et al.* (2019). However, this works do not consider the customization of the BERT model into a particular domain, customized attention module, and does not consider the event labels, which limits the performance for rare event prediction.

3.3 Methodology

Let us first give a formal definition of the multi-label classification task. We have a set $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y}$ that contains N samples. Under text classification problem, $\mathbf{x}_n$ represents input document, and $\mathcal{X}$ denotes the space of natural language text. $\mathbf{y}_n = [y_1, y_2, \dots, y_L] \in \mathcal{Y} = \{0, 1\}^L$ means L number of unique labels. Moreover, our goal is to find the mapping from input data (i.e., narrative) to the label space $\mathcal{Y}$ (i.e., event). In the following subsections, we will first review two state-of-the-art frameworks with Seq2seq Model and BERT Model, and then discuss our proposed BERT based method for the aviation event extraction.

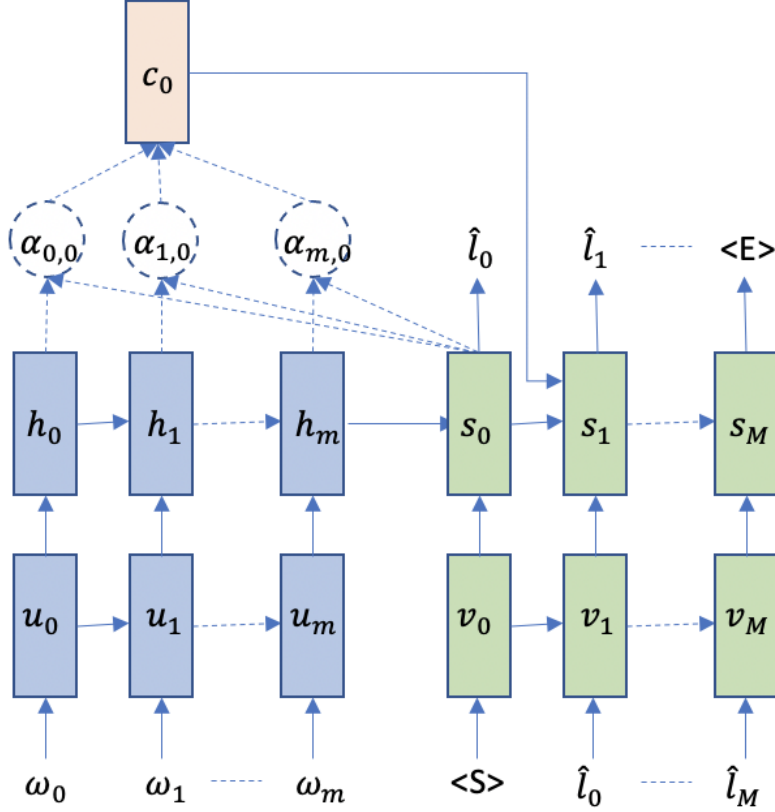


Figure 3.2: Encoder Decoder with Attention Architecture

3.3.1 Review of Sequence Generation Model

Sequence-to-sequence achieve great success in machine translation Bahdanau *et al.* (2014), abstractive summarization Nallapati *et al.* (2016), where the output is sequentially predicted based on the input data and the predictions from previous steps. With the assumption that labels have some type of dependency, we can formulate multi-label classification as a sequence generation task. Firstly, we need to represent $\mathbf{y}_n$ with an ordered set $\mathbf{Y}$ according to the label dependency defined by prior knowledge. $\mathbf{Y} = \{l_1, l_2, \dots, l_M\} \subseteq \mathcal{L}$ where $|\mathcal{L}| = 2^L$ represents all the potential combinations of labels. Furthermore, our input document $\mathbf{X} = [\omega_1, \omega_2, \dots, \omega_m]$ contains m words. We want to model the relationship between input $\mathbf{X}$ and its label set $\mathbf{Y}$ with a proba-

bilistic model $P(\mathbf{Y}|\mathbf{X}, \theta)$. According to the label dependency, the probabilistic model can be simplified as follows

$$P(\mathbf{Y}|\mathbf{X}) = P(l_1, l_2, \dots, l_M|\mathbf{x}) \quad (3.1)$$

$$= \prod_{i=1}^M P(l_i|l_{1:i-1}, \mathbf{X}) \quad (3.2)$$

The general framework of sequence generation can be described with an encoder decoder with attention mechanism. We will apply LSTMHochreiter and Schmidhuber (1997) as our encoder which is one of the most widely used framework for sequential modeling. We first mapping each word ω_i to a d dimensional space $\mathbf{u}_i \in \mathbb{R}^d$. And then the contextual word representation can be computed along with previous hidden state as following.

$$h_i = \text{LSTM}(\mathbf{h}_{i-1}, \mathbf{u}_i) \quad (3.3)$$

Furthermore, attention module is usually added enabling the model focus on the most relevant component of the document to the label. We first need to compute the attention weights which described the relationship between the i -th word and j -th output.

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{i=1}^m \exp(e_{ij})} \quad (3.4)$$

$$e_{ij} = a(\mathbf{h}_i, \mathbf{s}_j) \quad (3.5)$$

In the above function, α_{ij} represents the attention weights. It is calculated with e_{ij} , which is a relevance score between the i -th word and j -th output calculated by the function $a(*)$. Function $a(*)$ is a critical part for the attention which is also called compatibility function that can have various formulationsGalassi *et al.* (2020). $\mathbf{h}_i, \mathbf{s}_j$ represents the contextual representation of input words and hidden state of j -th

decoder output. With the attention weight, the context vector $\mathbf{c}_j$ can be calculated through a distribution function which is usually defined as follows.

$$\mathbf{c}_j = \sum_{i=1}^m \alpha_{ij} \mathbf{h}_i \quad (3.6)$$

Finally, let us introduce the decoder function, which helps us capture the label dependency. As illustrated in the Eq. 3.2, the prediction of next label l_j only depend on the inputs and all the previously predicted labels $l_{1:j-1}$. Here we use context vector $\mathbf{c}_j$ to represent important information contained in the input. We further use hidden state $\mathbf{s}_j$ to capture all information from $l_{1:j-1}$. With LSTM as our decoder, we can model the conditional probability as follows.

$$P(l_j | l_{1:j-1}, \mathbf{X}) = g(\mathbf{s}_j) \quad (3.7)$$

$$\mathbf{s}_j = \text{LSTM}(\mathbf{s}_{j-1}, [f(\hat{l}_{j-1}), \mathbf{c}_{j-1}]) \quad (3.8)$$

Function $g(*)$ is a mapping function that transform hidden state $\mathbf{s}$ into a probability measure of $P(l_j | l_{1:j-1}, \mathbf{X})$. $[f(\hat{l}_{j-1}), \mathbf{c}_{j-1}]$ represents the concatenation of label embedding $f(\hat{l})$ and the context vector $\mathbf{c}$. Here we use $\hat{l}_{j-1}$ to represent the predicted label at time $j - 1$ and $f(*)$ is a mapping function that converts the label to the embedding space. We will use v_j to represent $f(\hat{l}_{j-1})$ in Fig. 3.2 for simplicity.

3.3.2 Review of BERT Based Model

In this section, we will give a brief review on the existing BERT model. Unlike traditional sequential models like RNN, LSTM, BERT does not rely on recurrent structure to capture the sequential dependency. It applies self-attention to calculate the relationship between each word to the whole document. Fig .3.3 provides an example of the self-attention structure for a document with only four words. For each word ω , we have three embedding representations called Key, Value, and Query.

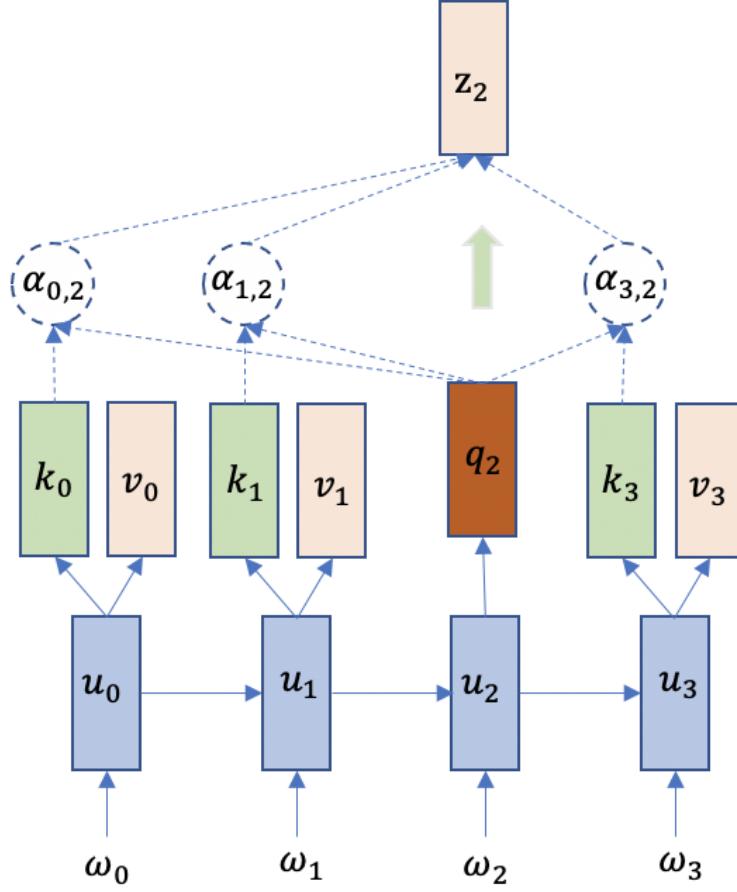


Figure 3.3: Self Attention Architecture

The self-attention mechanism resembles the memory access mechanism. Query $\mathbf{q}$ and Key $\mathbf{k}$ are used to compute the relevance score among each word. Value $\mathbf{v}$ contains the context information which is used to calculate the actual information of a word. Formally speaking, the contextual representation $\mathbf{z}_i$ of a word ω_i is computed as follows.

$$\mathbf{z}_i = \text{softmax}\left(\frac{\mathbf{q}_i \mathbf{k}^T}{\sqrt{d_k}}\right) \mathbf{v} \quad (3.9)$$

We first compute the dot products between Key-Value pairs and then divides them by a scaling factor $\sqrt{d_k}$, which is assumed to help improve the performance of the softmax behavior Vaswani *et al.* (2017). d_k denotes the dimension of the Key. With

the self-attention module, each word can borrow information from the context by exchanging information with other words in the document. However, each word may have different relations with the document, such as positional and syntactic roles. It inspires people to concatenate multiple independent self-attention modules where each of them has a unique representation of the word. In addition to the attention, each layer has a feed-forward network block that contains two linear layers with ReLU non-linearity. Residual connections He *et al.* (2016) are used for each attention and feed-forward network to solve degradation problems and ease the training process.

3.3.3 Proposed BERT Model for Aviation Event Extraction

In this section, we will discuss the proposed BERT model and how it can be combined with the attention mechanism for multi-label classification in the aviation accident report event extraction.

Multi-label Classification via BERT model

With BERT, we can easily access a more powerful contextual representation and replace the LSTM encoder. In this work, we propose two approaches to build a multi-label classifier with BERT, as shown in Fig. 3.4. BERT encoder produces a sequence of hidden representations of words, and it appends a special token $[CLS]$ which is calculated with the sequence of hidden states through linear layers and a Tanh activation function as described in Huggingface Wolf *et al.* (2019). The $[CLS]$ token can be served as an aggregate representation of the document. Thus, a naive approach to build a multi-label classifier with BERT is by simply adding a classification layer with the representation of $[CLS]$ token as shown in Fig. 3.4a. The classification layer is made up of a feed-forward layer and a Sigmoid function. Finally, We apply the cross-entropy loss function during the training phase. This multi-label classification model

is designed based on decomposing the prediction into several binary classification problem by assuming that each label is conditional independent based on the BERT representation, which doesn't consider the relationship of the labels and how each label is related to the narrative report.

Incorporating Event Embedding for the BERT model

Here, to model the relationship between labels and narrative reports, we propose to incorporate the attention module and event embedding on top of the BERT model. To do so, we build a similar framework as the seq2seq learning with BERT by replacing LSTM-encoder with BERT-encoder. As shown in Fig. 3.4b, we first collect the BERT representation $\mathbf{z}_i$ for each word ω_i .

Furthermore, we propose to use the embedding for each event label as $\mathbf{v}_j$. In this paper, we will investigate the following ways of coming up such event embedding: (1) Use the BERT model and the event name to obtain the event embedding. The idea is to utilize the word embedding from BERT not only for the narrative report but also the event labels. We first tokenize each label and then retrieve its word embedding from the BERT model. Finally, we sum up the embedding vectors of each word as the label embedding. (2) Learn the event embedding automatically from data. This method is to learn the event labels completely from data. In this case, such event labels will be learned in an end-to-end manner together with the multi-label classification model.

Incorporating the Attention Module for the BERT model

To link the event embedding and the word embedding, we propose to add the attention module on top of the BERT model by utilizing the word embedding and event embedding by computing the attention weight $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{i=1}^m \exp(e_{ij})}$, between word

representation $\mathbf{z}_i$ and the event embedding $\mathbf{v}_j$. Here, $e_{ij} = a(\mathbf{z}_i, \mathbf{v}_j)$ measures the similarity of the sentence i and event label j , similar to Eq (3.5). Finally, we propose to use the context vector c_i can be calculated as Eq (3.6) to encode the necessary information to predict each labels. Lastly, we can compute the probability of each label with $[\mathbf{v}_i, \mathbf{c}_i]$. Finally, we follow Luong’s attention mechanism Luong *et al.* (2015) to calculate the final prediction score with a Tanh function with a Softmax layer for the multi-label classification.

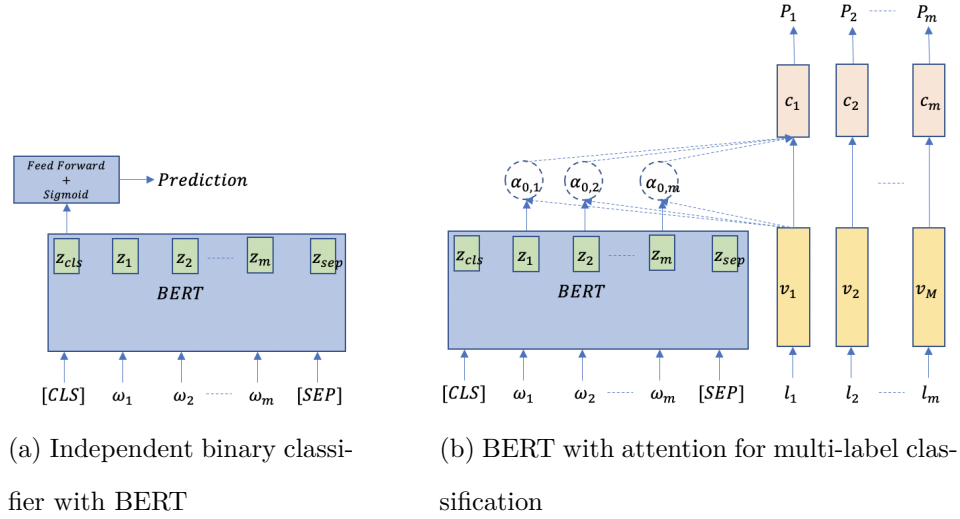


Figure 3.4: Different Architectures for Multi-label Classification with BERT

3.4 Experiment setup

In this section, we will describe the experimental setup for the proposed method. First, we will start with the data description in Section 3.4.1, data preprocessing in Section 3.4.2, and the implementation details in 3.4.3. Then, the evaluation metric in Section 3.4.4 and the results are discussed in Section 3.4.5. Additional results such as the impact of attention module, choice of event label embedding, and domain adaptation will be discussed in Section 3.4.6. Finally, we will visualize the attention

module for the BERT model for better model interpretation in Section 3.4.7.

3.4.1 Data Description

In this section, we will introduce the dataset and our preliminary analysis. The NTSB aviation accident database recorded information about civil aviation accidents since 1982. The data is publicly accessible in Microsoft Access database format NTS (2008). Since the coding taxonomy has a major difference before 2008 and after 2008, this work selects more consistently labeled reports from 1982 to 2008, ending with 62569 reports. As shown in Fig. 4.2, each report is labeled with a sequence of failure events called Phase, Occurrence, and Subject. There are 56 Phase codes, 58 Occurrence codes, and 1432 Subject codes. Phase describes the flights status of the aircraft such as "*Cruise*", "*Take-off*", "*Landing*". Occurrence illustrates the most critical events related to the accident. The subject introduces more details to describe the Occurrence further. This work identifies Occurrences from narrative reports by formulating the problem as a supervised multi-label classification problem. We randomly split the data into 90% for training and 10% for testing. Fig. 3.5 shows the distribution of different events and their relationship with the flight phases. The figure shows the sparsity issue where 36 events have fewer than 1000 instances, and 19 have fewer than 100 instances. Occurrence code 230 represents *in flight collision with terrain/water* which is the most frequent events. It can also be seen that 230 mostly happens during *Descent* Phase. Some Occurrence codes are explained in the figure. More details about the meaning of Occurrence codes can be found in Table. A.1 from Appendix.

3.4.2 Data Preprocessing

In this work, we aim at identifying events from the narrative reports. In order to construct meaningful input data, we first process the text data with a Python li-

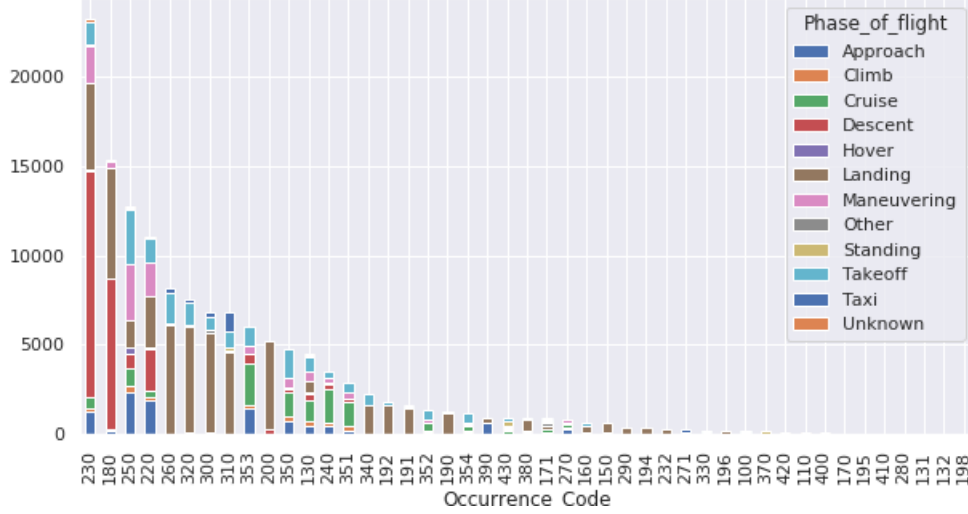


Figure 3.5: Bar Plot for Flight Phases

brary called Natural Language Toolkit Steven (2004). As a sequence of standard text processing procedures, we convert data into lower cases and remove the stop/punctuation words. Next, we conduct stemming and Lemmatization to reduce the size of the word corpus. For BR-SVM, it requires manually selected features. We followed reference Abedin *et al.* (2010) constructing N-gram features and further add term frequency-inverse document frequency(TF-IDF) Jing *et al.* (2002) using Sklearn package. For the Seq2seq model, it has been shown in Nallapati *et al.* (2016) that we can highly improve the model performance with a feature-rich-encoder. It means that, beyond word embedding, we also need to consider linguistic features like part-of-speech, named-entity, and TF-IDF. We will create a look-up table for the discrete features and categorize the continuous features. For BERT based encoder, we did not construct any linguistic features by hand. We take raw narrative reports as input to the BERT and tokenize them with pre-trained tokenizers accessed from Huggingface Wolf *et al.* (2019). It automatically identifies words that are within the pre-trained library. For the out-of-vocabulary (OOV) words, instead of simply converting them

with unknown, BERT tokenizer breaks a word into several subwords with a WordPiece algorithm Devlin *et al.* (2018); Sennrich *et al.* (2015). The algorithm is a data compression technique originally called Byte Pair Encoding (BPE) which replaces the most frequent pair of bytes iteratively with an unused byte. In the WordPiece algorithm, it adapts BPE for the word segmentation case.

3.4.3 Implementation Details

In this section, we will introduce the important hyperparameters of the model. For LSTM based framework, the word embedding size is set to 512. We apply bi-directional LSTM to capture sequential dependency from both directions. The hidden size of both encoder and decoder is 256. We apply AdamW as the optimizer in the training phase and switch to SGD to ensure a stable learning curve. We select the learning rate as 0.0003 and train 35 epochs for each algorithm. Moreover, we set the dropout rate to 0.3 to avoid the overfitting issue Srivastava *et al.* (2014). For BERT based framework, the hidden size is 768, and each document is truncated at the length of 512. Adam optimizer is used with $\beta_1 = 0.9$, $\beta_2 = 0.99$ and learning rate 0.00002 followed previous work Devlin *et al.* (2018) and drop out rate is set to 0.3. The experiments are conducted on Pytorch 1.4.0 on a workstation with Intel(R) Core(TM) i7-5930K CPU @3.50 GHz CPU and accelerated with a single NVIDIA GTX 1080 Ti GPU.

3.4.4 Evaluation Metrics

To evaluate the performance of three methods, we apply traditional evaluation of information retrieval systems, ie. Precision, Recall, and F1. As illustrated in Fig. 3.5, the event labels have an extended tail distribution. To avoid the events imbalance influencing the results too much, we mainly rely on the Micro-F1 measurement, which

can be viewed as a weighted average of the precision and recall Schütze *et al.* (2008) as follows.

$$P_{micro} = \frac{\sum_{j=1}^L tp_j}{\sum_{j=1}^L tp_j + fp_j},$$

$$R_{micro} = \frac{\sum_{j=1}^L tp_j}{\sum_{j=1}^L tp_j + fn_l},$$

$$F_{1-micro} = \frac{\sum_{j=1}^L 2tp_j}{\sum_{j=1}^L 2tp_j + fp_j + fn_l}$$

3.4.5 Performance Evaluation

In this section, we will compare the multi-label classification results for three methods Binary Relevance with SVM(BR-SVM), Seq2seq with LSTM(Seq2seq-LSTM), and BERT with attention (BERT+Attention). Tabel. 3.1 concludes the significant results for our methods. It is worth noting that the precision score for all three methods is pretty close. The significant difference among the three methods is the recall rate. Less than 50% events can be identified by BR-SVM. Moreover, Seq2seq-LSTM has made about 37% improvement over the baseline attributing to the deep representation of text data and flexible architecture to capture the label dependency. The BERT-based model further improves upon the LSTM model. As mentioned earlier, a pre-trained language model can reduce the training time and bring us a more robust representation of text data. Pre-training on a large language corpus provides a good starting point to fine-tune the model instead of training a complex model from scratch.

We further visualize precision, recall and F1 of all three methods BR-SVM, Seq2seq-LSTM, and BERT+Attention for each individual event label in Fig. 3.6, Fig. 3.7, and Fig. 3.8, respectively. The event labels are sorted according to their frequency in the dataset as Fig. 3.5. From the figures, it is clear that frequent event labels have much better performance than rare events. BR-SVM can maintain a good precision

Table 3.1: Performance Comparison of Different Methods

Methods	Micro F1	Micro Precision	Micro Recall
BR-SVM	0.5842	0.7241	0.4897
Seq2seq-LSTM	0.6958	0.7222	0.6713
BERT+Attention	0.7113	0.6974	0.7259

score for most events, but the recall rate is poor. For Seq2seq-LSTM, the improvement over BR-SVM on frequent events is obvious, but it cannot detect most of the rare events. From our results, it is worth noting that the BERT-based model not only makes improvements on frequent events, but the performance on rare events is also much better than the other two methods. In conclusion, BERT achieves the best micro-F1 score among the three methods. We can also see its ability to deal with data scarcity issues through performance improvement on rare events.

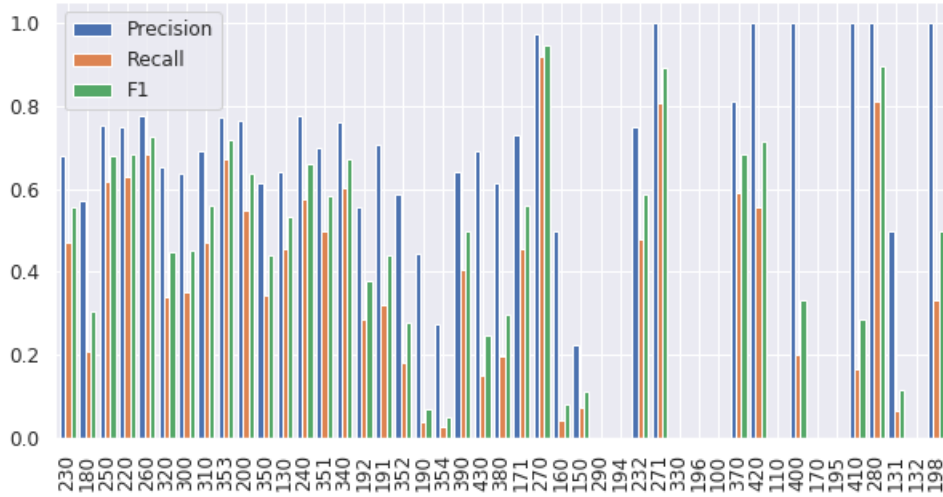


Figure 3.6: Precision, Recall, and F1 by Failure Events with BR-SVM.

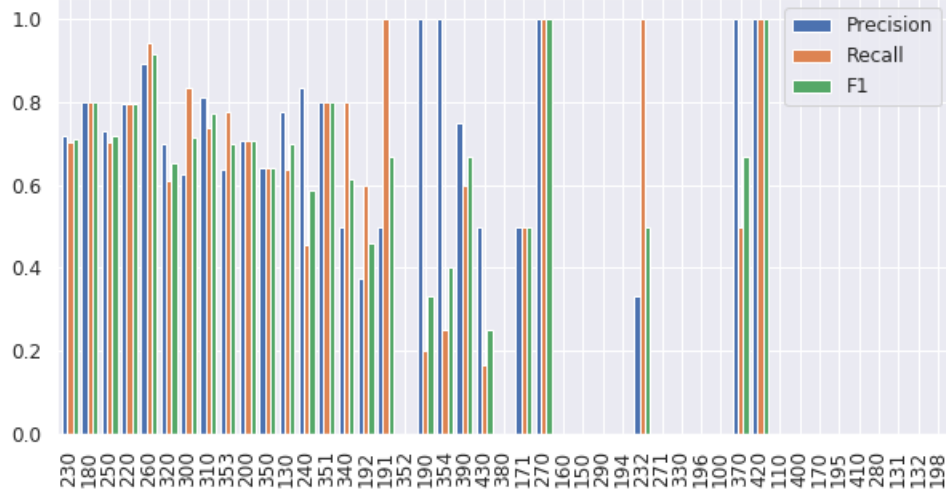


Figure 3.7: Precision, Recall, F1 by Failure Events with Seq2seq-LSTM

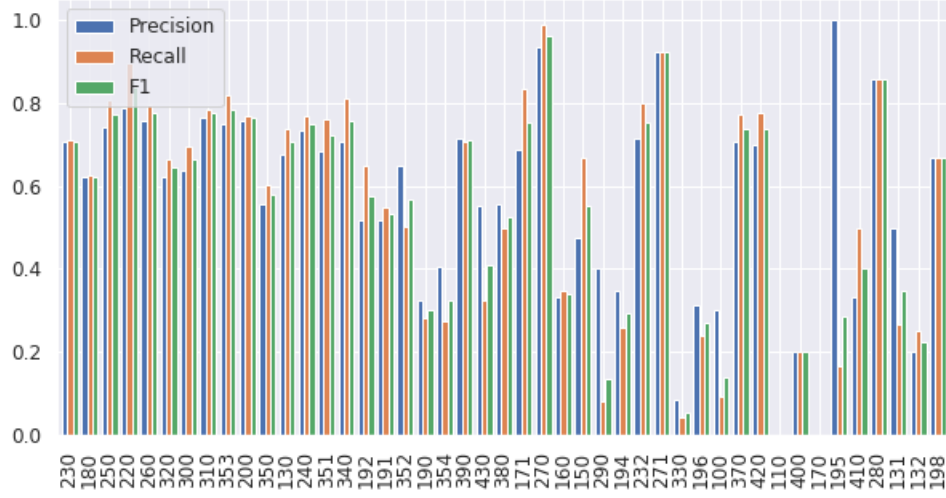


Figure 3.8: Precision, Recall, F1 by Failure Events with BERT+Attention

3.4.6 Sensitivity Analysis on the BERT Model

In this section, we will evaluate the impact of the attention module, the choice of label embedding, and the use of domain adaptation training on the BERT model.

The impact of attention on BERT: As mentioned in the methodology section, we introduced two implementations of BERT-based multi-label classifiers. Fig. 3.4a illustrates the simplest one, which directly makes a prediction based on the aggregate document representation. The other one in Fig. 3.4b includes the Luong Attention module, which helps interpret the model decision process. In this section, we will discuss more the influence of the attention module on model performance. Fig. 3.8 presents the performance of a basic BERT-based model. Moreover, following Fig. 3.9 presents the BERT model with attention. Comparing two figures, it is easy to find that more rare events can be identified by adding attention modules like 195 "*Other gear collapsed*" and 132 "*Rotor failure/malfunction*". Table. 3.2 shows a comparison of two BERT-based methods. After adding the attention module, we can identify more rare events and improve the recall rate. Nevertheless, it led to a challenge to be precise about all retrieved events. However, in general, the attention module helps improve the overall Micro F1 score. The intuition about the attention mechanism is that it enables each event label to focus on the specific location of the narrative reports. As shown in Fig. 4.2, each event label is only related to one sentence among all reports. It will be a challenge to identify all events with a single vector that represents the whole document as Fig. 3.4a. With attention mechanism, we will create a different contextual vector for different event labels by focusing on the most relevant component of the narrative reports. It makes more sense for the case of multi-label classification, primarily when each event label is only related to a few sentences within the report. Furthermore, the attention module can be a much more helpful interpretation for us as Fig. 3.11. It helps us to examine if our model is meaningful. Moreover, we can further design more useful architecture according to manual analysis on the attention results.

Exploration of the event label embedding: For either Seq2seq models or

Table 3.2: Explore Influence of Attention on BERT Classifier

Methods	Micro F1	Micro Precision	Micro Recall
BERT	0.7106	0.7138	0.7073
BERT+Attention	0.7113	0.6974	0.7259



Figure 3.9: Precision, Recall, F1 by Failure Events for BERT without Attention.

BERT-based frameworks, we need to convert event labels into embedding space so that we can compute the attention weights. It has been shown that a good label embedding can achieve good performance in zero/few-shot learning tasks Akata *et al.* (2013). To be more specific, we can convey more useful information for the following computation by representing one-hot labels with semantic vectors. In this work, we will evaluate the following two event label embedding methods. 1) BERT: embedding: We will use the BERT embedding for the event embedding. 2) Learned Embedding: Instead of using the pre-defined embedding from BERT, we will use end-to-end learning to automatically learn the event embedding fully from data. Fig. 3.10 presents low dimension mapping of these two types of label embedding. We set label embed-

ding size to 768 and project it to 2 dimensions using t-distributed stochastic neighbor embedding (t-SNE) Van der Maaten and Hinton (2008). Here we use the same color and shape to denote the event label that has similar meanings.

The two types of label event embedding are shown in Fig. 3.10. Both methods are able to group similar events together in the embedding space. However, BERT embedding neglects the contextual information provided within the narrative reports and can be easily influenced by a single word in the event label. We can tell this issue by looking at the far distance among "250", "260" in Fig. 3.10a. For *250: Loss of control - in flight* and *260: Loss of control - on ground/water*, both events are related to the "Loss of control" and therefore are mapped closer in Fig. 3.10a. However, these two "Loss of control" events happen in different flight phases and therefore are not mapped as close in Fig. 3.10b. We further compare the accuracy with these two types of label embedding in Table. 3.10b. From the results, we found that the completely learned event embedding methods out-performs the BERT embedding.

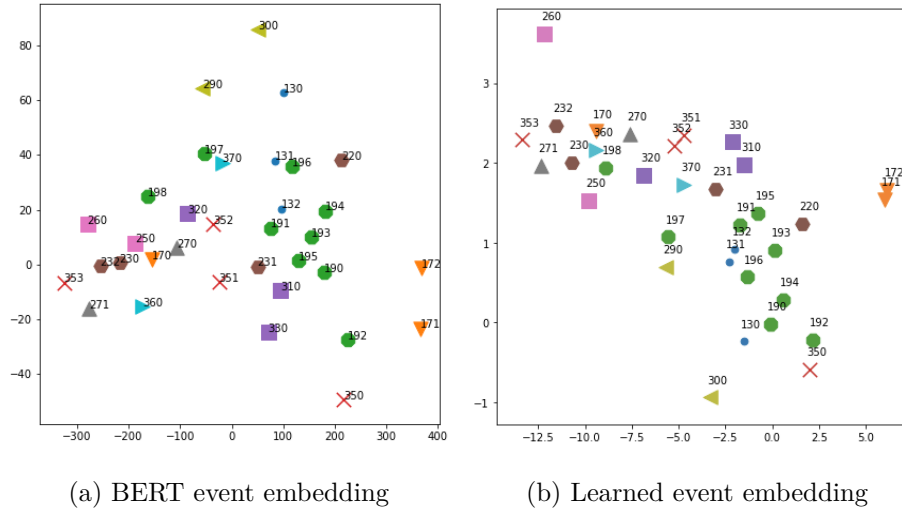


Figure 3.10: Embedding Visualization with TSNE

Table 3.3: Exploration of Label Embedding

Methods	Micro F1	Micro Precision	Micro Recall
Learned Embedding	0.7113	0.6974	0.7259
BERT Embedding	0.6942	0.6993	0.6892

3.4.7 BERT Model Interpretation and Attention Visualization

Beyond the accuracy improvement achieved by deep learning methods, another benefit is that it can help us understand its decision-making process through the attention mechanism. Fig. 3.5 demonstrates how BERT processed a narrative report. We illustrate the process with the example we shown in Fig. 4.2. When we pass a report to BERT, it is first processed by BERT tokenizer, which converts all the words into tokens within the BERT corpus. Those words not included in the corpus are replaced with their subwords starting or ending with *#* symbol. For example, *alternator* is not a common word seen in its dictionary, thus, it is converted to two subwords *alter* and *##nator*. Similarly, *altimeter* is split into *alt* and *##imeter*. After tokenizing the reports, BERT concatenates the tokens with two special tokens *[CLS]* and *[SEP]*. *[CLS]* represents a starting token, the hidden state of which is usually considered as an aggregated sequence representation for downstream tasks. *[SEP]* is a separation token to differentiate two sentences for the NSP task. Finally, to keep the same length of all documents, *[PAD]* token is added to extend the short reports. With the processed report, we can access its contextualized representation through the BERT encoder and then calculate the event probability through the decoder. By adding an attention module, we can visualize the decision process of the BERT model. As shown in Fig. 4.2, there are three associated Occurrence labels with this report *Airframe/component/system/failure/malfunction*, *Loss of control - in flight* and

in flight collision with terrain/water. The attention module contains L independent weight score for each label. In this example, we will present the attention weight for label *Airframe/component/system/failure/malfunction*. In Fig. 3.11, we highlight the words with large weight. Moreover, the words that have the highest weights are *lost alternator* and *lost vacuum pump* which are good indicators showing there is some system failure within the aircraft. From Fig. 4.2, we see that the corresponding Subject codes are *Electrical system*, *alternator* and *Vacuum system* which match the keywords identified by the proposed model. In summary, through visualizing the attention results, we can be more confident about the events identified by the deep learning model. It also provides us a way to examine our training model and improve it through designing a more advanced attention structure.

[CLS] during if ##r flight night pilot reported air route traffic control center control
 ler lost alter ##nator switched stand ##by generator . he requested lower altitude
 cloud lost cockpit lighting . he reported loss compass looking clear area . as control
 ler attempting provide no - g ##yr ##o vector nearest airport pilot reported variou
 s problem flight instrument including alt ##imeter stated know whether could fly s
 traight level . he reported alt ##imeter working still instrument meteorological cond
 ition lost vacuum pump . he told controller know bank indicator d ##g hs ##i pro
 viding conflicting information . the pilot subsequently could maintain heading provid
 ed controller consistent altitude profile next several minute . his last transmission s
 aid descent trying pull . radar data showed series 360 - degree left turn followed t
 urn right . the last turn left computed 5 . 48 ##7 g load factor 80 - degree angle
 bank . the first turn right 4 . 213 g bank angle 76 degree . the vertical stabilize #
 #r horizontal stabilize ##r out ##board section right wing separated flight . the pilo
 t attended rec ##urrent cessna 210 flight simulator training day accident failed me
 et course standard if ##r proficiency . he routinely lost control aircraft training decl
 ined if ##r training . the stand ##by electric generator power turn coordinator g #
 #yr ##o sl ##avi ##ng cabin light hs ##i information heated pit ##ot tube auto #
 #pi ##lot . descent - un ##con ##tro ##lled cruise [SEP] [PAD] [PAD] [PAD]

Figure 3.11: Attention Visualization for Airframe Malfunction

3.5 Conclusion

This work focuses on building an accurate information extraction framework that can identify events in aviation narrative reports on a large scale. We formulate the problem as a multi-label classification task and compare three different types of algorithms. We consider Binary Relevance as our baseline method and further compare with another two state-of-the-art deep learning methods Seq2seq-LSTM and BERT. Our experiments show that the pre-trained BERT model achieves the best results by improving the recall rate for rare events. We further discuss several modifications to BERT based framework. 1) We study the influence of the attention mechanism. It shows that the attention module can further improve the results and can also provide good interpretability. 2) We explore different strategies for label embedding. The result validates that the proposed learned label embedding is better than original BERT embedding.

HIERARCHICAL CLASSIFICATION FOR AVIATION ACCIDENT REPORTS

4.1 Introduction

In air traffic management, identifying the safety risk and supporting the safety enhancements to mitigate risk is of great importance for the next generation of national air transportation systems. National Transportation Safety Board (NTSB) has investigated and collected more than 60000 aviation accident and incident reports with labeled sequences of accident events. It is of great importance to improve aviation safety by preventing repeat accidents. Engineers in NTSB have spent tremendous time recording past aviation accidents and understanding the crucial causes of past accidents. Since the accident is usually recorded in the raw text format, the root cause analysis requires experts with domain understanding. However, as more and more accident data is collected, such analyzing strategy becomes very inefficient.

With the development of Natural Language Processing (NLP) techniques, people started to design more efficient text mining tools to automatically identify all the accident events from the accident report using the historical accident report data with identified critical events. When new accident reports are available, the underlying events can be automatically identified by leveraging powerful supervised learning methods. There are many existing works on analyzing aviation incident reports. For example, Yelundur et al. proposed to apply the Bayesian logistic regression to classify four major events, which are Loss of separation, Deviation and ATC anomalies, Ground and landing events, and Loss of control (Yelundur *et al.*, 2016). Some other examples for applying the traditional NLP models include latent semantic analysis

and semantic lexicon learning (Abedin *et al.*, 2010; Robinson, 2018). However, due to the limited capacity of these traditional machine learning models, they failed to utilize a large amount of aviation corpus and cannot model the contextual relationship between words.

In recent years, deep learning models have been proposed to analyze accident report data. Wu et al. proposed an attention-based long short-term memory (LSTM) to identify the six most frequent impact factors from the incident reports recorded from Aviation Safety Reporting System (ASRS) (Dong *et al.*, 2021). Another work based on ASRS proposed identifying the 14 most important cause types such as physical factors, physical environment, proficiency, etc. Those works have achieved satisfying results on the limited (Abedin *et al.*, 2010). Zhao et al have proposed another method of identifying 58 major causes in the NTSB accident report using the LSTM model. Recently, Bidirectional Encoder Representations from Transformers (BERT) model has been proposed in the Natural Language Processing domain (?). The BERT model has also been applied to accident report modeling. For example, a BERT-based question answering system has been proposed based on the ASRS dataset(?). However, the major limitation of existing NLP models on the accident report data is that we can only identify coarse categories from accident reports. For example, it is very challenging for the algorithms to tell what is the exact physical factors of the accidents. Thus, existing works still require experts' effort to specify the details within those categories.

There are two major challenges if we want to identify the fine categories from the accident reports. We will use the accident reports from the National Transportation Safety Board (NTSB) as an example to illustrate the challenges as follows: 1) Multiple accident events for each accident report: in general, there can be multiple accident events corresponding to each accident report. The number of accident events in each

report is also typically unknown. 2) Lack of data for some fine-level categories: when we are trying to build a supervised learning model, we must ensure enough samples for all categories. For example, according to the event taxonomy defined by NTSB, there are 58 occurrence codes in the coarse-level labels and 1432 subject codes in the fine-level labels. In this case, since the coarse category is always a general abstract of many possible causes, it often accumulates enough samples for us to build a supervised learning model. However, as a subset of coarse categories, many fine categories might suffer a few-shot learning problem (Wang *et al.*, 2020). For example, there are 1191 fine categories have less than 100 accident reports and 602 fine categories have less than 10 samples in the NTSB data. 3) Data imbalance issue: as the size of fine categories is much larger than the coarse categories, it resembles the natural non-uniformly distribution in real-world scenarios. The fine categories always follow a long-tail distribution pattern as shown in Fig. 4.1. This phenomenon makes the model tend to be over-confident in the frequent categories and hard to recognize the rare categories. The two challenges mentioned here are commonly seen when dealing with a dataset with a large label size. In this paper, we propose to deal with the problems by utilizing the category taxonomy and view this problem as a hierarchical classification task.

In the NTSB accident report, the accident events may consist of a hierarchical structure. In most cases, the label hierarchy is usually organized as a directed acyclic graph. The parent nodes represent a high-level abstract of all its children. Fig. 4.2 presents an example of label taxonomy in the aviation domain defined by NTSB. The figure shows that flight control systems and electrical systems are coarse categories. They provide general information about what went wrong during the accident. To classify the fine-level category, such as flap control system or rudder control system, since the sample size for coarse-level categories is much larger than for fine-level



Figure 4.1: Label Statistics for 100 Most Frequent Fine-level Labels in NTSB

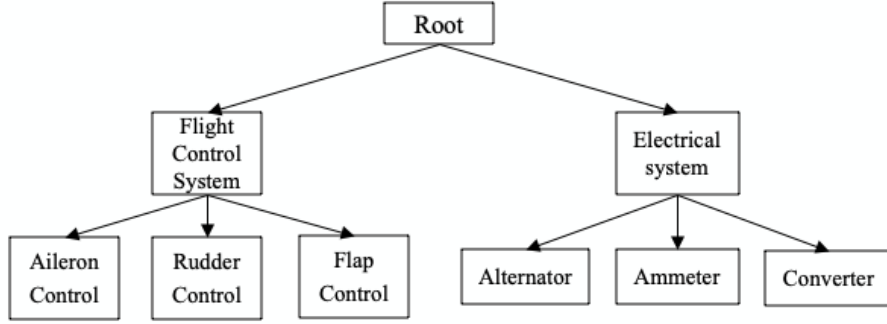


Figure 4.2: An Example of Label Taxonomy Defined by NTSB

categories, it is much easier to discriminate between the flight control system and the electrical system. On the other hand, it is challenging for the model directly recognize aileron control or rudder control due to the limited sample size. To tackle this challenge, we propose to bring the hierarchical information from the labels into the model instead of treating each category level independently.

In the literature, bringing the label hierarchy to the classification problem is often known as the hierarchical classification, which has shown the capacity to improve fine-level prediction accuracy in the case of unbalanced data and small sample sizes for

some categories. The hierarchical classification approaches are typically grouped into two directions: (1) Local approaches train multiple classifiers according to the label taxonomy, and the prediction follows a top-down manner. (2) Global approaches introduce a single classifier that can handle different levels of categories together (Silla and Freitas, 2011). The local approaches train category-wise models or level-wise models. Level-wise models follow a top-down manner where coarse categories are predicted first, and the fine level is predicted together with coarse-level information. It is much easier to capture local information from the data. However, such an approach often suffers from the error propagation problem as fine level prediction requires very accurate coarse-wise models. Global approaches treat the problem as a whole which might not capture the local information very well. However, it usually utilizes the hierarchical information much better by simultaneously optimizing for both coarse categories and fine categories together. For more detailed literature on general hierarchical classification, please refer to Section 4.2.

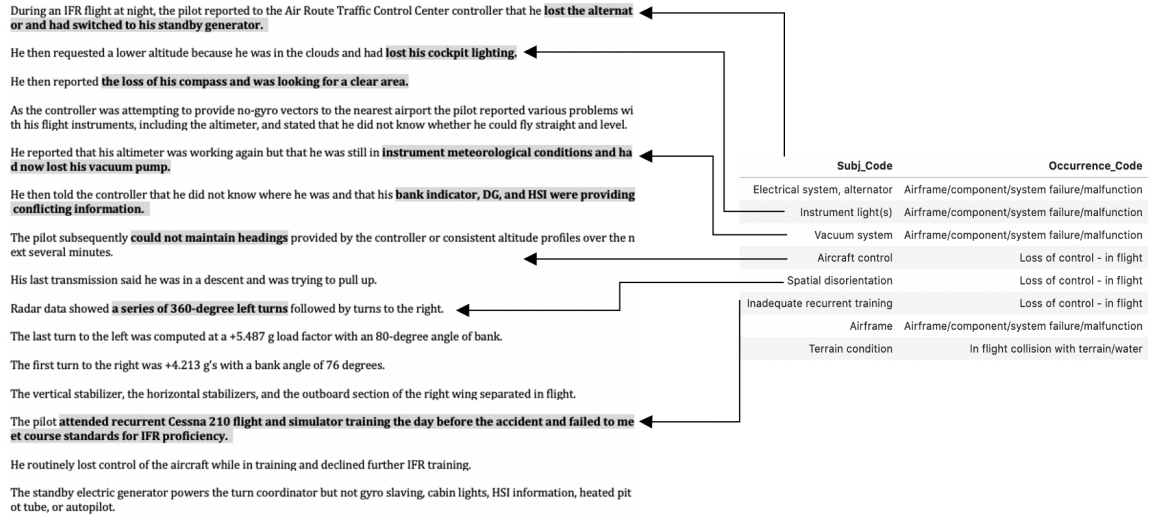


Figure 4.3: Aviation Accident Report to Event Labels

In conclusion, we propose to combine the hierarchical multi-label classification

approaches with a state-of-the-art BERT model to identify multiple events with fine-level categories from aviation accident reports. Our experiments are conducted based on the reports collected by NTSB. As shown in Fig. 4.3, the accident report is associated with multiple event labels. The occurrence code represents a coarse-level description of the accident, and the subject code represents a fine-level description. Given that the size of subject labels is too large, the traditional multi-label classification algorithm cannot achieve satisfying results due to data sparsity and data imbalance. This work aims to improve the fine-level classification by incorporating the event taxonomy into the hierarchical classification. The proposed method takes advantage of the state-of-the-art BERT model to handle the raw text data with the complex contextual word dependency. Furthermore, related to the hierarchical multi-label classification, the proposed method takes the advantage of both the local and global approaches for hierarchical classification by integrating hierarchical regularization, hierarchical attention, and hierarchical label distribution penalty to extract multiple events simultaneously from the accident report.

In summary, the major contributions of this paper are listed as follows:

1. We formulate the cause identification of aviation accident reports by combining the hierarchical classification and the state-of-the-art BERT model. By comparing with the BERT-based flat classification model, we demonstrate that the hierarchical information in the event taxonomy can benefit the fine-level classification accuracy.
2. We propose a novel hierarchical classification framework that benefits from both the local and global approaches. More specifically, our proposed hierarchical classification approach consists of three major components:
 - (a) Recursive regularization: Recursive regularization is added to encourage

the model parameter to be similar for the sibling nodes to the parent nodes.

- (b) Hierarchical attention: Hierarchical attention is added to guide the attention module of the fine-level model using the coarse-level information.
 - (c) Hierarchical Label distribution penalty: We propose a novel hierarchical label distribution penalty term. The component can further improve the classification accuracy on rare labels by dealing with the overconfidence issue on frequent labels.
3. We have applied the proposed method to the NTSB accident report data and have demonstrated the improvement of the multi-label classification accuracy, especially for rare events.

The paper is organized as follows. We first review existing works on text mining on narrative reports and state-of-art hierarchical classification methods in the section. 4.2. We further introduce some basic notations and the proposed methods in Section 4.3. The experiment results will be discussed in the next Section 4.4. Moreover, we finally conclude the paper in Section 4.5.

4.2 Literature Review

We would like to review the related literature in both the areas of accident report analysis and hierarchical classification. More specifically, from the application perspective, we will first give a brief overview of existing works on automating the information extraction for narrative reports in Section 4.2.1. From a methodology perspective, we will review the recent progress on hierarchical classification in Section 4.2.2.

4.2.1 Information Retrieval for Narrative Reports

It is necessary to provide an efficient prevention strategy when an accident happens by analyzing past accidents. Therefore, more researchers started developing tools to automatically extract information from narrative reports. The fundamental of the algorithms is the meaningful event taxonomy or impact factors defined by domain experts. There are various types of taxonomic structures defined early in the aviation domain. Three codifications are designed specifically for ASRS, which are the ASRS codification, the X-Form, and the Cinq-Demi (Ferryman *et al.*, 2006). The ASRS codification is currently used to describe the accident in the ASRS database, which presents an abstract of the accident through the "Chains of Events" and "Human performance consideration". There are nine coarse level categories from the ASRS codification: Aircraft, Events, Maintenance Factors, Etc. Under each course category, multiple fine categories are included. For example, around 180 subcategories are under the Aircraft category, and around 130 subcategories are under the Event category. Another widely used accident reporting system is NTSB. It designed three-level categories to describe the accident: Phase, Occurrence, and Subject. There are around 56 Phase categories, 58 Occurrence categories, and 1432 Subject categories. Based on those well-defined taxonomies, many researchers have started designing efficient information extraction frameworks to automate the accident reporting system in the aviation domain. Posse et al. developed algorithms to extract the information from the ASRS dataset according to the X-Form taxonomy (Posse *et al.*, 2005). They proposed a template-based framework to identify the related keywords in the reports, and 20 ASRS reports were selected for validation purposes. The potential issue with this method is to develop an exhaustive set of keywords and expressions. Later work by Muhammad et al. proposed an automatic keyword extraction strategy via semi-

supervised semantic lexicon learning and validated the efficiency of the model with all 140599 reports from 1998 to 2007 (Abedin *et al.*, 2010). To further improve the classification accuracy, many advanced machine learning methods are applied, such as Naive Bayes (Shi *et al.*, 2017), Bayesian logistic regression (Yelundur *et al.*, 2016), and latent semantic analysis (Robinson, 2018). It is worth noting that all the previous works are only conducted on small label sizes. As mentioned by Ludovic *et al.*, fine-grained investigation of sources of danger is very challenging (Tanguy *et al.*, 2016). It is very difficult for the experts to tell the difference between two closely related values at a detailed level. Existing works can only be implemented over the coarse level identification. In this work, we propose a framework trying to identify the fine-level categories by using the category taxonomy.

4.2.2 Hierarchical Classification

In many real-world applications, we can see the hierarchical category structure. For example, in the e-commerce system, products are classified into categories and subcategories (Karamanolakis *et al.*, 2020). In the healthcare system, the taxonomy of patients' symptoms helps the doctors diagnose much easier (Ruggero *et al.*, 2019). Motivated by those hierarchical representations, researchers start incorporating knowledge from the hierarchical taxonomy into traditional classification methods. Most existing hierarchical classification methods fall into two categories which are local approaches and global approaches (Silla and Freitas, 2011). Local approaches focus on building independent classifiers for each category or level from the taxonomy. It aims at extracting the most comprehensive local information from the samples. They usually follow a top-down prediction strategy according to the taxonomy (Koller and Sahami, 1997; Costa *et al.*, 2007; Kiritchenko *et al.*, 2004). One major problem of the local approaches is the error-propagation and the expensive computational cost.

Since multiple classifiers are developed independently, the model parameters increase rapidly, and the bias from different classifiers will accumulate. Different from local approaches, global approaches treat the problem as a whole. It builds a single model and predicts categories from different levels together (Cerri *et al.*, 2012; Cai and Hofmann, 2004; Gopal and Yang, 2013). However, global approaches might not capture the local information very well. They require fewer training resources and are more sensitive about the relationship across different levels among all categories. Considering all the pros and cons of the two methods, recent works have started building hybrid methods to take advantage of both approaches. Jonatas et al. proposed a hybrid approach through a recurrent neural network. The output of each layer corresponds to a level from the category taxonomy. By adding a global output layer as the final layer of the network, the proposed method optimizes local loss and global loss simultaneously. To further enhance the relationship between the input and labels, a hierarchical attention-based approach is proposed, which also fall into the category of hybrid approach (Huang *et al.*, 2019). By leveraging the hierarchical structure, the methods propagate the attention weight top-down. Inspired by the aforementioned methods, we proposed a novel hierarchical attention-based framework.

Unlike previous work, since our task suffers a long-tail distribution problem, we need to find an efficient solution to deal with the data sparsity and imbalance challenge. Recent studies have shown that we can leverage the hierarchical structure to deal with those problems. A new sampling strategy is proposed to deal with the imbalanced hierarchical dataset (Pereira *et al.*, 2021). Wu et al. proposed a rejection strategy based on the label taxonomy to identify the uncertain examples (Wu *et al.*, 2020). Other works focusing on the few-shot learning settings have also proved the value of label hierarchy. Considering the relationship among siblings belonging to the same level of the category, Xu et al. introduced Label Distribution Learning to solve

the small training set issue (Geng, 2016; Xu and Geng, 2019). Li et al. proposed a novel hierarchical feature extractor that achieves the state-of-the-art results for a few-shot learning task (Li *et al.*, 2019). Another two-stage few-shot learning approach is proposed in (Liu *et al.*, 2020). It argued that as the size of fine-level labels grows dramatically, the limited sample size for each category can hardly provide enough information for existing models to distinguish those. However, the sample size from the coarse-level categories is usually sufficient for a reliable supervised model. The paper develops a memory augmented hierarchical classification network based on those observations. Multi-layer perceptron and K-nearest-neighbor are developed for coarse prediction and fine prediction respectively due to their sample size. Following this line of research, we also design a two-stage approach and feed the reliable coarse-level predictions into our fine-level model for fine-tuning purposes. We guide the model training process with a recursive regularization term and a label distribution penalty to improve the prediction accuracy on few-shot categories.

4.3 Methodology

In this section, we will introduce the proposed model. More specifically, we will first review the related methods in Section 4.3.1. We will then introduce the proposed hierarchical multi-label classification BERT model in Section 4.3.2.

4.3.1 Review of Related Methodology

In this subsection, we will review the three related methods in handling the hierarchical labels including the hierarchical attention model in Section 4.3.1, recursive regularization in Section 4.3.1, and label distribution learning in Section 4.3.1.

Review of Hierarchical Attention

The attention mechanism is widely used in recent deep learning approaches. It helps us build the connection between labels and input. By adding the attention module, we can extract the most relevant component of the input to the label (Vaswani *et al.*, 2017). To leverage the label hierarchy information through the attention module, hierarchical attention is proposed by Huang *et al.* (2019). With the assumption that coarse level information can help us narrow down the relevant component for fine level prediction, we can propagate coarse level attention to fine level by building better semantic representation. Let $S^h \in \mathbb{R}^{|C^h| \times d_a}$ denote the embedding vector for h^{th} level of the category. $|C^h|$ represents the label size and d_a is a hyperparameter for the embedding dimension. The attention weight for h^{th} category level can be calculated as

$$W_{att}^h = \text{Softmax}(S^h \cdot O_h), \quad O_h = \tanh(W_s^h \cdot V_h^T), \quad (4.1)$$

where $V_h \in \mathbb{R}^{N \times 2u}$ is the semantic representation of the input and N is the length of the input. We further send V_h through a fully connected layer with weight matrix $W_s^h \in \mathbb{R}^{d_a \times 2u}$ to get $O_h \in \mathbb{R}^{d_a \times N}$. After getting the attention weight, we will further calculate the weighted text-category attention matrix $K^h \in \mathbb{R}^{|C^h| \times N}$ with the local prediction on h^{th} level of the category P_L^h

$$K^h = \text{Broadcast}(P_L^h) \otimes W_{att}^h \quad (4.2)$$

Here, the Broadcast operation is to make the shape of P_L^h compatible with attention matrix W_{att}^h so that we can conduct the element-wise multiplication $\otimes$. K_h is further averaged along the category dimension and then we can get the vector $\tilde{K}^h \in \mathbb{R}^N$ that shows the most relevant component from our input to the h^{th} category level. Finally, the updated semantic representation can be calculated with the weight attention

vector $\tilde{K}^h$

$$V_{h+1} = \omega^h \otimes \mathbf{V}, \quad \omega^h = \text{Broadcast}(\tilde{K}^h) \quad (4.3)$$

It is worth noting that the attention weight is propagating to the lower level through constructing a new semantic representation. In our framework, we simplify the calculation and let coarse-level attention directly influence the attention weight on the fine level.

Review of Recursive Regularization

Recursive regularization is first proposed by Gopal et al. as a global strategy to deal with hierarchical classification. It encourages the model parameters among sibling nodes in the label hierarchy to be similar (Gopal and Yang, 2013). Later work extends this method to a deep learning setting (Peng *et al.*, 2018). Given the observation that coarse-level categories usually have more training samples than fine-level categories. It is easier to get the optimal parameters for the coarse category. Thus, regularizing the children nodes to have similar parameters to its parent will help improve the fine-level classification even though we have fewer training samples on fine categories. To be formal, let w_i be the parameters in the final fully-connected layer for all categories. l_i^j represents the children of l_i . Then, the recursive regularization term can be added to the loss function as shown in Eq. (4.4).

$$\lambda(\mathbf{W}) = \sum_{l_i \in L} \sum_{l_i^{(j)}} \frac{1}{2} \|w_{l_i} - w_{l_i^j}\|^2 \quad (4.4)$$

Here, the final loss function will be $H + C\lambda(\mathbf{W})$, where H is the cross-entropy loss. Inspired by this formulation, our model also implements a similar regularization idea. However, since we mainly express the hierarchical information through the attention module, we will add this penalty by embedding vectors from different levels of categories.

Review of Label Distribution Learning

Another algorithm that can deal with the small training set issue through the label hierarchy is called label distribution learning (Xu and Geng, 2019; Geng, 2016). It is argued in the paper that the fine classifiers tends to over-fitting due to the small training size. We can leverage the relationship among siblings in the label hierarchy to get additional supervision information. As mentioned in the paper, Label Distribution Learning is a more general learning framework for single-label learning or multi-label learning. Under the label distribution learning setting, each instance x_i is associated with a label distribution $D_i = \{d_{x_i}^{y_1}, d_{x_i}^{y_2}, \dots, d_{x_i}^{y_n}\}$ where y_j is the j^{th} label. $d_{x_i}^{y_j} \in [0, 1]$ represents the degree of y_j to instance x_i where $\sum_j d_{x_i}^{y_j} = 1$. $d_{x_i}^{y_j}$ can also be represented as the form of conditional probability where $d_{x_i}^{y_j} = P(y_j|x_i)$.

To be more formal, Let $\mathcal{X} \in \mathbb{R}^q$ denotes the input space and $\mathcal{Y} = \{y_1, y_2, \dots, y_c\}$ is the complete set of labels. Given a training set $S = \{(x_1, D_1), (x_2, D_2), \dots, (x_n, D_n)\}$ Then the goal of the Label Distribution Learning becomes learn a conditional probability mass function $p(y|x)$ from S, where $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. Let the parameter θ to control the conditional probability $p(y|x, \theta)$. Then we need to find the optimal parameter set to generate similar distribution to D_i given the instance x . Multiple criteria can be used to evaluate the difference. Kullback-Leibler (KL) divergence is a frequently used measurement to describe the similarity between two distributions. Moreover, the optimization objective can be formulated as follows:

$$\theta^* = \arg \min_{\theta} \sum_i \sum_j (d_{x_i}^{y_j} \ln \frac{d_{x_i}^{y_j}}{p(y_j|x_i, \theta)}). \quad (4.5)$$

Under the hierarchical classification setting, the true label distribution D is usually not achievable. Moreover, there are multiple ways to construct the meaningful label distribution according to the label hierarchy. One possible way to get the label distribution mentioned by Xu and Geng (2019) is through leveraging the knowledge

of the Number of Common Nodes with the True Path (NCNTP). We propose a novel way to achieve label distribution in our work. We propose to use the Bayes rule to get the fine-level distribution through our prediction of the coarse-level distribution.

4.3.2 Proposed Model for Hierarchical Multi-label Classification

In this section, we will introduce the proposed Proposed Model for the Hierarchical Multi-label classification model and combine it with the BERT model to analyze the accident report data. Section 4.3.2 presents the overall problem definition. Section 4.3.2 presents the overall model architecture. Section 4.3.2 presents the architecture of coarse-level model. The three major components of the fine-level model are introduced in Section 4.3.2, 4.3.2, and 4.3.2, respectively. Finally, the loss function integrates all the component together is introduced in Section 4.3.2.

Problem definition

This paper aims to design an information extraction framework for aviation accident reports. As shown in Fig.4.3, there is a two-level description of the reports, which are called Occurrence and Subject. Occurrence is a coarse level event description containing 56 categories, while Subject is a fine level description with 1432 categories. Motivated by the label hierarchy, we want to build a coarse-to-fine hierarchical classification model to address the major challenges of the lack of enough samples in many fine-level categories.

Under the supervised learning setting, our problem can be formulated as follows: Given a set of training samples $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n, \mathbf{z}_n)\}_{n=1}^N \subseteq \mathcal{X} \times \mathcal{Y} \times \mathcal{Z}$. Since our input are accident reports, each $\mathbf{x}_n$ represents the input documents and $\mathcal{X}$ denotes the space of the natural language text where $\mathcal{X} = \{\omega_1, \omega_2, \dots, \omega_M\}$ can be viewed as a sequence of words. $\mathbf{y}_n = [y_1, y_2, \dots, y_{L_1}] \in \mathcal{Y} = \{0, 1\}^{L_1}$ denotes L_1 number of

coarse-level categories. $\mathbf{z}_n = [z_1, z_2, \dots, z_{L_2}] \in \mathcal{Z} = \{0, 1\}^{L_2}$ denotes L_2 number of fine-level categories. Our goal is to get an accurate fine-level prediction based on our observations.

$$\min_{\theta} \mathbb{E}_{\{\mathbf{x}, \mathbf{y}, \mathbf{z}\} \sim \mathcal{D}} - \log \mathbf{Pr}(\mathbf{z} | \mathbf{x}; \theta) \quad (4.6)$$

where θ denotes the model parameters. Based on the observation that coarse level categories usually have enough samples for the training purpose. In order to further improve the fine level prediction, the objective function can be break into two components as follows:

$$\mathbf{Pr}(\mathbf{z} | \mathbf{x}; \theta) = \sum_{\mathbf{y} \in \mathcal{Y}} \mathbf{Pr}(\mathbf{z} | \mathbf{x}, \mathbf{y}; \theta_f) \mathbf{Pr}(\mathbf{y} | \mathbf{x}; \theta_c), \quad (4.7)$$

where θ_f and θ_c represent the fine-level model parameters and coarse-level model parameters respectively. Thus, the problem can be transformed into a two-stage decision framework. Firstly, we need to design a coarse-level classifier. Then we need to design a fine-level classifier that can utilize the information predicted from the coarse model. It is worth noting that the proposed procedure is similar to the top-down prediction framework of the local approaches in hierarchical classification. To mitigate the error propagation issue, the information from the coarse models is not added to fine models directly as traditional local hierarchical classification approaches (Koller and Sahami, 1997). We build a hierarchical attention module to get better fine-level feature representation with the guidance of coarse-level information. We further optimized embedding vectors for coarse categories and fine categories together following a global approach framework.

Overview of the model architecture

Figure 4.4 presents an overview of the proposed framework. There are three major components of the framework: a coarse-level classifier, a fine-level feature extractor,

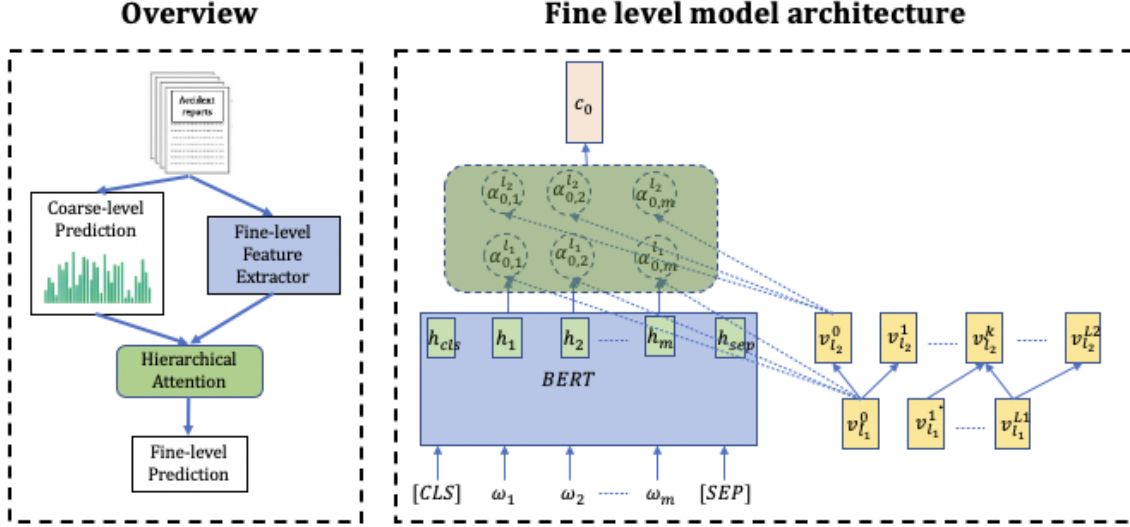


Figure 4.4: Overview of the Two Stage Hierarchical Attention.

and a hierarchical attention module. Our methods provide coarse-level prediction through fine-tuning on a pre-trained BERT model (?). BERT (Bidirectional Encoder Representation from Transformers) is one of the most powerful pre-trained deep language models that have achieved the state of the art results in many text mining tasks. A BERT-based classifier can provide satisfying results with a sufficient amount of coarse data. Therefore, it can further guide our fine classifiers. We first construct the deep representation for the accident reports with BERT for fine-level prediction. Then, the text features will feed into the hierarchical attention module and the coarse category distribution and make the fine-level prediction. Finally, we develop two-loss penalty terms according to the label distribution and label hierarchy to deal with the long-tail distribution over fine categories. In the following sections, we will mainly introduce the major components of the fine-level classifier.

BERT-based feature extractor

In order to get meaningful text representation, we apply BERT model to encode text into embedding space to extract the text embedding for the future use. As shown in Figure 4.4, the input document is a sequence of M words $\mathbf{x} = [\omega_1, \omega_2, \dots, \omega_M]$. We apply BERT model to get the semantic representation of the text data. The discrete word ω_i are first mapped into Key $\mathbf{k}_i$, Value $\mathbf{v}_i$ and Query $\mathbf{q}_i$ according to the self-attention mechanism. The contextual representation of the word $\mathbf{z}_i$ is computed as follows

$$\mathbf{z}_i = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}}{\sqrt{d_k}}\right) \mathbf{v} \quad (4.8)$$

The BERT model is built upon multiple self-attention layers and feed-forward connections. In the last layer, we can get the semantic representation of each word based on its correlation to all other words in the document. Here we will use $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_M] \in \mathbb{R}^{M \times u}$ to represent the final embedding vector calculated by BERT where u is the embedding dimension. To further regularize the overfitting issue, a drop-out layer is added to the final layer of the BERT model.

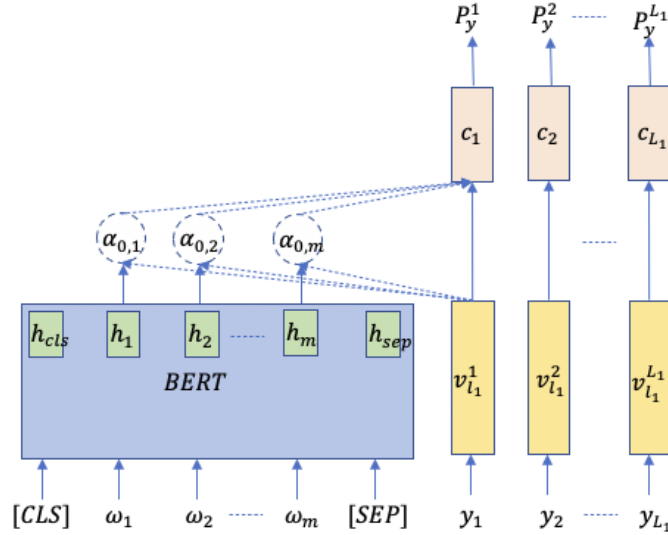


Figure 4.5: Coarse Level Model Architecture

Coarse-level model

We will then train a coarse-level multi-label classification model based on the extracted features from the BERT-based feature extractor to be used as the local approach to guide the fine-level multi-label classification. The coarse-level prediction model consists of the following components: a BERT-based feature extractor, an attention module for identifying the coarse-level relevance score, and the final BCE loss function for multi-label classification. Figure. 4.5 illustrates the architecture of the coarse-level model. Let $V_{l_1} = [\mathbf{v}_{l_1}^1, \mathbf{v}_{l_1}^2, \dots, \mathbf{v}_{l_1}^{L_1}] \in \mathbb{R}^{L_1 \times u}$ denote the coarse label embedding. We first calculate the relevance score between document and labels as follows:

$$e_{ij} = \mathbf{v}_{l_1}^i \cdot \mathbf{h}^j, \quad (4.9)$$

where e_{ij} is the relevance score between i^{th} label and j^{th} word in the document. The attention weight can be further calculated through a Softmax function.

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{i=1}^m \exp(e_{ij})} \quad (4.10)$$

With the attention weight, the contextual vector $\mathbf{c}_i \in \mathbb{R}^u$ for the i^{th} label can be calculated with the hidden vector as follows:

$$\mathbf{c}_i = \sum_{j=1}^m \alpha_{ij} \mathbf{h}_j. \quad (4.11)$$

Let $P_y = [P_y^1, P_y^2, \dots, P_y^{L_2}] \in \mathbb{R}^{L_2}$ denote the predicted probability for coarse-level labels which can be calculated through a fully connected layer and the Sigmoid function as follows:

$$P_y^i = \frac{1}{1 + \exp(\mathbf{w}_f \cdot \mathbf{c}_i + \mathbf{b}_f)}, \quad (4.12)$$

where $\mathbf{w}_f \in \mathbb{R}^u$ and $\mathbf{b}_f \in \mathbb{R}^u$ are the weight and bias vector for the final fully connected layer. Finally, we optimize the binary cross entropy loss between the true coarse label

and the predicted probability as follows:

$$\mathcal{L}_{BCE}(\mathbf{y}, P_{l_1}) = - \sum_{i=1}^{L_1} [y^i \log(P_{l_1}^i) + (1 - y^i) \log(1 - P_{l_1}^i)], \quad (4.13)$$

Hierarchical attention

After getting the embedding representation of the accident reports $\mathbf{H}$, we will make the fine-level prediction through the hierarchical attention module. We first calculate the coarse-level relevance score similar to coarse-level attention as follows:

$$K_{l_1} = V_{l_1} \cdot \mathbf{H}^T, \quad (4.14)$$

where $K_{l_1} \in \mathbb{R}^{L_1 \times M}$ is the relevance score matrix between the context and coarse labels under fine-level model. We further collect coarse category distribution $P_y \in \mathbb{R}^{L_1}$ from coarse classifier. The coarse category distribution helps us understand the importance of the attention weight from different categories. By applying an element-wise multiplication, we can get the weighted coarse-level relevance score matrix $O_{l_1} \in \mathbb{R}^{L_1 \times M}$ as follows:

$$O_{l_1} = \text{Broadcast}(P_y) \otimes K_{l_1}, \quad (4.15)$$

where $\text{Broadcast}()$ is an operation to expand P_y so that it has the same dimension as O_{l_1} . Our weighted attention matrix $O_{l_1} = (O_{l_1}^1, O_{l_1}^2, \dots, O_{l_1}^{L_1}) \in \mathbb{R}^{L_1 \times M}$ comprises L_1 vectors where each of them presents the category-wise word relevance within the input documents. In order to get a global attention weight score for each word, We further take a summation over the category dimension and apply the Softmax function to ensure all the computed weights sum to 1 as follows:

$$W_{l_1} = \text{Softmax}\left(\sum_{i=1}^{L_1} O_{l_1}^i\right), \quad (4.16)$$

where $W_{l_1} \in \mathbb{R}^M$ represents the global word attention weight for each input instance.

After calculating the coarse-level attention, we can further develop the fine-level attention weight. Let us first define fine category embedding $V_{l_2} \in \mathbb{R}^{L_2 \times u}$ and the attention weight matrix can be calculated as follows:

$$W_{l_2} = \text{Softmax}(V_{l_2} \cdot H^T), \quad (4.17)$$

where $W_{l_2} \in \mathbb{R}^{L_2 \times M}$ shows the attention weight for each fine-level category. To further incorporate the coarse level information, we build a linear combination between coarse-level attention and fine-level attention as follows:

$$W'_{l_2} = W_{l_2} + \lambda \cdot \text{Broadcast}(W_{l_1}), \quad (4.18)$$

where λ controls how much we want to leverage the coarse-level guidance. When $\lambda = 0$, the formulation becomes a flat classification framework. As we increase λ , the attention weight will rely more on the coarse level information.

After getting the the updated attention weights, we can further calculate the category-wise contextual representation as follows:

$$C = W'_{l_2} \cdot H, \quad (4.19)$$

where $C = (C^1, C^2, \dots, C^{L_2}) \in \mathbb{R}^{L_2 \times u}$ represents L_2 contextual representation for each category according its relationship to the input document.

Then, the fine-level prediction can be achieved through a fully connected layers and an activation function as follows:

$$P_{l_2} = \sigma(\phi([C \oplus V_{l_2}] \cdot W + \mathbf{b})), \quad (4.20)$$

where $W \in \mathbb{R}^{2u \times 1}$ and $\mathbf{b} \in \mathbb{R}^{L_2}$ represent the parameters belong to the final fully connected layer. $\oplus$ denotes the concatenation operation to concatenate C and V_{l_2} together. ϕ is the the non-linear activation function where we apply RELU in our context.

Finally, the proposed model will optimize the binary cross-entropy loss between the true label distribution and the predicted distribution as follows:

$$\mathcal{L}_{BCE}(\mathbf{z}, P_{l_2}) = - \sum_{j=1}^{L_2} [z^j \log(P_{l_2}^j) + (1 - z^j) \log(1 - P_{l_2}^j)]. \quad (4.21)$$

Recursive regularization

The fine-level prediction can be highly improved through the top-down prediction method given coarse-level information. However, as mentioned in the section.4.3.1, we still do not have enough training samples for many fine categories. That means we can not get optimal parameters for the corresponding embedding representation V_{l_2} . On the other hand, training samples for the coarse level are usually sufficient. Based on the assumption that the parent category should share similarities with the children category, we can optimize the fine-level embedding by letting it close to its connected coarse-level embedding. We incorporate it into our loss function as a similarity constraint. Let $\mathbf{v}_{l_1}^i \in \mathbb{R}^u, \forall i = 1, \dots, L_1$ represents the embedding vectors for coarse categories. For i^{th} coarse category, the connected fine-level category embedding are $\mathbf{v}_{l_2}^j, \forall j \in \pi(i)$. Inspired by the recursive regularization framework in Eq. (4.4), the constraint can be expressed as follows:

$$\mathcal{L}_{similarity} = \sum_{i=1}^{L_1} \sum_{j \in \pi(i)} \frac{1}{2} \|\mathbf{v}_{l_1}^i - \mathbf{v}_{l_2}^j\|. \quad (4.22)$$

Label distribution penalty

The recursive regularization term helps us build more connections between parent labels and children labels across the label hierarchy. To further improve the rare label identification, we can also use the sibling labels of the rare label. As introduced in section.4.3.1, a proper label distribution can add additional supervision information to the training process. For those rare labels, we can enrich the label space by

leveraging the frequent labels that share similar information to the rare labels. In this work, we propose to use the coarse label distribution to help build fine label distribution according to the strength of the connection between parents and children. Let $P_{l_1}^i, \forall i = 1, \dots, L_1$ be the coarse level probability calculated by the coarse classifiers. For j^{th} fine category, we use $w_j^i, \forall i = 1, \dots, L_1, \forall j = 1, \dots, L_2$ to denote its connection with the i^{th} coarse category. The strength of connection w_j^i is calculated as follows:

$$w_j^i = \frac{\#(y_i, z_j)}{\#(y_i)}, \quad (4.23)$$

where $\#(y_i, z_j)$ represents the number of co-occurrence of coarse category y_i and fine category z_j and $\#(y_i)$ represents number of occurrence of y_i . The probability of j^{th} fine category can be calculated as:

$$d^j = \frac{\sum_{i=1}^{L_1} P_{l_1}^i w_j^i}{\sum_{i=1}^{L_1} \sum_{j=1}^{L_2} P_{l_1}^i w_j^i}. \quad (4.24)$$

We further use KL divergence to measure the distance between the predicted fine distribution and the expected distribution calculated through coarse distribution as follows:

$$\mathcal{L}_{distribution} = \sum_j (d^j \ln \frac{d^j}{P_{l_2}^j}). \quad (4.25)$$

Loss function

After adding the distribution penalty, our final objective becomes:

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda_1 \mathcal{L}_{similarity} + \lambda_2 \mathcal{L}_{distribution}, \quad (4.26)$$

where $\mathcal{L}_{BCE}$ is the multi-label classification loss defined in Eq. (4.21), $\mathcal{L}_{similarity}$ is the recursive regularization defined in Eq. (4.22), $\mathcal{L}_{distribution}$ is the hierarchical label distribution loss, defined in Eq. (4.25), and λ_1 and λ_2 to control the impact of

recursive regularization and label distribution penalty. The best combination of those parameters is found through experiments. When $\lambda_1 = 0$, the framework becomes a local hierarchical classification approach and the fine-level embedding vectors are optimized relying on local information. As λ_1 increases, the framework becomes a global approach where the fine-level embedding is regularized by the coarse-level embedding. When $\lambda_2 = 0$, the label space is under a multi-label learning setting. As λ_2 increases, the framework becomes a label distribution learning setting. It helps enrich the label space and provides additional information for the rare labels.

4.4 Experiment Setup

In this section, we will introduce more details about the experiment design. An overview of the NTSB dataset will be given in Section 4.4.1. In Section 4.4.2, we will briefly talk about the implementing platform. We will further introduce the models for comparison and the comparison methods in section 4.4.4 and section 4.4.3. After introducing the experiment setup, the major results are discussed in Section 4.4.5 and Section 4.4.6.

4.4.1 Data Description

In this project, we mainly work with the aviation accident reports collected by NTSB. From 1982 to 2008, NTSB collects around 62569 accident reports following the same hierarchical labeling strategy. There are 56 Phase codes, 58 Occurrence codes, and 1432 Subject codes (NTS, 2008). The data is publicly accessible and stored in Microsoft Access database format. Figure 4.1 presents the frequency of different labels in the dataset, which suffers a long tail distribution. As shown in the figure, the most frequent subject labels are 19200, 20000, and 20200, with 30886, 24565, and 19231 samples. Those labels represent the most frequent critical events in the aviation

accidents which are *Terrain Condition*, *Weather Condition*, *Object Condition*. 250 Subject labels have more than 100 samples, 397 labels have more than 50 samples, and 602 labels have less than 10 samples. We select around 90% accident reports for training and use the rest of those for testing during the modeling process. We further calculate the strength of connection for Occurrence and Subject labels with the training data as mentioned in Section.4.3.2. Figure 4.6 gives a brief overview of the connection matrix with the most frequent 100 Subject labels. The X-axis shows the subject labels, and the y-axis shows the occurrence labels. The color of each entry represents the frequency of labels from two levels occurring together.

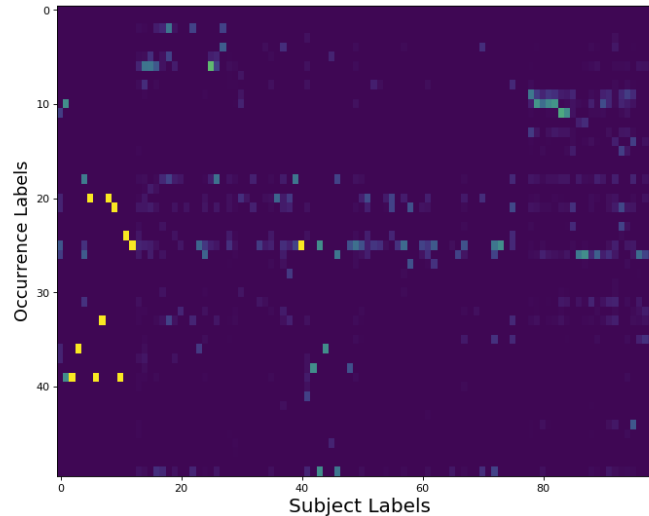


Figure 4.6: Connection Matrix Between Occurrence Labels and Subject Labels

4.4.2 Implementation Details

We apply the BERT model to extract the input documents’ semantic representation. The hidden size of the embedding vector is set to 768. To mitigate the overfitting issue, the dropout rate is set to 0.2. We further design the embedding dimension for the hierarchical attention same as the BERT model considering the consistency. We

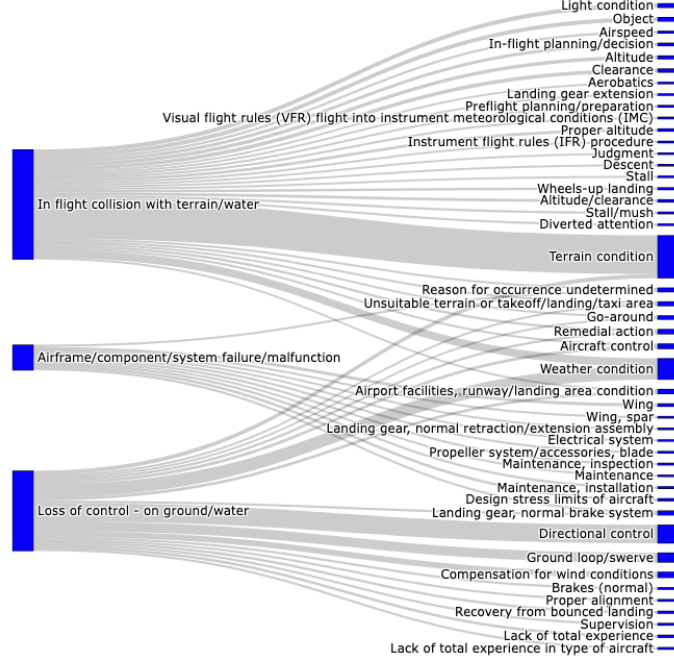


Figure 4.7: An Example of Label Taxonomy Defined by NTSB

test different λ to control the impact from coarse-level attention and get the best result when $\lambda = 2$. For different combination of the three-loss terms $\mathcal{L}_{BCE}$, $\mathcal{L}_{similarity}$ and $\mathcal{L}_{distribution}$, we finally set the corresponding weight to 1, $\lambda_1 = 0.01$ and $\lambda_2 = 0.001$. The experiments are all conducted on Pytorch 1.4.0 on a workstation with Intel(R) Core(TM) i7-5930K CPU @3.50 GHz CPU and accelerated with a single NVIDIA GTX 1080 Ti GPU.

4.4.3 Evaluation Metrics

In the hierarchical classification task, we apply the most common measurements used: precision, recall, and F1. Firstly, we will present a global score through micro averaging to compare the performance of different models. As shown in the following equations, since the micro measurements did not calculate label-wise accuracy, it

illustrates the overall accuracy for all samples.

$$\begin{aligned}
P_{micro} &= \frac{\sum_{j=1}^L tp_j}{\sum_{j=1}^L tp_j + fp_j}, \\
R_{micro} &= \frac{\sum_{j=1}^L tp_j}{\sum_{j=1}^L tp_j + fn_j}, \\
F_{1-micro} &= \frac{\sum_{j=1}^L 2tp_j}{\sum_{j=1}^L 2tp_j + fp_j + fn_j}
\end{aligned}$$

However, micro-f1 tends to favor categories that have large sample sizes when we have an imbalanced dataset. In our problem, the subject labels have a long tail distribution as shown in Fig. 4.1. Supervised learning is more capable to learn a good classifier for the frequent labels which will lead to a pretty high micro-f1 score even though all rare categories are not identified. Thus, we further evaluate the rare category identification using macro-averaging measurements. The macro-averaging calculates the label-wise accuracy and then takes the average of them as follows.

$$\begin{aligned}
P_{macro} &= \frac{1}{L} \sum_{j=1}^L \frac{tp_j}{tp_j + fp_j}, \\
R_{macro} &= \frac{1}{L} \sum_{j=1}^L \frac{tp_j}{tp_j + fn_j}, \\
F_{1-macro} &= \frac{1}{L} \sum_{j=1}^L \frac{2tp_j}{2tp_j + fp_j + fn_j}
\end{aligned}$$

To present a fair comparison over frequent categories and rare categories, we further divide all labels into different groups according to their sample size and calculate the group-wise marco averaging.

4.4.4 Models for Comparison

The proposed model is compared with several state-of-the-art models as baselines, which include:

- **BR-SVM** (Abedin *et al.*, 2010): Under the binary relevance schema, the support vector machine can be treated as a flat classification method for the multi-label task.
- **SGM** (Yang *et al.*, 2018): The sequence generation model is another flat classification approach that treats the multi-classification problem as a sequence generation task.
- **BERT** (Devlin *et al.*, 2018): Bert model provides us a solid baseline to evaluate the effectiveness of the proposed modules.

The variants of the proposed models are listed here:

- **HABERT**: Hierarchical attention-based BERT model is the fundamental structure of the proposed model. It leverages the label hierarchy through the hierarchical attention module.
- **HABERT-R**: HABERT-R is a variant of HABERT considering the similarity between parent and children nodes through recursive regularization.
- **HABERT-L**: HABERT-L is a variant of HABERT considering the fine category distribution through the coarse category distribution.
- **HABERT-RL**: HABERT-RL is a variant of HABERT using both recursive regularization and label distribution learning penalty.

4.4.5 Performance Evaluation

This section provides a global evaluation of the proposed methods through micro-averaging. It tells us how different models perform over the whole dataset regardless of the frequency of the categories. Table 4.1 presents the results of the experiments.

Table 4.1: Global Comparison for Different Models with Micro-precision, Micro-recall and Micro-f1

Models	Metrics		
	Micro-Precision	Micro-Recall	Micro-F1
BR-SVM	0.5356	0.3386	0.4608
SGM	0.5489	0.4179	0.4745
BERT	0.5408	0.4339	0.5069
HABERT	0.5805	0.4978	0.5360
HABERT-R	0.5861	0.4995	0.5394
HABERT-L	0.5257	0.5205	0.5231
HABERT-RL	0.5354	0.5345	0.5350

The results show that all the Bert-based models outperform SVM and LSTM models in terms of the F1 score. The original BERT model highly improved the recall rate since the pre-trained model reduces the demand for training samples. We can also validate the efficiency of the hierarchical attention module where HABERT has 6% improvements on f1 over BERT. It made a big improvement over both precision rate and recall rate. We further show the impact by adding the recursive regularization term. It has been shown that HABERT-R achieves the best precision score and F1 score overall models. It proves the efficiency of adding the additional supervision according to the label hierarchy. The f1 score drops a little bit by adding the label distribution penalty. However, the recall rate for both HABERT-L and HABERT-RL increases significantly over other models. The label distribution penalty can help the model identify more rare categories sacrificing some precision accuracy. Overall, the proposed hierarchical attention and recursive regularization can help us get better

fine-level classification. Moreover, label distribution learning can further improve rare category classification.

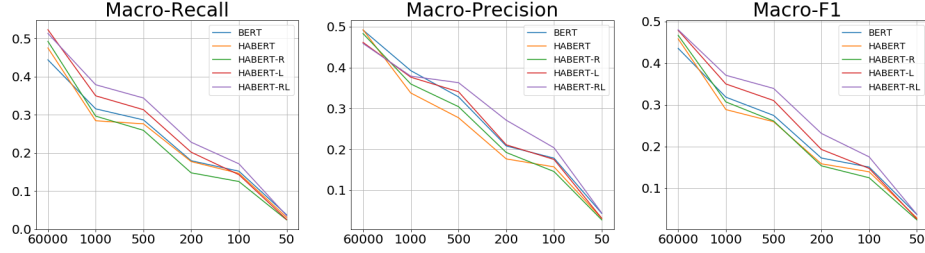


Figure 4.8: Model Comparison under Different Sample Size

Table 4.2: Macro F1 for Different Models under Different Sample Size

Models	Sample Size					
	1000-60000	500-1000	200-500	100-200	50-100	10-50
BERT	0.4356	0.3184	0.2746	0.1723	0.1506	0.0368
HABERT	0.4583	0.2885	0.2594	0.1584	0.1393	0.0285
HABERT-R	0.4668	0.3069	0.2612	0.1535	0.1249	0.0239
HABERT-L	0.4789	0.3500	0.3102	0.1929	0.1474	0.0261
HABERT-RL	0.4801	0.3708	0.3392	0.2313	0.1751	0.0370

4.4.6 Rare Category Identification

We further provide more experiment results to discuss the influence of sample size on the model performance. To ensure all categories have the same contribution, we will apply macro-averaging metrics in this section. We divide all data into 6 groups according to their sample size. After the processing, 46 categories have sample sizes from 1000 to 60000, and 43 categories have sample sizes from 500 to 1000, which can provide sufficient samples for training. 69 categories have sample sizes within 200 -

Table 4.3: Macro Precision for Different Models under Different Sample Size

Models	Sample Size					
	1000-60000	500-1000	200-500	100-200	50-100	10-50
BERT	0.4916	0.3923	0.3282	0.2080	0.1782	0.0431
HABERT	0.4923	0.3375	0.2770	0.1763	0.1568	0.0307
HABERT-R	0.4831	0.3596	0.3041	0.1920	0.1454	0.0271
HABERT-L	0.4615	0.3762	0.3407	0.2108	0.1750	0.0310
HABERT-RL	0.4588	0.3788	0.3629	0.2710	0.2036	0.0438

Table 4.4: Macro Recall for Different Models under Different Sample Size

Models	Sample Size					
	1000-60000	500-1000	200-500	100-200	50-100	10-50
BERT	0.4443	0.3157	0.2868	0.1797	0.1522	0.0378
HABERT	0.4754	0.2845	0.2764	0.1774	0.1465	0.0311
HABERT-R	0.4926	0.2966	0.2594	0.1479	0.1251	0.0245
HABERT-L	0.5232	0.3499	0.3135	0.2019	0.1429	0.0255
HABERT-RL	0.5136	0.3789	0.3442	0.2282	0.1715	0.0355

500, 92 categories have sample sizes within 100 - 200, and 146 categories have sample sizes within 50 - 100. Those categories provide fewer training samples that belong to the few-shot learning settings. Other categories with less than 50 samples are usually very challenging for deep learning methods. 399 categories have sample sizes within 10 - 50, and 602 categories have sample sizes less than 10.

Figure 4.8 presents macro-level comparison for different models under different sample size. We directly take the average of precision, recall, and f1 for categories

Table 4.5: Category-wise F1 Score Comparison under Few-shot Learning Setting

Category	Model				
	BERT	HABERT	HABERT-R	HABERT-L	HABERT-RL
IDENTIFICATION OF AIRCRAFT ON RADAR (28)	0.0	0.0	0.0	0.0	0.381
EXHAUST SYSTEM, CLAMP (39)	0.1053	0.0	0.1818	0.0	0.381
TRAFFIC ADVISORY (137)	0.0	0.0	0.0	0.0282	0.3546
WAKE TURBULENCE (99)	0.2513	0.0	0.1951	0.1748	0.3288
LUBRICATING SYSTEM, OIL FILTER/SCREEN (50)	0.1778	0.0	0.0	0.2105	0.3273
MAINTENANCE, LUBRICATION (70)	0.0408	0.0	0.0	0.0	0.3168
VACUUM SYSTEM (53)	0.2581	0.0	0.1818	0.1053	0.3077
ELEVATOR TRIM (72)	0.1488	0.1224	0.2278	0.2581	0.3368
LOSS OF TAIL ROTOR EFFECTIVENESS (85)	0.252	0.1356	0.1176	0.1176	0.3404
MIXTURE CONTROL, CABLE (53)	0.2717	0.1951	0.289	0.3077	0.3876
ENG ASSEMBLY, BLOWER /IMPELLER/INTEGRAL (77)	0.1835	0.1983	0.3055	0.313	0.3812
REMOVAL OF CONTROL/GUST LOCK (39)	0.2278	0.2278	0.2278	0.2535	0.3855
FLT CONTROL SYST, WING SPOILER SYSTEM (42)	0.2813	0.2951	0.2035	0.2105	0.45
ICE/FROST REMOVAL FROM AIRCRAFT (133)	0.2045	0.2105	0.1509	0.2443	0.3121
FUSELAGE, SEAT (86)	0.3173	0.3226	0.2148	0.3048	0.4202
LANDING GEAR, STEERING SYSTEM (128)	0.1928	0.2449	0.1224	0.2456	0.3415

belonging to each group. The x-axis shows the upper bound for the corresponding group, and the y-axis shows the metric’s value. The figure shows that the performance from all methods decreases as the sample size decreases. The proposed HABERT-RL

can improve the HABERT for all groups in terms of macro-f1. We can get a more accurate evaluation through Table 4.2, Table 4.4 and Table 4.3. For frequent categories, HABERT-RL has made 4.7% improvements over HABERT when the sample size is larger than 1000 in terms of macro-f1. Larger improvements are made when the sample size decreases. 28.5% improvement is made for groups 500 -1000. 30.8% improvement is made for groups 200 -500. 46.0% improvement is made for groups 100-200. For categories with less than 100 samples, none of the models can perform well, and the improvements are not very significant. 25.7% improvement is made for group 50 -100. 29.8% improvement is made for group 10-50. The most significant improvement our model made is within the categories with a small sample size from 50 - 200. It proves the effectiveness of the label distribution penalty.

We further present the F1 score for selected categories under a few-shot learning setting. As shown in Table 4.5, the left column are the names of each category and the corresponding sample size for each category is included in the parentheses. Those are categories that have sample sizes between 10 and 200. By applying the proposed penalty terms, HABERT-RL can achieve significant improvements over other methods. For rare categories such as IDENTIFICATION OF AIRCRAFT ON RADAR and TRAFFIC ADVISORY, only HABERT-RL is able to identify those from the reports. For categories that have slightly more samples such as ICE/FROST REMOVAL FROM AIRCRAFT and LANDING GEAR STEERING SYSTEM, HABERT-RL can also make further improvements over other methods.

4.5 Conclusion

This work aims to discuss the feasibility of building an information extraction system for fine-level events in the aviation domain. Our work leverages the event taxonomy defined by the domain expert from NTSB and converts the problem into a

hierarchical classification task. Since the fine-level events have a very large label size, the long-tail distribution of the data leads to the biggest challenge for implementing an accurate algorithm. To tackle this challenge, we propose to extend the state-of-the-art BERT model with a novel multi-label hierarchical classification model. First, we develop a hierarchical attention module to introduce coarse-level information into the fine category classification process. Our experiment validates the efficiency of this module, where we achieve 17% improvements over widely used BR-SVM in terms of micro-F1. The second component is to provide additional supervision to the fine-level parameters through recursive regularization. The experiment shows that the proposed HABERT-R achieves the best performance among all models. Finally, we discuss how to improve the classification accuracy for categories with small training samples. We propose a label distribution penalty term into the model and evaluate the rare category identification through the macro-F1 score. Our results show that the label distribution penalty can significantly improve the rare category classification accuracy. 46% improvement is made when the sample size is between 100 and 200. In summary, we are the first work discussing the possibility of extracting fine-level information in the aviation domain.

CONCLUSION

In this work, we aim at improving aviation safety through analyzing narrative reports on a large scale. In chapter 2, we develop hierarchical sequential models for failure events in an accident. The proposed method is built upon an Encoder-Decoder framework while incorporating hierarchical events by representing events with a hierarchical embedding. The experiment results have validated the efficiency of the method based on the NTSB database. While it achieves success compared to traditional methods, one big challenge to applying it directly to the aviation system is the lack of high-quality data. Conventionally, people manually identify those events from aviation accident reports which is highly subjective and inconsistent. Thus, we propose to develop an automatic event extraction framework using multi-label classification in Chapter 3. Specifically, we formulate the problem as a supervised learning task by classifying each narrative report into multiple event labels. We compare three state-of-the-art methods Binary Relevance Support Vector Machine, Seq2seq Learning, and BERT. Furthermore, we propose three variations on BERT based framework. 1) We add the attention module to create a contextual representation of the reports according to different labels. 2) We compare three types of event embedding initialization and propose to adjust BERT-based embedding through event sequence modeling. 3) We consider adapting the BERT model to the aviation domain by training it with aviation text data. With extensive experiments, we provide a solid benchmark for information extraction in the aviation domain. As an extension of the second work, we target extracting more specific information from the narrative reports in Chapter 4. Different from previous work, the label size is increased from 58 to 1432. The problem

becomes more challenging since more labels have a minimal sample size. We propose two directions to tackle this extreme multi-label classification problem. Firstly, we revisited event embedding representation. By letting similar labels have close embedding representation, we can enlarge the sample size for rare events. Secondly, we borrow the hierarchical decoder structure from Chapter 2. We believe we can further improve the recall rate on the Subject level by incorporating the relationship among different abstract levels.

REFERENCES

- “National transportation safety board, aviation coding manual”, <https://app.nts.gov/avdata/codman.pdf> (2008).
- Abedin, M., V. Ng and L. Khan, “Cause identification from aviation safety incident reports via weakly supervised semantic lexicon construction”, *Journal of Artificial Intelligence Research* **38**, 569–631 (2010).
- Ai, Q., Y. Zhang, K. Bi, X. Chen and W. B. Croft, “Learning a hierarchical embedding model for personalized product search”, in “Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval”, pp. 645–654 (2017).
- Akata, Z., F. Perronnin, Z. Harchaoui and C. Schmid, “Label-embedding for attribute-based classification”, in “Proceedings of the IEEE conference on computer vision and pattern recognition”, pp. 819–826 (2013).
- Bahdanau, D., K. Cho and Y. Bengio, “Neural machine translation by jointly learning to align and translate”, *arXiv preprint arXiv:1409.0473* (2014).
- Bengio, Y., R. Ducharme, P. Vincent and C. Jauvin, “A neural probabilistic language model”, *Journal of machine learning research* **3**, Feb, 1137–1155 (2003).
- Cai, L. and T. Hofmann, “Hierarchical document categorization with support vector machines”, in “Proceedings of the thirteenth ACM international conference on Information and knowledge management”, pp. 78–87 (2004).
- Cerri, R., R. C. Barros and A. C. de Carvalho, “A genetic algorithm for hierarchical multi-label classification”, in “Proceedings of the 27th annual ACM symposium on applied computing”, pp. 250–255 (2012).
- Chalkidis, I., M. Fergadiotis, P. Malakasiotis and I. Androutsopoulos, “Large-scale multi-label text classification on eu legislation”, *arXiv preprint arXiv:1906.02192* (2019).
- Chang, W.-C., H.-F. Yu, K. Zhong, Y. Yang and I. Dhillon, “X-bert: extreme multi-label text classification with using bidirectional encoder representations from transformers”, *arXiv preprint arXiv:1905.02331* (2019).
- Chu, B., V. Madhavan, O. Beijbom, J. Hoffman and T. Darrell, “Best practices for fine-tuning visual classifiers to new domains”, in “European conference on computer vision”, pp. 435–442 (Springer, 2016).
- Collobert, R., J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu and P. Kuksa, “Natural language processing (almost) from scratch”, *Journal of machine learning research* **12**, Aug, 2493–2537 (2011).
- Connell, L., “Incident reporting: The nasa aviation safety reporting system”, *GSE Today* pp. 66–68 (1999).

- Costa, E. P., A. C. Lorena, A. C. Carvalho, A. A. Freitas and N. Holden, “Comparing several approaches for hierarchical classification of proteins with decision trees”, in “Brazilian symposium on bioinformatics”, pp. 126–137 (Springer, 2007).
- Dembczynski, K., W. Cheng and E. Hüllermeier, “Bayes optimal multilabel classification via probabilistic classifier chains”, in “ICML”, (2010).
- Desell, T., S. Clachar, J. Higgins and B. Wild, “Evolving neural network weights for time-series prediction of general aviation flight data”, in “International Conference on Parallel Problem Solving from Nature”, pp. 771–781 (Springer, 2014).
- Devlin, J., M.-W. Chang, K. Lee and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding”, arXiv preprint arXiv:1810.04805 (2018).
- Ding, X., Y. Zhang, T. Liu and J. Duan, “Knowledge-driven event embedding for stock prediction”, in “Proceedings of coling 2016, the 26th international conference on computational linguistics: Technical papers”, pp. 2133–2142 (2016).
- Dong, T., Q. Yang, N. Ebadi, X. R. Luo and P. Rad, “Identifying incident causal factors to improve aviation transportation safety: Proposing a deep learning approach”, *Journal of Advanced Transportation* **2021** (2021).
- Du, N., H. Dai, R. Trivedi, U. Upadhyay, M. Gomez-Rodriguez and L. Song, “Recurrent marked temporal point processes: Embedding event history to vector”, in “Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining”, pp. 1555–1564 (2016).
- Ericson, C. A. and C. Ll, “Fault tree analysis”, in “System Safety Conference, Orlando, Florida”, vol. 1, pp. 1–9 (1999).
- FAA, M., “Nextgen implementation plan”, Washington, DC, Available: http://www.faa.gov/nextgen/media/ng2011implementation_plan.pdf (accessed March 18, 2011) (2013).
- Ferryman, T. A., C. Posse, L. J. Rosenthal, A. N. Srivastava and I. C. Statler, “What happened, and why: Toward an understanding of human error based on automated analyses of incident reports—vol. ii”, Tech. rep., NASA/TP–2006-213490 (2006).
- Galassi, A., M. Lippi and P. Torroni, “Attention in natural language processing”, *IEEE Transactions on Neural Networks and Learning Systems* (2020).
- Geng, X., “Label distribution learning”, *IEEE Transactions on Knowledge and Data Engineering* **28**, 7, 1734–1748 (2016).
- Gilchrist, W., “Modelling failure modes and effects analysis”, *International Journal of Quality & Reliability Management* (1993).
- Goodfellow, I., Y. Bengio and A. Courville, *Deep learning* (MIT press, 2016).

- Goodfellow, I., J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio, “Generative adversarial nets”, in “Advances in neural information processing systems”, pp. 2672–2680 (2014).
- Gopal, S. and Y. Yang, “Recursive regularization for large-scale classification with hierarchical and graphical dependencies”, in “Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining”, pp. 257–265 (2013).
- Granroth-Wilding, M. and S. Clark, “What happens next? event prediction using a compositional neural network model”, (2016).
- Han, J. and Y. Fu, “Mining multiple-level association rules in large databases”, *IEEE Transactions on knowledge and data engineering* **11**, 5, 798–805 (1999).
- Hänninen, M., “Bayesian networks for maritime traffic accident prevention: benefits and challenges”, *Accident Analysis & Prevention* **73**, 305–312 (2014).
- He, K., X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition”, in “Proceedings of the IEEE conference on computer vision and pattern recognition”, pp. 770–778 (2016).
- Hill, F., K. Cho and A. Korhonen, “Learning distributed representations of sentences from unlabelled data”, in “Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies”, pp. 1367–1377 (Association for Computational Linguistics, San Diego, California, 2016), URL <https://www.aclweb.org/anthology/N16-1162>.
- Hochreiter, S. and J. Schmidhuber, “Long short-term memory”, *Neural computation* **9**, 8, 1735–1780 (1997).
- Hong, C., J. Yu, J. Wan, D. Tao and M. Wang, “Multimodal deep autoencoder for human pose recovery”, *IEEE Transactions on Image Processing* **24**, 12, 5659–5670 (2015).
- Howard, J. and S. Ruder, “Universal language model fine-tuning for text classification”, arXiv preprint arXiv:1801.06146 (2018).
- Hu, L., J. Li, L. Nie, X.-L. Li and C. Shao, “What happens next? future subevent prediction using contextual hierarchical lstm”, in “Thirty-First AAAI Conference on Artificial Intelligence”, (2017).
- Huang, W., E. Chen, Q. Liu, Y. Chen, Z. Huang, Y. Liu, Z. Zhao, D. Zhang and S. Wang, “Hierarchical multi-label text classification: An attention-based recurrent network approach”, in “Proceedings of the 28th ACM international conference on information and knowledge management”, pp. 1051–1060 (2019).
- Jing, L.-P., H.-K. Huang and H.-B. Shi, “Improved feature selection approach tfidf in text mining”, in “Proceedings. International Conference on Machine Learning and Cybernetics”, vol. 2, pp. 944–946 (IEEE, 2002).

- Karamanolakis, G., J. Ma and X. L. Dong, “Txtract: Taxonomy-aware knowledge extraction for thousands of product categories”, arXiv preprint arXiv:2004.13852 (2020).
- Kim, C. and J. Kim, “A recommendation algorithm using multi-level association rules”, in “Proceedings IEEE/WIC International Conference on Web Intelligence (WI 2003)”, pp. 524–527 (IEEE, 2003).
- Kiritchenko, S., S. Matwin, A. F. Famili *et al.*, “Hierarchical text categorization as a tool of associating genes with gene ontology codes”, in “European workshop on data mining and text mining in bioinformatics”, pp. 30–34 (2004).
- Koller, D. and M. Sahami, “Hierarchically classifying documents using very few words”, Tech. rep., Stanford InfoLab (1997).
- Kumaran, G. and J. Allan, “Text classification and named entities for new event detection”, in “Proceedings of the 27th annual international ACM SIGIR conference on Research and development in information retrieval”, pp. 297–304 (2004).
- Lee, J., W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So and J. Kang, “Biobert: a pre-trained biomedical language representation model for biomedical text mining”, *Bioinformatics* **36**, 4, 1234–1240 (2020).
- Leveson, N., “A new accident model for engineering safer systems”, *Safety science* **42**, 4, 237–270 (2004).
- Li, A., T. Luo, Z. Lu, T. Xiang and L. Wang, “Large-scale few-shot learning: Knowledge transfer with class hierarchy”, in “Proceedings of the IEEE/CVF conference on computer vision and pattern recognition”, pp. 7212–7220 (2019).
- Li, Y., N. Du and S. Bengio, “Time-dependent representation for neural event sequence prediction”, arXiv preprint arXiv:1708.00065 (2017).
- Liu, J., W.-C. Chang, Y. Wu and Y. Yang, “Deep learning for extreme multi-label text classification”, in “Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval”, pp. 115–124 (2017).
- Liu, L., T. Zhou, G. Long, J. Jiang and C. Zhang, “Many-class few-shot learning on multi-granularity class hierarchy”, *IEEE Transactions on Knowledge and Data Engineering* (2020).
- Luong, M.-T., H. Pham and C. D. Manning, “Effective approaches to attention-based neural machine translation”, arXiv preprint arXiv:1508.04025 (2015).
- Manning, C. and H. Schutze, *Foundations of statistical natural language processing* (MIT press, 1999).
- McCormick, T., C. Rudin and D. Madigan, “A hierarchical model for association rule mining of sequential events: An approach to automated medical symptom prediction”, (2011).

- Nallapati, R., B. Zhou, C. Gulcehre, B. Xiang *et al.*, “Abstractive text summarization using sequence-to-sequence rnns and beyond”, arXiv preprint arXiv:1602.06023 (2016).
- Nam, J., J. Kim, E. L. Mencía, I. Gurevych and J. Fürnkranz, “Large-scale multi-label text classification—revisiting neural networks”, in “Joint european conference on machine learning and knowledge discovery in databases”, pp. 437–452 (Springer, 2014).
- O’Madadhain, J., J. Hutchins and P. Smyth, “Prediction and ranking algorithms for event-based network data”, ACM SIGKDD explorations newsletter **7**, 2, 23–30 (2005).
- Peng, H., J. Li, Y. He, Y. Liu, M. Bao, L. Wang, Y. Song and Q. Yang, “Large-scale hierarchical text classification with recursively regularized deep graph-cnn”, in “Proceedings of the 2018 world wide web conference”, pp. 1063–1072 (2018).
- Pereira, R. M., Y. M. Costa and C. N. Silla Jr, “Toward hierarchical classification of imbalanced data using random resampling algorithms”, Information Sciences **578**, 344–363 (2021).
- Perrow, C., “1984: Normal accidents: living with high-risk technologies. new york: Basic books”, (1984).
- Peters, M. E., M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee and L. Zettlemoyer, “Deep contextualized word representations”, arXiv preprint arXiv:1802.05365 (2018).
- Posse, C., B. Matzke, C. Anderson, A. Brothers, M. Matzke and T. Ferryman, “Extracting information from narratives: An application to aviation safety reports”, in “2005 IEEE Aerospace Conference”, pp. 3678–3690 (IEEE, 2005).
- Radford, A., K. Narasimhan, T. Salimans and I. Sutskever, “Improving language understanding by generative pre-training”, (2018).
- Rao, A. H. and K. Marais, “Identifying high-risk occurrence chains in helicopter operations from accident data”, in “15th AIAA Aviation Technology, Integration, and Operations Conference”, p. 2848 (2015).
- Rao, A. H. and K. Marais, “High risk occurrence chains in helicopter accidents”, Reliability Engineering & System Safety **170**, 83–98 (2018).
- Rao, A. H. and K. Marais, “A state-based approach to modeling general aviation accidents”, Reliability Engineering & System Safety **193**, 106670 (2020).
- Reason, J., “Managing the risks of organizational accidents, isbn 1 84014 104 2”, (1997).
- Rezaie Moghaddam, F., S. Afandizadeh and M. Ziyadi, “Prediction of accident severity using artificial neural networks”, International Journal of Civil Engineering **9**, 1, 41–48 (2011).

- Robinson, S. D., “Multi-label classification of contributing causal factors in self-reported safety narratives”, *Safety* **4**, 3, 30 (2018).
- Rudin, C., B. Letham, A. Salieb-Aouissi, E. Kogan and D. Madigan, “Sequential event prediction with association rules”, in “Proceedings of the 24th annual conference on learning theory”, pp. 615–634 (2011).
- Ruggero, C. J., R. Kotov, C. J. Hopwood, M. First, L. A. Clark, A. E. Skodol, S. N. Mullins-Sweatt, C. J. Patrick, B. Bach, D. C. Cicero *et al.*, “Integrating the hierarchical taxonomy of psychopathology (hitop) into clinical practice.”, *Journal of consulting and clinical psychology* **87**, 12, 1069 (2019).
- Rumelhart, D. E., G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors”, *nature* **323**, 6088, 533–536 (1986).
- Sahoo, R. K., A. J. Oliner, I. Rish, M. Gupta, J. E. Moreira, S. Ma, R. Vilalta and A. Sivasubramaniam, “Critical event prediction for proactive management in large-scale computer clusters”, in “Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining”, pp. 426–435 (2003).
- Schuster, W. and W. Ochieng, “Performance requirements of future trajectory prediction and conflict detection and resolution tools within sesar and nextgen: Framework for the derivation and discussion”, *Journal of Air Transport Management* **35**, 92–101 (2014).
- Schütze, H., C. D. Manning and P. Raghavan, *Introduction to information retrieval*, vol. 39 (Cambridge University Press Cambridge, 2008).
- Sennrich, R., B. Haddow and A. Birch, “Neural machine translation of rare words with subword units”, *arXiv preprint arXiv:1508.07909* (2015).
- Shi, D., J. Guan, J. Zurada and A. Manikas, “A data-mining approach to identification of risk factors in safety management systems”, *Journal of management information systems* **34**, 4, 1054–1081 (2017).
- Silla, C. N. and A. A. Freitas, “A survey of hierarchical classification across different application domains”, *Data Mining and Knowledge Discovery* **22**, 1, 31–72 (2011).
- Srinivasan, P., V. Nagarajan and S. Mahadevan, “Mining and classifying aviation accident reports”, in “AIAA Aviation 2019 Forum”, p. 2938 (2019).
- Sriram, B., D. Fuhry, E. Demir, H. Ferhatosmanoglu and M. Demirbas, “Short text classification in twitter to improve information filtering”, in “Proceedings of the 33rd international ACM SIGIR conference on Research and development in information retrieval”, pp. 841–842 (2010).
- Srivastava, N., G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting”, *The journal of machine learning research* **15**, 1, 1929–1958 (2014).

- Steven, B., “Nltk: the natural language toolkit in proceedings of the acl 2004 on interactive poster and demonstration sessions”, in “Association for Computational Linguistics”, p. 31 (2004).
- Sun, C., X. Qiu, Y. Xu and X. Huang, “How to fine-tune bert for text classification?”, in “China National Conference on Chinese Computational Linguistics”, pp. 194–206 (Springer, 2019).
- Tanguy, L., N. Tulechki, A. Urieli, E. Hermann and C. Raynal, “Natural language processing for aviation safety reports: From classification to interactive analysis”, *Computers in Industry* **78**, 80–95 (2016).
- Tsoumakas, G., I. Katakis and I. Vlahavas, “Random k-labelsets for multilabel classification”, *IEEE transactions on knowledge and data engineering* **23**, 7, 1079–1089 (2010).
- Van der Maaten, L. and G. Hinton, “Visualizing data using t-sne.”, *Journal of machine learning research* **9**, 11 (2008).
- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention is all you need”, in “Advances in neural information processing systems”, pp. 5998–6008 (2017).
- Wang, S., J. Tang, Y. Wang and H. Liu, “Exploring implicit hierarchical structures for recommender systems.”, in “IJCAI”, pp. 1813–1819 (2015).
- Wang, Y., Q. Yao, J. T. Kwok and L. M. Ni, “Generalizing from a few examples: A survey on few-shot learning”, *ACM computing surveys (csur)* **53**, 3, 1–34 (2020).
- Wiegmann, D. A. and S. A. Shappell, *A human error approach to aviation accident analysis: The human factors analysis and classification system* (Routledge, 2017).
- Wolf, T., L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Huggingface’s transformers: State-of-the-art natural language processing”, *arXiv preprint arXiv:1910.03771* (2019).
- Wu, T.-Y., P. Morgado, P. Wang, C.-H. Ho and N. Vasconcelos, “Solving long-tailed recognition with deep realistic taxonomic classifier”, in “European Conference on Computer Vision”, pp. 171–189 (Springer, 2020).
- Xu, C. and X. Geng, “Hierarchical classification based on label distribution learning”, in “Proceedings of the AAAI Conference on Artificial Intelligence”, vol. 33, pp. 5533–5540 (2019).
- Yang, P., X. Sun, W. Li, S. Ma, W. Wu and H. Wang, “Sgm: sequence generation model for multi-label classification”, *arXiv preprint arXiv:1806.04822* (2018).
- Yelundur, A., C. Giannella, K. Megerdooimian and C. Pfeifer, “Event classification in foreign language aviation reports”, *International Journal of Knowledge Engineering and Data Mining* **4**, 1, 54–73 (2016).

- You, Y., J. Li, S. Reddi, J. Hseu, S. Kumar, S. Bhojanapalli, X. Song, J. Demmel, K. Keutzer and C.-J. Hsieh, “Large batch optimization for deep learning: Training bert in 76 minutes”, arXiv preprint arXiv:1904.00962 (2019).
- Zhang, J., L. Zi, Y. Hou, D. Deng, W. Jiang and M. Wang, “A c-bilstm approach to classify construction accident reports”, *Applied Sciences* **10**, 17, 5754 (2020).
- Zhang, M.-L., Y.-K. Li, X.-Y. Liu and X. Geng, “Binary relevance for multi-label learning: an overview”, *Frontiers of Computer Science* **12**, 2, 191–202 (2018).
- Zhang, X. and S. Mahadevan, “Bayesian neural networks for flight trajectory prediction and safety assessment”, *Decision Support Systems* **131**, 113246 (2020).
- Zhang, X. and S. Mahadevan, “Bayesian network modeling of accident investigation reports for aviation safety assessment”, *Reliability Engineering & System Safety* **209**, 107371 (2021).
- Zhang, X., P. Srinivasan and S. Mahadevan, “Sequential deep learning from ntsb reports for aviation safety prognosis”, *Safety science* **142**, 105390 (2021).
- Zhao, X., H. Yan and Y. Liu, “Hierarchical tree-based sequential event prediction with application in the aviation accident report”, in “2021 IEEE 37th International Conference on Data Engineering (ICDE)”, pp. 1925–1930 (IEEE, 2021).
- Zhong, B., X. Pan, P. E. Love, L. Ding and W. Fang, “Deep learning and network analysis: Classifying and visualizing accident narratives in construction”, *Automation in Construction* **113**, 103089 (2020).
- Zhu, S. and Y. Xie, “Crime event embedding with unsupervised feature selection”, in “ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)”, pp. 3922–3926 (IEEE, 2019).

APPENDIX A
NTSB DATA

Table A.1: Codes for Occurrence Defined by NTSB

Code	Meaning	Code	Meaning
100	Abrupt maneuver	197	Gear not retracted
132	Rotor failure/malfunction	198	Gear retraction on ground
171	Fire	200	Hard landing
191	Main gear collapsed	210	Hazardous materials leak/spill
195	Other gear collapsed	230	In flight collision with terrain/water
220	In flight collision with object	231	Wheels down landing in water
250	Loss of control - in flight	232	Wheels up landing
300	Nose over	240	In flight encounter with weather
351	Loss of engine power (total) mechanical failure/malfunction	260	Loss of control - on ground/water
370	Propeller/rotor contact to person	270	Midair collision
430	Miscellaneous/other	271	Collision between aircraft (other than midair)
110	Altitude deviation uncontrolled	280	Near collision between aircraft
120	Cargo shift	290	Nose down
130	Airframe/component/system failure/malfunction	310	On ground/water collision with object
131	Propeller failure/malfunction	320	On ground/water collision with terrain/water
140	Decompression	330	On ground/water encounter with weather
150	Ditching	340	Overrun
160	Dragged wing rotor pod float or tail/skid	350	Loss of engine power
170	Fire/explosion	352	Loss of engine power (partial) mechanical failure/malfunction
172	Explosion	353	Loss of engine power(total) nonmechanical
180	Forced landing	354	Loss of engine power (partial) - nonmechanical
190	Gear collapsed	355	Engine tearaway
192	Nose gear collapsed	360	Propeller blast or jet exhaust/suction
193	Tail gear collapsed	380	Roll over
194	Complete gear collapsed	390	Undershoot
196	Gear not extended	400	Undetermined
420	Missing aircraft	410	Vortex turbulence encountered
0	nan	999	End

ProQuest Number: 29257554

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by ProQuest LLC (2022).

Copyright of the Dissertation is held by the Author unless otherwise noted.

This work may be used in accordance with the terms of the Creative Commons license or other rights statement, as indicated in the copyright statement or in the metadata associated with this work. Unless otherwise specified in the copyright statement or the metadata, all rights are reserved by the copyright holder.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

Microform Edition where available © ProQuest LLC. No reproduction or digitization of the Microform Edition is authorized without permission of ProQuest LLC.

ProQuest LLC
789 East Eisenhower Parkway
P.O. Box 1346
Ann Arbor, MI 48106 - 1346 USA